
Criando agentes de atendimento profissionais no Whatsapp

Asimov Academy

ASIMOV

Conteúdo

1. Introdução e Arquitetura de Agentes de IA	9
O que vamos construir?	9
A Stack Tecnológica	9
Hospedagem: Railway	9
Orquestração: n8n	10
Integração: API Oficial da Meta	10
Bancos de Dados e Memória	10
O Fluxo da Informação	10
02. Configurando sua Infraestrutura na Railway	12
Escolhendo o Plano Ideal	12
O Poder dos Templates	13
Passo a Passo da Instalação:	13
Configuração Inicial do n8n	14
03. Preparação e Pré-requisitos da API Oficial (Meta)	15
Por que a Meta exige tantos pré-requisitos?	15
Etapa inicial: criando a conta base	15
Criando um Gmail para a operação	16
Criando a conta no Facebook	16
O aquecimento da conta	16
O que é aquecer a conta?	16
Quanto tempo aquecer?	17
Criando e aquecendo a página do Facebook	17
Preenchendo a página corretamente	17
Verificação com número de WhatsApp	17
E se eu quiser fazer tudo mais rápido?	17
Criando o Business Manager	18
O que é o Business Manager?	18
Criando o portfólio empresarial	18
Possíveis bloqueios no início	18
Criando a conta de anúncios	18
Ajustando o site da empresa	19
O que o site precisa ter?	19
Enviando o site para verificação	19

Separando os dados para a verificação da empresa	19
Quais dados devem ser separados?	19
Enviando essas informações	20
A verificação é obrigatória	20
Resumo do processo	20
04. Criando o Aplicativo na Meta e Configurando o Número	21
Acessando a área de desenvolvedores da Meta	21
Criando o novo aplicativo	21
Autenticação e segurança da conta	21
Iniciando a configuração da API do WhatsApp	22
Escolhendo o número que será usado	22
Cadastro do Perfil e Verificação	23
Configurando a forma de pagamento	23
Registrando o número de telefone	24
O que será necessário	24
Gerando o token de acesso	24
Passo para gerar o token	25
Usando o Postman para registrar o telefone	25
Parâmetros necessários	25
Tipo de autenticação	26
Definindo o PIN de registro	26
Enviando a requisição	26
Verificando o status do número	26
Testando o envio de mensagem	27
05. Conectando o n8n à Meta via Webhooks	28
O que é um Webhook?	28
Configuração Inicial no n8n	28
Ativação no Painel da Meta	29
O Desafio da Verificação (Hub Challenge)	30
Passo a Passo do Código de Resposta:	30
06. Ativando o webhook em produção e configurando o envio de mensagens no n8n	31
O que muda depois da validação do webhook?	31
Ativando o campo de mensagens na Meta	31
Ajustando o webhook no n8n para produção	32
Removendo o node de resposta manual	32
Alterando os parâmetros do webhook	32

Testando o recebimento de mensagens	33
Colocando o aplicativo em produção	34
Onde fazer isso	34
Informações obrigatórias para publicação	34
O que é o ideal aqui?	35
Cuidado com links com rastreamento	35
Recomendação importante	35
Publicando o aplicativo	35
Adicionando um node para envio de mensagens	36
Criando a credencial da Meta no n8n	36
Business Account ID	36
Access Token temporário	37
Configurando o node de envio	37
Testando o envio de mensagem	38
Entendendo a limitação do token temporário	39
Gerando um token permanente na Business Manager	39
Criando um usuário admin do sistema	39
Atribuindo o aplicativo a esse usuário	39
Gerando o token permanente	40
Escolhendo o aplicativo	40
Definindo a duração	40
Permissões necessárias	40
Verificação de segurança	40
Atualizando a credencial no n8n	41
Um cuidado essencial com o token permanente	41
7. Conectando as credenciais de apoio do agente no n8n	42
Por que usar serviços em cloud?	42
Preparando o Supabase	42
Criando a tabela principal de clientes	43
Entendendo o papel de cada campo	45
Conectando o Supabase no n8n	45
Campos principais da credencial	45
Criando a Vector Store no Supabase	46
O que é uma Vector Store?	48
Configurando as políticas da base no Supabase	48
Por que isso é necessário?	51
Conectando o Postgres no n8n	51

Conectando o Redis	52
Escolhendo o modelo de IA	52
Criando a credencial da OpenAI	53
Visão geral da infraestrutura criada	53
08. Estruturando o fluxo do agente no n8n	55
O ponto de partida do fluxo	55
Evitando que o fluxo processe mensagens enviadas pelo próprio sistema	55
Como identificar isso	56
Usando um node If	56
Organizando os dados da mensagem recebida	57
Variáveis principais	57
Verificando se o contato já existe no Supabase	58
Fazendo a busca	59
Ativando Always Output Data	59
Criando o lead quando ele ainda não existe	60
Campos enviados na criação	60
Lógica dessa etapa	62
Separando o fluxo por tipo de mensagem	62
Três caminhos principais	62
Tratando mensagens de texto	63
Por que usar um node No Operation?	63
Tratando mensagens de imagem	63
Buscando a URL da mídia	63
Autenticação	64
Fazendo o download da imagem	64
Enviando a imagem para análise	66
Tratando mensagens de áudio	67
Buscando a URL do áudio	68
Baixando o arquivo de áudio	68
Transcrevendo com a OpenAI	68
Criando uma única variável de mensagem	68
Exemplo conceitual	69
Usando o Redis para armazenar mensagens em curto prazo	69
Guardando a mensagem na lista	70
Ativando a opção Tail	70
Aguardando um tempo para capturar mensagens “picotadas”	70
Por que esperar?	71

Descobrimos se essa execução é a mais recente	71
O que essa comparação significa?	72
Encerrando execuções antigas	72
Agrupando as mensagens em uma única entrada	73
Exemplo conceitual	73
Limpando o buffer do Redis	73
Passando a mensagem consolidada para o agente principal	74
Campo <code>userMessage</code>	74
Campo <code>systemMessage</code>	74
Criando um segundo agente para formatar a resposta	74
Por que fazer isso?	75
Papel desse segundo agente	75
Transformando a resposta formatada em lista	76
Separando a lista em itens individuais	76
Enviando as mensagens de forma humanizada	77
O papel do <code>Wait</code>	78
Configurando o envio	78
Conclusão	79
09. Configurando a memória do agente de IA	80
Memória do agente: separando as conversas por telefone	80
Por que usar o telefone como chave?	81
Janela de contexto	81
Estrutura e Criação da Tabela <code>n8n_chat_histories</code>	82
Criação da tabela no Supabase	82
10. Como estruturar o prompt do agente de IA	83
O que o prompt realmente faz	83
A estrutura recomendada para o prompt	83
Seção 1 — Papel	84
O que entra nessa seção?	84
Exemplo conceitual	84
Por que isso melhora o resultado?	85
Técnicas úteis dentro da seção de papel	85
Role Prompting	85
Emotion Prompting	85
Seção 2 — Instruções	85
O que entra nessa seção?	86

Exemplo conceitual	86
Técnicas úteis na seção de instruções	86
Chain of Thought Prompting	86
Few-Shot Prompting	86
Seção 3 — Tools	87
O que é uma tool nesse contexto?	87
Por que explicar isso no prompt?	88
Seção 4 — Contexto operacional	88
Exemplos do que pode entrar aqui	88
Exemplo conceitual	88
Usando emoção também no contexto	89
Seção 5 — Regras	89
O que entra aqui?	89
Um princípio importante	89
Exemplos de regras úteis	90
Por que a ordem dessas seções importa	90
Estrutura resumida do prompt	90
Como esse prompt se encaixa no fluxo do n8n	91
Conclusão	91
11. Escrevendo o prompt do agente de IA	92
Por que escrever o prompt em Markdown?	92
Estrutura geral do prompt	92
Seção 1 — Papel	93
Exemplo de seção de papel	93
O que essa seção precisa deixar claro	93
Seção 2 — Instruções	93
Estrutura proposta para o atendimento	94
Exemplo de seção de instruções	94
Por que incluir exemplos?	95
Seção 3 — Tools	95
Ferramentas descritas no exemplo	95
Exemplo de seção de tools	96
O que essa seção resolve	96
Seção 4 — Contexto operacional	96
Exemplo de seção de contexto operacional	96
Por que isso ajuda?	97

Seção 5 — Regras	97
Exemplo de seção de regras	97
Um ponto importante sobre regras	98
Prompt completo em Markdown	98
Como evoluir esse prompt depois dos testes	100
Conclusão	100
12. Criando as ferramentas do agente de IA	101
Por que separar tarefas em tools?	101
Tool 1 — Buscar cursos	101
Nome e descrição da tool	102
Parâmetro enviado para a tool	103
Escrevendo o prompt da tool de cursos	103
Papel	103
Trilhas	103
Regras	104
Estrutura da tool de cursos	104
Escolhendo o modelo da tool	106
Tool 2 — Dados Asimov com Supabase Vector Store	106
Nome e descrição	107
Tabela utilizada	109
O que é RAG, na prática?	109
Exemplo prático	109
Tool 3 — Transferir atendimento	110
Quando usar essa tool	110
Criando um subfluxo	111
Parâmetro enviado para o subfluxo	111
Atualizando o status do contato no Supabase	112
Objetivo dessa atualização	113
Lógica da transferência	114
Buscando o histórico da conversa	115
Tabela utilizada	116
Preparando as mensagens para resumo	116
Exemplo conceitual	116
Criando um agente para resumir a conversa	116
Estrutura do prompt de resumo	117
Prompt para resumir a conversa	117
Visão geral das tools criadas	117

Resumo do que construímos	118
13. Finalizando as tools e preparando a pipeline de RAG	119
Enviando a notificação de transferência por e-mail	119
Montando o corpo do e-mail com variáveis	120
Criando a pipeline de RAG para a Vector Store	121
Organizando o conteúdo antes de subir para o RAG	121
O que é um chunk?	122
Entendendo a lógica do RAG	122
Embeddings, na prática	122
Fazendo um chunk por trilha ou curso	123
Script para separar texto em chunks	123
Por que isso funciona bem neste caso?	124
Subindo os chunks um por vez	124
Mantendo o mesmo modelo de embeddings	125
Configurando o carregamento do documento	126
Resultado esperado na Vector Store	126
Ajustando o fluxo principal antes dos testes	127
Corrigindo a referência da mensagem de texto	127
Bloqueando contatos transferidos	128
Limpando a memória antes dos testes	129
Validando o comportamento do agente	129
Teste com texto	129
Teste com áudio	130
Teste com buffer	130
Validando a consulta ao RAG	130
Criando uma segunda pipeline de RAG com arquivos	130
Etapas principais	131
Conclusão	131
14. Encerramento e próximos passos	132
O que foi construído ao longo da jornada	132
O que esse conhecimento permite fazer	132
Como evoluir a partir daqui	133
Uma base para aplicação prática e comercial	133
Conclusão	134

1. Introdução e Arquitetura de Agentes de IA

Bem-vindo à jornada de construção de agentes de inteligência artificial profissionais integrados ao WhatsApp. Neste capítulo, vamos entender o escopo do projeto e detalhar a “Stack Tecnológica”(o conjunto de ferramentas e serviços) que utilizaremos para criar um sistema robusto, escalável e inteligente.

O que vamos construir?

O objetivo deste material é capacitá-lo a criar uma infraestrutura completa de IA. Isso envolve desde a configuração de servidores até a integração com a **API Oficial da Meta**. Diferente de soluções amadoras, este método permite que você utilize os modelos mais avançados do mercado, como o **GPT-5 (OpenAI)**, **Claude (Anthropic)** ou **Gemini (Google)**.

Além de responder mensagens, nosso agente será capaz de:

- **Executar RAG (Retrieval-Augmented Generation):** Consultar bases de dados externas em tempo real.
- **Transferência de Atendimento:** Identificar quando um atendente real deve assumir a conversa.
- **Interagir via API Oficial:** Garantir a segurança e a credibilidade da conta junto à Meta.

A Stack Tecnológica

Para que um agente de IA funcione com alta performance, precisamos de uma arquitetura bem definida. Abaixo, detalhamos cada componente que será utilizado:

Hospedagem: Railway

A infraestrutura será hospedada no **Railway**. Escolhemos esta plataforma pela sua velocidade de deploy (instalação) e facilidade de gerenciamento.

- **Vantagem:** O Railway permite subir serviços como n8n, Redis e Postgres em poucos minutos.
- **Modelo de Cobrança:** Ele utiliza o sistema de “pay-as-you-go” (pague pelo uso), o que reduz os riscos de quedas por excesso de memória, pois o servidor escala conforme a demanda.

Orquestração: n8n

O **n8n** será o “cérebro” da nossa operação. Ele é uma ferramenta de automação baseada em fluxos (workflow) que conectará todas as pontas: o WhatsApp, a IA e os bancos de dados.

Integração: API Oficial da Meta

Utilizaremos a **API Oficial do WhatsApp Cloud**. Isso é fundamental para projetos profissionais, pois evita banimentos comuns em APIs não oficiais e permite a verificação oficial da empresa (o selo de credibilidade).

Bancos de Dados e Memória

- **Supabase**: Utilizado como banco de dados principal e também como **Vector Store** para o sistema de RAG.
- **Postgres**: Responsável por armazenar a memória das interações (histórico de conversas) de cada usuário.
- **Redis**: Funcionará como um **Buffer de Mensagens**.

O que é um Buffer? Sabe quando um usuário envia várias mensagens picotadas (ex: “Oi”, “Tudo bem?”, “Quero um curso”)? Sem um buffer, a IA responderia três vezes. O **Redis** agrupa essas mensagens e as entrega de uma só vez para a IA, economizando custos e tornando a conversa mais humana.

O Fluxo da Informação

Para entender como tudo se conecta, imagine o seguinte caminho percorrido por uma mensagem:

1. **WhatsApp**: O usuário envia uma mensagem.
2. **Webhook (n8n)**: A mensagem é capturada pela automação.
3. **Redis (Buffer)**: A mensagem aguarda alguns segundos para verificar se o usuário enviará mais textos.
4. **Agente de IA**: As mensagens agrupadas são enviadas para o modelo de linguagem.
5. **Consulta RAG (Opcional)**: Se o usuário perguntar algo específico (ex: “Quais são as regras de futebol americano?”), a IA busca essa informação no banco de dados externo do **Supabase** em tempo real.

6. **Memória (Postgres):** A interação é salva no histórico para que a IA lembre do que foi dito anteriormente.
7. **Resposta:** A IA envia a resposta final de volta ao usuário pelo WhatsApp.



Este é o esqueleto do nosso projeto. No próximo capítulo, passaremos para a parte prática: o deploy dessa infraestrutura no Railway.

02. Configurando sua Infraestrutura na Railway

Neste capítulo, aprenderemos como subir a infraestrutura necessária para o seu agente de IA. Utilizaremos a **Railway**, uma plataforma de nuvem (Cloud) que se destaca pela facilidade e velocidade, permitindo que serviços complexos sejam instalados em poucos cliques.

Escolhendo o Plano Ideal

A Railway oferece diferentes planos de acordo com a necessidade do seu projeto. Para a maioria dos usuários, o plano inicial de **US\$ 5,00 por mês** é mais do que suficiente.

The screenshot displays the Railway pricing page with four distinct plans. The 'Free' plan is at the top, followed by 'Hobby', 'Pro', and 'Enterprise Custom'. Each plan includes a list of features and a 'Deploy' button. The 'Pro' plan is highlighted with a purple border.

Plan	Price	Key Features	Deploy Button
Free	\$0 per month	Start with a 30-day free trial with \$5 credits, then \$1 per month; Up to 1 vCPU / 0.5 GB RAM per service; 0.5 GB of volume storage; Community support	Deploy a new project
Hobby	\$5 minimum usage	Includes \$5 of monthly usage credits; After credits are used, you'll only be charged for extra resource usage; Up to 48 vCPU / 48 GB RAM per service; Up to 6 replicas, at 8 vCPU / 8 GB RAM per replica; Up to 5 GB storage; Single developer workspace; Community support; 7-Day Log History; Global regions	Deploy with Hobby
Pro	\$20 minimum usage	Includes \$20 of monthly usage credits; After credits are used, you'll only be charged for extra resource usage; Up to 1,000 vCPU / 1 TB RAM per service; Up to 50 replicas, at 32 vCPU / 32 GB RAM per replica; Up to 1 TB storage; Unlimited workspace seats included; Railway Support; 30-Day Log History; Concurrent global regions	Deploy with Pro
Enterprise Custom	Custom	90-Day Log History; Single Sign-On; 18-Month Audit Log Retention; Role-Based Access Control; HIPAA BAAs; Support SLOs; Highest compute limits; Dedicated VMs; Bring your own cloud	Contact Sales

- **Plano Free (US\$ 5 de créditos):** Oferece teste gratuito de 30 dias com créditos de US \$5.00 créditos e US \$1.00 por mês com 0.5 de RAM. Ideal para projetos pequenos e testes.
- **Plano Hobby (US\$ 5):** Oferece 8 GB de RAM, o que suporta bem os fluxos de automação.
- **Plano Pro (US\$ 20):** Oferece 32 GB de RAM, ideal para equipes ou operações em escala.

Diferente de outros servidores, na Railway você pode cancelar a qualquer momento, sem a obrigatoriedade de planos anuais.

O Poder dos Templates

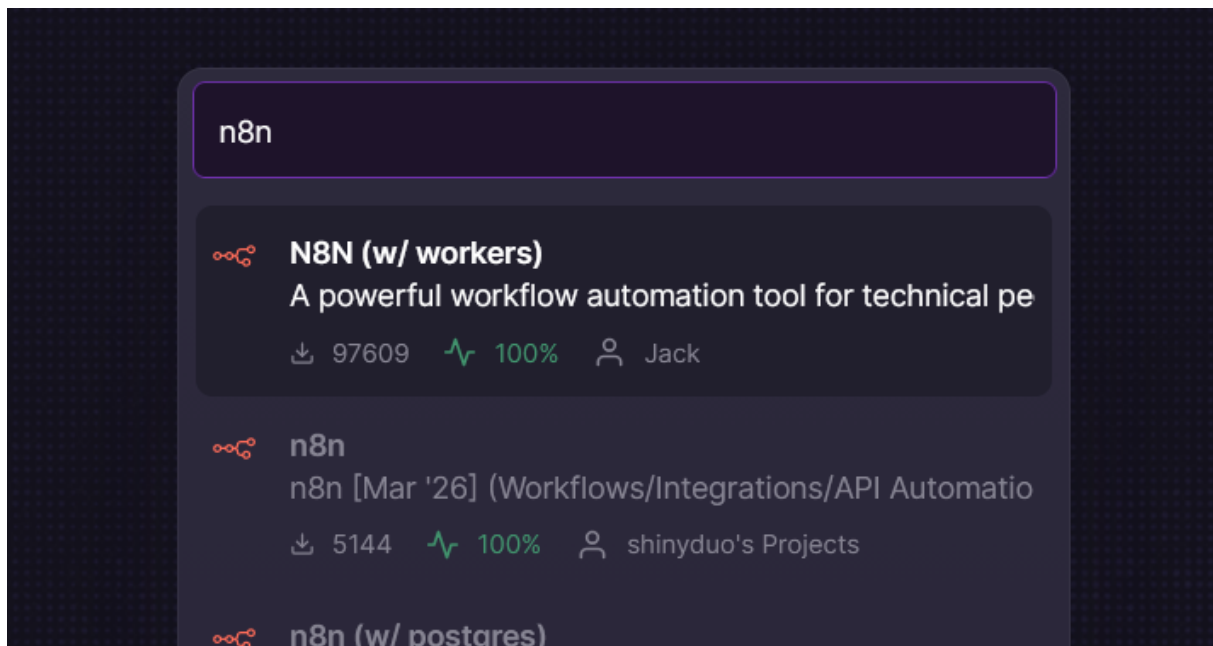
Para facilitar a instalação, utilizaremos um **Template**. Em computação, um template é um modelo pré-configurado que instala vários softwares de uma só vez.

O template que selecionaremos instala quatro serviços essenciais de uma única vez:

1. **n8n (Primary)**: A interface onde construiremos nossos fluxos.
2. **n8n (Worker)**: Responsável por processar as tarefas em segundo plano.
3. **Redis**: Nosso buffer de mensagens (cache).
4. **Postgres**: O banco de dados para a memória do agente.

Passo a Passo da Instalação:

1. Acesse o seu **Dashboard** na Railway e clique no botão **“New”**.
2. Selecione a opção **“Template”**.
3. Busque pelo template oficial do n8n - **N8N (w/ workers)** (o mais popular, com mais de 96 mil downloads).
4. Clique em **“Deploy”**.



O processo de *deploy* (colocação em funcionamento) é extremamente rápido. O Postgres e o Redis costumam ficar prontos em 15 segundos, e a estrutura completa em menos de um minuto.

Configuração Inicial do n8n

Após o término do deploy, você terá acesso ao link de produção do seu n8n (Primary Production). Ao clicar nele, você precisará configurar sua conta de proprietário:

1. **Criação de Conta:** Insira seu e-mail, nome, sobrenome e crie uma senha segura.
2. **Ativação de Recursos:** O n8n enviará uma chave para o seu e-mail para ativar funcionalidades extras da edição de comunidade de forma gratuita.
3. **Finalização:** Copie a chave recebida no e-mail e cole-a no painel do n8n para liberar todas as ferramentas de construção do seu agente.

Agora que temos nossa “casa” pronta e o n8n funcionando, o próximo passo é entender o que a Meta exige para que possamos usar o WhatsApp de forma profissional.

03. Preparação e Pré-requisitos da API Oficial (Meta)

Antes de começar a usar a API Oficial do WhatsApp com n8n, existe uma etapa que muita gente tenta apressar, mas que é decisiva para o sucesso da implantação: a preparação da estrutura da empresa dentro do ecossistema da Meta.

Diferente de outras integrações mais simples, a API Oficial do WhatsApp exige uma base empresarial minimamente validada. Isso acontece porque a Meta é bastante rigorosa com segurança, credibilidade e comprovação de que a empresa realmente existe. Em outras palavras, antes de pensar em fluxo, automação e mensagens, é preciso preparar corretamente a conta, a página, o Business Manager e a verificação da empresa.

Neste capítulo, você vai entender quais são esses pré-requisitos, por que eles são importantes e qual é a ordem ideal para fazer tudo com mais segurança.

Por que a Meta exige tantos pré-requisitos?

A API Oficial do WhatsApp é um recurso empresarial. Isso significa que ela não foi pensada para uso casual, pessoal ou improvisado. Como ela permite automações, integrações e comunicação profissional com clientes, a Meta tenta garantir que apenas empresas reais e minimamente organizadas possam utilizá-la.

Na prática, isso significa que a plataforma avalia sinais como:

- autenticidade da conta criada
- consistência das informações da empresa
- existência de presença digital
- relação entre o site, os dados empresariais e a conta da Meta
- validação de identidade e de documentos

Esse processo pode parecer burocrático no início, mas ele existe para reduzir spam, fraudes e uso indevido da plataforma. Por isso, seguir corretamente essas etapas aumenta bastante a chance de aprovação e reduz riscos de bloqueio.

Etapa inicial: criando a conta base

A primeira parte do processo consiste em criar a estrutura inicial que será usada para montar a operação empresarial.

Criando um Gmail para a operação

O primeiro passo recomendado é criar uma conta de Gmail específica para essa operação. Esse e-mail será usado para criar a conta no Facebook que servirá como base para o Business Manager.

Um detalhe importante é **não usar, nesse momento, um e-mail com domínio da empresa** para criar a conta do Facebook. Em vez disso, o ideal é usar um Gmail comum, criado apenas para esse processo. Isso ajuda a criar uma base mais estável para a conta inicial e evita alguns problemas comuns de validação logo no começo.

Criando a conta no Facebook

Com o Gmail pronto, o próximo passo é criar uma conta no Facebook com esse e-mail. Depois disso, basta seguir o processo normal da plataforma: confirmar o e-mail, validar o telefone e preencher os dados básicos da conta.

Até aqui, parece um cadastro comum. Mas é justamente no próximo ponto que começa uma das partes mais importantes do processo.

O aquecimento da conta

Depois de criar a conta no Facebook, a recomendação é não partir imediatamente para a criação do Business Manager. Antes disso, é preciso “aquecer” essa conta.

O que é aquecer a conta?

Aquecer a conta significa usar esse perfil como uma pessoa real usaria por alguns dias. Na prática, isso inclui ações simples, como:

- postar alguma foto
- navegar pela plataforma
- adicionar algumas pessoas
- interagir de forma básica com a conta

A ideia é fazer com que a Meta entenda que aquele perfil não foi criado de forma artificial apenas para montar uma estrutura empresarial rapidamente. Se a conta for criada e logo em seguida começar a abrir BM, conta de anúncios e outros ativos empresariais, a plataforma pode interpretar esse comportamento como suspeito.

Quanto tempo aquecer?

A recomendação é fazer esse aquecimento por alguns dias, algo entre **cinco dias e uma semana**. Para quem está começando, o ponto principal é: não tente acelerar essa parte. Esse “tempo de maturação” da conta ajuda a reduzir as chances de bloqueio logo no início do processo.

Criando e aquecendo a página do Facebook

Depois que o perfil pessoal estiver minimamente aquecido, o próximo passo é criar uma página no Facebook.

Preenchendo a página corretamente

Essa página deve ser preenchida da forma mais completa possível. Isso inclui:

- endereço
- imagem
- informações complementares
- publicações iniciais

Além disso, também é recomendado postar algum conteúdo ou alguma informação na página para que ela não fique vazia. A lógica é parecida com a do perfil pessoal: uma página sem contexto, sem conteúdo e recém-criada pode passar menos credibilidade para a Meta.

Verificação com número de WhatsApp

Ainda nessa etapa, é necessário a verificação com algum número de WhatsApp. Esse passo ajuda a fortalecer a legitimidade da estrutura criada e contribui para a consistência da conta como um todo.

E se eu quiser fazer tudo mais rápido?

O recomendado é aquecer tanto o perfil pessoal quanto a página por alguns dias. Mas, se houver urgência, a orientação mínima é aguardar pelo menos **4 horas** antes de seguir para a próxima etapa.

Mesmo assim, é importante entender que essa aceleração aumenta o risco da conta ser interpretada como atividade suspeita.

Criando o Business Manager

Com a base inicial preparada, chega o momento de criar a estrutura empresarial dentro da Meta.

O que é o Business Manager?

O Business Manager, também chamado de **BM**, é o ambiente empresarial da Meta. É nele que a empresa organiza seus ativos, permissões e integrações.

No contexto deste curso, ele será essencial porque a API Oficial do WhatsApp não fica isolada. Ela faz parte de uma estrutura empresarial maior, associada à conta da Meta.

Criando o portfólio empresarial

Agora vamos criar um **portfólio empresarial** e adicionar todas as informações da empresa nesse novo ambiente. Aqui começa uma fase mais sensível do processo, porque a Meta pode aplicar verificações extras.

Possíveis bloqueios no início

Por se tratar de uma conta nova, é comum aparecer alguma notificação de bloqueio ou de validação adicional. Quando isso acontecer, pode ser necessário enviar uma foto segurando um documento, como RG ou carteira de motorista. Normalmente, esse tipo de liberação pode acontecer em até 24 horas.

Atenção: Receber uma validação extra não significa necessariamente que algo deu errado. Em muitos casos, isso faz parte das verificações de segurança da própria Meta.

Criando a conta de anúncios

Depois de criar o Business Manager e passar pelas primeiras verificações, precisamos criar uma conta de anúncios. A conta de anúncios entra como parte do fortalecimento da qualidade da conta empresarial. O processo envolve preencher os dados com as informações do CNPJ e adicionar também um cartão de crédito.

Mesmo que o seu objetivo principal não seja anunciar, esse passo ajuda a consolidar a estrutura empresarial e mostra para a Meta que existe uma operação legítima por trás da conta.

Ajustando o site da empresa

Depois da criação da BM e da conta de anúncios, entra uma etapa muito importante: a adaptação do site da empresa.

O que o site precisa ter?

O site precisa estar compatível com as exigências de verificação da Meta. O rodapé do site deve conter as mesmas informações que serão cadastradas na BM, como:

- nome da empresa
- CNPJ
- endereço
- telefone da empresa

Todos esses dados precisam estar visíveis no site.

Enviando o site para verificação

Depois de verificar se o site está corretamente ajustado, o próximo passo é acessar o centro de segurança dentro da BM e enviar o site para verificação.

Separando os dados para a verificação da empresa

Com a estrutura inicial montada, é hora de organizar as informações da empresa que serão enviadas no processo de verificação da Meta.

Quais dados devem ser separados?

A aula orienta reunir os seguintes dados:

- nome da empresa
- CNPJ
- e-mail
- telefone
- endereço da empresa

Além disso, há um ponto muito importante sobre o telefone: ele deve ser o mesmo número cadastrado no CNPJ, e precisa estar apto a receber ligação ou SMS para validação por código.

Enviando essas informações

Esses dados devem ser enviados no centro de segurança da Meta, em formato de PDF, para completar o processo de verificação.

Mais do que simplesmente montar um arquivo, o importante aqui é garantir que as informações estejam organizadas, corretas e coerentes com os dados oficiais da empresa.

A verificação é obrigatória

A API Oficial do WhatsApp só pode ser usada por empresas verificadas. Ou seja, sem essa etapa concluída, a implantação não estará pronta para avançar com segurança.

Resumo do processo

Pode parecer um processo demorado, mas ele é essencial para evitar bloqueios e garantir que a empresa esteja apta a usar a API Oficial do WhatsApp da forma correta.

Você pode acessar o mapa mental completo desse processo no Material Complementar disponível para download nesse link: [Domine a API Oficial do WhatsApp com n8n.zip](#).

04. Criando o Aplicativo na Meta e Configurando o Número

Depois de preparar a estrutura da empresa e concluir as validações iniciais, o próximo passo é criar o aplicativo dentro da plataforma da Meta. Esse aplicativo será a base da integração com a API Oficial do WhatsApp. É por meio dele que a empresa passa a ter acesso às configurações necessárias para cadastrar o número, gerar token de acesso, registrar o telefone e testar o envio de mensagens.

Neste capítulo, você vai aprender a criar esse aplicativo, associá-lo ao Business Manager correto, adicionar um número novo de telefone e concluir o processo de registro até o momento em que a API já consegue enviar uma mensagem de teste com sucesso. Tudo será apresentado em sequência, de forma que tu possas reproduzir cada etapa com segurança.

Acessando a área de desenvolvedores da Meta

A criação do aplicativo é feita no portal de desenvolvedores da Meta ([Meta for Developers](#)). Esse ambiente concentra os aplicativos usados para integrações com APIs da empresa, incluindo a API Oficial do WhatsApp.

Ao acessar esse portal, o primeiro passo é entrar na seção **Meus Apps**. Dentro dessa área, basta clicar em **Criar Aplicativo** para iniciar a configuração.

Criando o novo aplicativo

Ao iniciar a criação, será necessário preencher algumas informações básicas:

- Dê um nome ao app (ex: “API_Oficial_Empresa”).
- Em casos de uso, procure por **“Conectar-se com clientes pelo WhatsApp”** e selecione essa opção.
- Selecione o seu **Portfólio Empresarial (BM)** verificado.
- Em requisitos não precisa preencher nada.

Se estiver tudo certo, basta avançar até chegar ao painel principal do aplicativo.

Autenticação e segurança da conta

Antes de acessar o painel do aplicativo, a Meta pode solicitar uma etapa adicional de segurança. Em muitos casos, isso envolve autenticação em dois fatores com envio de código por ligação ou SMS.

Por isso, é importante que a conta utilizada tenha acesso a um número de telefone capaz de receber esse tipo de verificação.

Criando agentes de atendimento profissionais no Whatsapp

Esse tipo de validação é comum e faz parte do processo de proteção da conta. Depois que ela for concluída, o acesso ao painel do aplicativo será liberado normalmente.

Iniciando a configuração da API do WhatsApp

Com o aplicativo criado, o próximo passo é abrir o caso de uso relacionado ao WhatsApp e iniciar a configuração da API.

Dentro do painel do aplicativo, deve-se selecionar a opção **conectar-se com os clientes pelo WhatsApp** e depois clicar em **começar a usar a API**. A partir daí, a Meta abre a área de configuração do número de telefone que será usado na integração.

The screenshot shows the Meta WhatsApp API configuration interface. On the left is a sidebar with navigation options: 'Casos de uso > Personalizar', 'Personalizar caso de uso', 'Permissões e recursos', 'Início rápido', 'Configuração da API' (highlighted), 'Configuração', 'Recursos', and 'Integração de Provedor de Tecnol...'. The main content area is titled 'Personalizar caso de uso' and includes a 'Token de acesso' section with a 'Gerar token de acesso' button. Below this is the 'Enviar e receber mensagens' section, which is divided into two steps. 'Etapa 1: Seleção de números de telefone' shows a 'De' field with '+55 11 5296 2972', an 'Identificação do número de telefone' field with '713649455175523', and an 'Identificação da conta do WhatsApp Business' field with '1555182578782782'. There is also an 'Até' field with 'US +1' and a 'Número de telefone' field. 'Etapa 2: Envie mensagens com a API' provides instructions on how to send a test message and includes a code block for a curl command.

```
1 curl -i -X POST \
2 https://graph.facebook.com/v22.0/713649455175523/messages \
3 -H 'Authorization: Bearer <access token>' \
4 -H 'Content-Type: application/json' \
5 -d '{ "messaging_product": "whatsapp", "to": "", "type": "template", "template": { "name": "hello_world", "language": { "code": "en_US" } } }'
```

É nessa etapa que o número será adicionado à conta do WhatsApp Business.

Escolhendo o número que será usado

Para utilizar a API Oficial do WhatsApp, é necessário cadastrar um número de telefone exclusivo para esse fim.

Esse número pode ser obtido de duas formas principais:

- usando um chip novo

- contratando um número virtual em um provedor externo

O ponto mais importante aqui é que esse número precisa ser **completamente novo**, isto é, nunca pode ter sido usado antes em uma conta do WhatsApp. Essa é uma exigência da própria Meta. Se você tentar usar um número que já tenha passado por alguma dessas situações, o cadastro pode falhar ou gerar conflito.

Depois de ter escolhido qual número usar, clique no campo “**De**”, onde vai estar um número de teste e selecione “**Adicionar telefone**”.

Cadastro do Perfil e Verificação

Depois de ter escolhido qual número usar, clique no campo “**De**”, onde vai estar um número de teste e selecione “**Adicionar telefone**”. Uma janela irá aparecer para o preenchimento das seguintes informações:

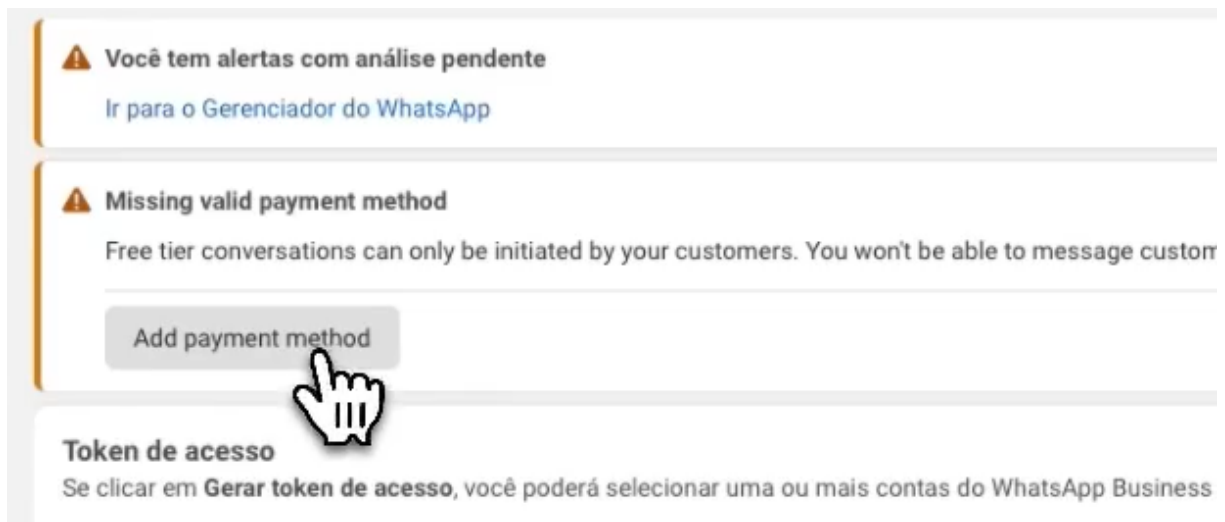
1. **Nome de Exibição:** Use um nome idêntico ou muito próximo à razão social da sua empresa para evitar recusas da Meta.
2. **Categoria:** Escolha o setor (ex: Serviços Profissionais).
3. **Verificação:** Insira o número e escolha “**Ligação Telefônica**” ou “**SMS**”, para receber o código de verificação.

Se tudo estiver correto, o número será adicionado com sucesso à conta do WhatsApp Business.

Configurando a forma de pagamento

Depois de cadastrar o número, ainda faltam algumas etapas antes de poder usá-lo efetivamente. A primeira delas é adicionar uma forma de pagamento válida.

Dentro das configurações da conta criada, é necessário acessar a área de pagamento e cadastrar um cartão de crédito. Isso é exigido para que a conta esteja apta a operar corretamente com a infraestrutura da API do WhatsApp.



Depois de adicionar o cartão, a página pode ser recarregada para atualizar o status das pendências.

Registrando o número de telefone

Adicionar o número na Meta não significa que ele já está totalmente pronto para uso. Ainda é necessário **registrá-lo**.

Essa etapa é feita por meio de uma requisição específica, usando a coleção oficial disponível no Postman.

O que será necessário

Para registrar o número, você vai precisar de três informações:

- a versão da API
- o **phone_number_id**
- o token de acesso

Esses dados são obtidos no próprio painel da Meta.

Gerando o token de acesso

Antes de ir ao Postman, é necessário gerar o token associado ao número cadastrado.

No painel da Meta, o processo consiste em selecionar o número adicionado e gerar o token de acesso correspondente. Esse token será usado para autenticar a requisição de registro no Postman.

É importante salvar esse token com cuidado, porque ele será utilizado logo em seguida.

Passo para gerar o token

- No campo “**Ate**”, que fica abaixo de onde cadastramos o nosso número virtual, preencha com seu número pessoal.
- Em seguida clique em **Gerar token de acesso**.
- Uma janela irá aparecer. Selecione seu número e clique em avançar.

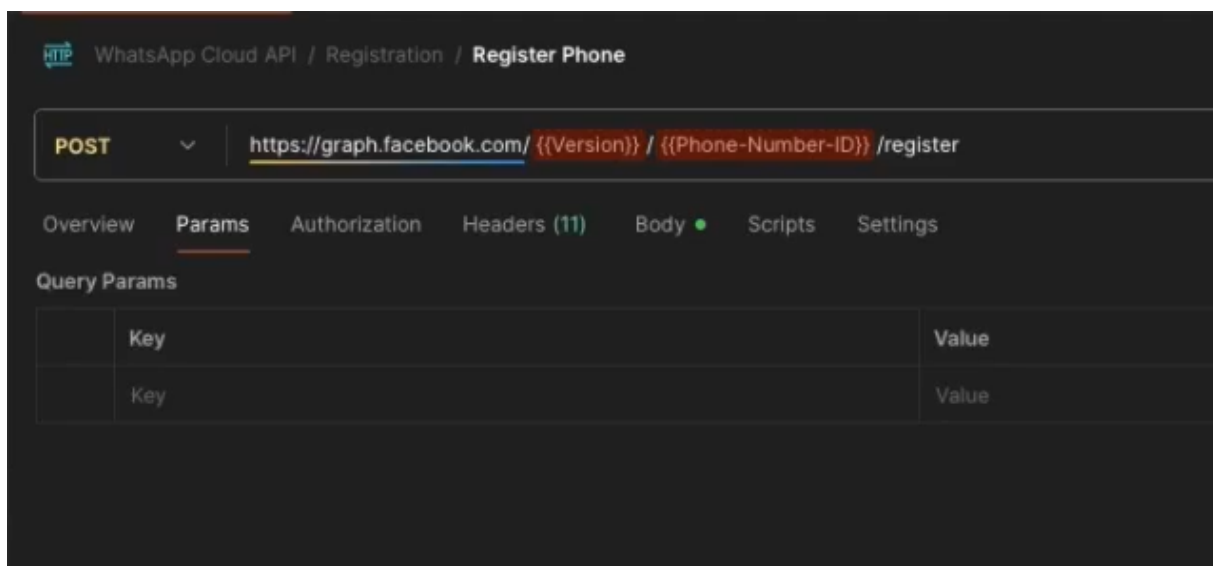
Usando o Postman para registrar o telefone

Depois de gerar o token, o próximo passo é abrir o Postman, clicando no botão **Executar no Postman**, e criar uma conta. Em seguida, deve-se usar a coleção da **WhatsApp Cloud API** e abrir a pasta de registro de telefone: **Registration > Register phone**.

Parâmetros necessários

Na requisição de registro, será necessário preencher:

- version
- phone_number_id
- token de autenticação



A versão da API é um valor como este: v22.0

O **phone_number_id** é a identificação interna do número cadastrado na Meta.

Todas essas informações estão disponíveis no painel de configuração da API.

Tipo de autenticação

Na aba **Authorization** do Postman, deve-se selecionar o tipo de autenticação da Meta API e colar o token gerado anteriormente.

Definindo o PIN de registro

Na aba **Body**, é necessário configurar um PIN de 6 dígitos. Esse PIN faz parte do processo de registro do número. Um exemplo simples seria:

```
{
  "messaging_product": "whatsapp",
  "pin": "123456"
}
```

Esse é apenas um exemplo didático. Em um ambiente real, o ideal é escolher um PIN fácil de guardar, mas não tão óbvio.

Enviando a requisição

Depois de preencher tudo, basta enviar a requisição. Se o registro for concluído corretamente, a resposta indicará sucesso com um retorno semelhante a este:

```
{
  "success": true
}
```

Esse retorno mostra que o número foi registrado com sucesso.

Verificando o status do número

Com o número registrado, o próximo passo é voltar ao painel da Meta e verificar o status da conexão.

Inicialmente, pode acontecer de o número aparecer como **pendente** e o nome de exibição ainda estar **em análise**. Depois de atualizar a página, o status tende a mudar para **conectado**.

Testando o envio de mensagem

Depois que o número estiver conectado, já é possível fazer um teste de envio de mensagem.

A própria plataforma disponibiliza um template inicial, geralmente o **Hello World**, que pode ser enviado ao seu número pessoal para verificar se a integração está funcionando corretamente.

Se a mensagem for recebida no WhatsApp, isso confirma que:

- o número foi cadastrado corretamente
- a conta está conectada à API
- o processo de registro foi concluído com sucesso

Em outras palavras, esse teste é a comprovação prática de que a integração inicial funcionou.

Com isso concluído, a estrutura principal da integração já está pronta. O próximo passo natural é conectar essa conta ao n8n, especialmente por meio da configuração do webhook que permitirá a troca de mensagens entre a Meta e os fluxos automatizados.

05. Conectando o n8n à Meta via Webhooks

Neste capítulo, vamos configurar o “sistema de escuta” do seu agente. Para que o n8n saiba quando alguém enviou uma mensagem no WhatsApp, utilizamos um **Webhook**.

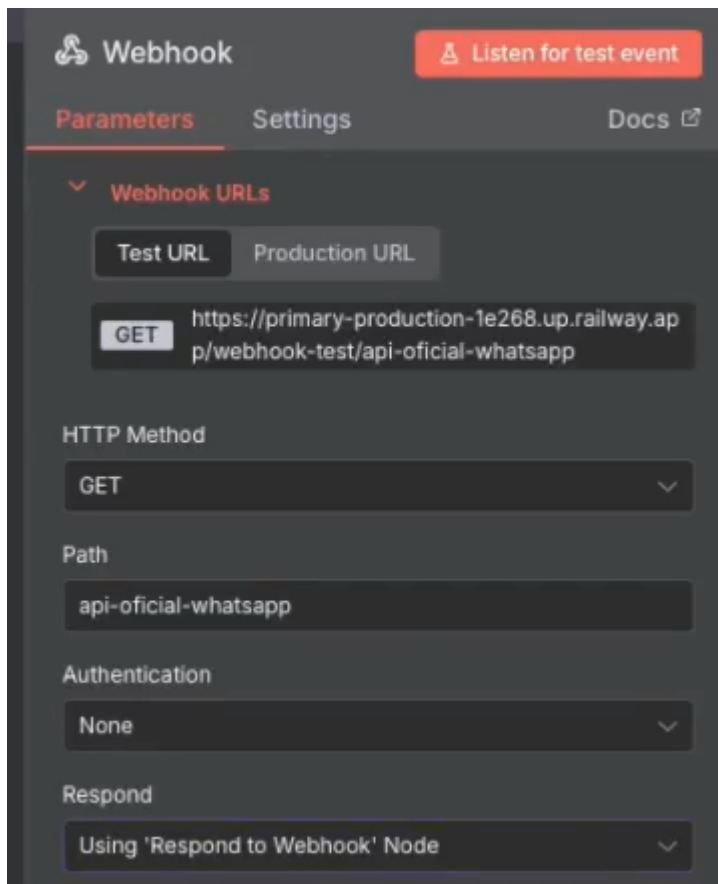
O que é um Webhook?

Para iniciantes, imagine o Webhook como um “carteiro digital” que fica esperando uma carta (mensagem) chegar. Assim que a Meta recebe uma mensagem no seu número, ela a “empurra” instantaneamente para o endereço (URL) que configuramos no n8n.

Configuração Inicial no n8n

O primeiro passo é criar o ponto de entrada da nossa automação:

1. No n8n, crie um novo workflow e adicione o nó **Webhook**.
2. **Método HTTP**: Selecione **GET**.
 - Usamos GET apenas para a primeira validação da Meta. Depois de conectado, mudaremos para POST.
3. **Path**: Escolha um nome amigável, como **api-oficial-whatsapp**.
4. **Respond**: Mude para **Using ‘Respond to Webhook’ Node**
5. **URL de Produção**: Copie a URL de **Production**. É fundamental que o fluxo no n8n esteja como **Active** para que a Meta consiga validar o endereço.



Ativação no Pannel da Meta

Com o fluxo ativo no n8n, volte ao painel do seu aplicativo na Meta:

1. Vá em **Configuração > Webhooks**.
2. Em **URL de callback**, cole a URL de produção do n8n.
3. Em **Verificar token**, digite uma palavra simples (ex: token).
4. Clique em **Verificar e salvar**. Se tudo estiver correto, a conexão será estabelecida.

Webhook
Para receber alertas quando chegar uma mensagem ou quando o status de uma mensagem mudar, você precisa configurar um ponto de extremidade do Webhooks para seu ap

URL de callback ⓘ

https://primary-production-1e268.up.railway.app/webhook/api-oficial-whatsapp

Verificar token ⓘ

token

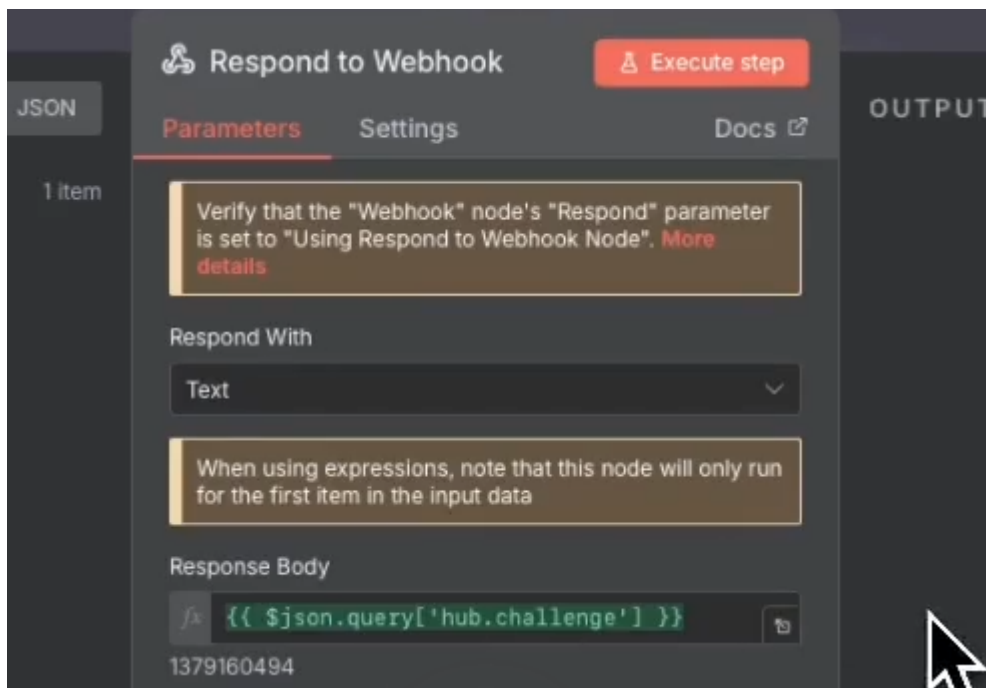
☐ Anexe um certificado de cliente às solicitações de webhook. [Saiba mais](#).

O Desafio da Verificação (Hub Challenge)

A Meta exige que seu servidor (n8n) prove que está “ouvindo” antes de liberar o envio de mensagens. Ela envia um código chamado `hub.challenge`, e seu fluxo deve responder exatamente com esse mesmo código.

Passo a Passo do Código de Resposta:

1. Após o nó de Webhook, adicione o nó **Respond to Webhook**.
2. Configure a opção **Respond With** como **Text**.
3. No campo de texto, utilize a seguinte expressão para capturar o código da Meta: `{{ $json.query["hub.challenge"] }}`.



Com essa etapa concluída, a Meta já consegue reconhecer e validar o endpoint do seu workflow no n8n. Isso significa que a base da comunicação entre as duas plataformas está pronta.

Esse é um momento importante da implantação, porque o webhook validado passa a ser o canal por onde os eventos do WhatsApp chegarão ao seu fluxo. Nos próximos passos, a tendência é evoluir dessa validação inicial para o recebimento de mensagens e o processamento dessas informações dentro da automação.

06. Ativando o webhook em produção e configurando o envio de mensagens no n8n

No capítulo anterior, o webhook foi validado com sucesso na Meta. A partir disso, a integração já tem uma base funcional: a Meta conhece o endpoint do n8n e consegue se comunicar com ele.

Agora começa uma etapa decisiva da implantação. O objetivo deste capítulo é transformar aquele webhook de validação em um webhook de produção, publicar o aplicativo da Meta corretamente, conectar a credencial de envio no n8n e substituir o token temporário por um token permanente. Com isso, sua estrutura ficará pronta para receber mensagens reais e também para responder por meio da API Oficial do WhatsApp.

O que muda depois da validação do webhook?

Durante a validação inicial, o webhook foi configurado apenas para responder ao desafio enviado pela Meta. Por isso, o fluxo usava:

- método **GET**
- node **Respond to Webhook**
- retorno manual do **hub.challenge**

Essa configuração serve para provar que o endpoint existe e consegue responder corretamente à Meta. Mas, depois que a validação termina, o objetivo muda. Agora o webhook precisa passar a **receber eventos reais**, como mensagens enviadas pelos usuários.

Por esse motivo, a estrutura do fluxo precisa ser simplificada.

Ativando o campo de mensagens na Meta

Dentro da configuração do webhook na Meta, existe uma etapa em que você ativa o recebimento de eventos relacionados a mensagens. No painel da Meta, em **Webhooks**, localize **messages** e clique em **Assinar (Subscribe)**.



Depois dessa ativação, a integração já pode começar a enviar dados reais para o n8n quando alguém conversar com o número conectado.

Em termos práticos, isso significa que o webhook deixa de existir apenas para validação técnica e passa a fazer parte do funcionamento normal da integração.

Ajustando o webhook no n8n para produção

Depois que a validação já foi concluída, o fluxo do n8n precisa ser atualizado.

Removendo o node de resposta manual

O node **Respond to Webhook** foi útil apenas no momento da validação inicial. Como essa etapa já terminou, ele pode ser removido.

Isso acontece porque agora o objetivo não é mais devolver manualmente o **hub.challenge**, e sim receber requisições reais da Meta.

Alterando os parâmetros do webhook

No node Webhook, o campo de resposta (**Respond**) deve voltar para a opção padrão (**Immediately**). Em vez de responder usando um node separado, o próprio node Webhook passa a seguir o comportamento padrão do n8n.

Outra mudança importante é o método HTTP. Durante a validação, ele precisava ser: GET. Agora, para receber os eventos reais da Meta, ele deve ser alterado para: **POST**

Essa troca é essencial, porque os eventos de mensagem chegam ao webhook por meio de requisições **POST**.

The screenshot shows the 'Parameters' tab of an n8n workflow configuration. At the top, there are tabs for 'Parameters', 'Settings', and a red button labeled 'Listen for test event'. Below the tabs, the 'Webhook URLs' section is expanded, showing two buttons: 'Test URL' and 'Production URL'. Under 'Production URL', there is a 'POST' method and a text input field containing the URL 'http://localhost:5678/webhook/api-oficial-whatsapp'. Below this, the 'HTTP Method' is set to 'POST' in a dropdown menu. The 'Path' is set to 'api-oficial-whatsapp' in a text input field. The 'Authentication' is set to 'None' in a dropdown menu. Finally, the 'Respond' option is set to 'Immediately' in a dropdown menu.

Depois dessas mudanças, basta salvar o workflow. A partir daí, ele já estará preparado para receber mensagens reais enviadas ao número conectado.

Testando o recebimento de mensagens

Com o webhook ajustado para produção, o próximo passo é fazer um teste simples: enviar uma mensagem para o número conectado à API e verificar se o n8n recebe esse evento.

Quando o teste funciona, o node Webhook registra uma nova execução. Nessa execução, normalmente aparecem informações como:

- número de telefone do remetente
- nome do contato

- conteúdo da mensagem
- tipo da mensagem

Por exemplo, se alguém enviar um texto como “Oi”, esse conteúdo aparecerá na execução do webhook junto com os demais metadados da mensagem.

Esse é um ponto muito importante, porque confirma que o gatilho do fluxo já está operacional.

Colocando o aplicativo em produção

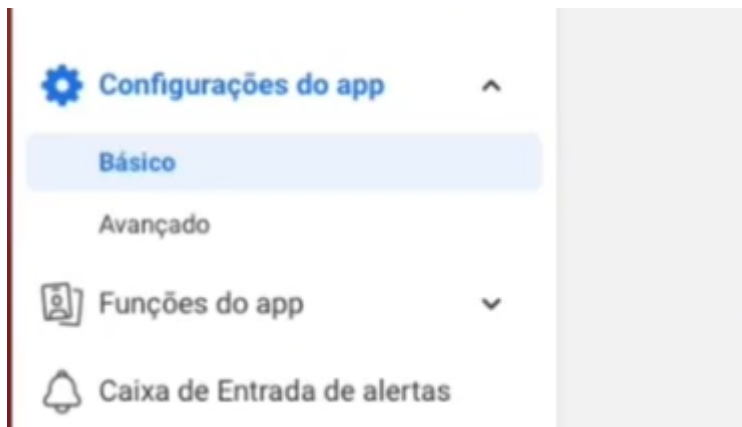
Até esse momento, o aplicativo criado na Meta ainda está em modo de desenvolvimento. Isso é suficiente para testes, mas não é o estado ideal para uso real em um projeto de produção. Por isso, o próximo passo é publicar o aplicativo.

Onde fazer isso

Dentro do painel da Meta, acesse:

- **Configurações do app**
- **Básico**

Nessa área, será necessário preencher algumas informações obrigatórias antes da publicação.



Informações obrigatórias para publicação

Para publicar o aplicativo, a Meta exige pelo menos duas URLs importantes:

- página de **Termos de Serviço**

- página de **Política de Privacidade**

Essas páginas devem estar disponíveis no site da sua própria empresa.

O que é o ideal aqui?

O ideal é que sua empresa tenha duas páginas próprias e públicas no site principal:

- uma página com os termos de uso ou termos de serviço
- uma página com a política de privacidade

Essas páginas ajudam a Meta a validar que o aplicativo está ligado a uma operação real e minimamente estruturada.

Cuidado com links com rastreamento

Ao copiar essas URLs, é importante usar apenas o endereço limpo da página, sem parâmetros extras de rastreamento no final do link. Antes de salvar, vale testar a URL no navegador para verificar se ela abre corretamente.

Recomendação importante

Mesmo que existam improvisos temporários em ambientes de teste, o caminho correto é sempre utilizar páginas da própria empresa. Para uma implantação profissional, essa parte não deve ser tratada como detalhe. Ela faz parte da credibilidade do aplicativo.

Publicando o aplicativo

Depois de preencher corretamente os campos exigidos, a Meta passa a exibir a informação de que todas as configurações necessárias do app foram concluídas. Quando essa confirmação aparecer, você já pode acessar a área de publicação e colocar o app em produção.

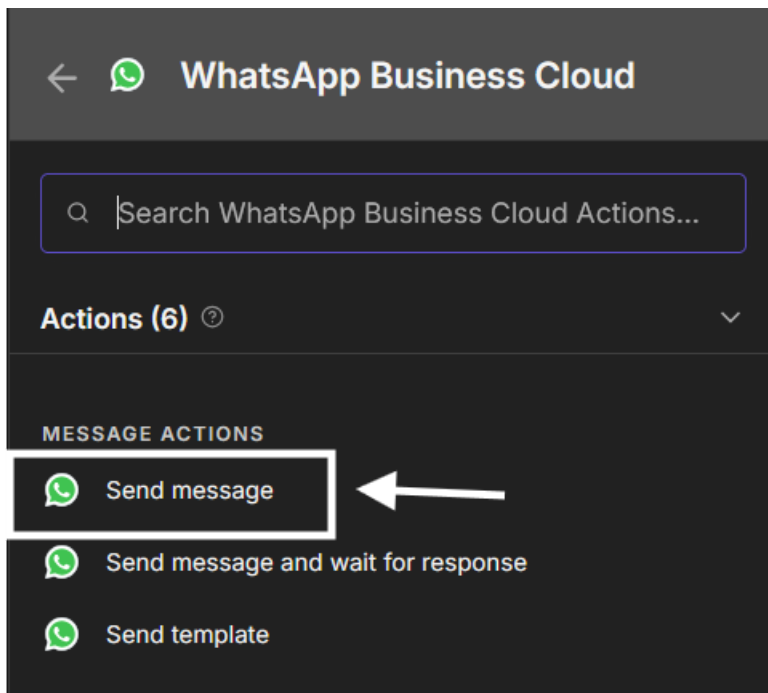
A publicação do app está disponível no painel da Meta na aba **Publicar**. Nessa página você terá o botão **Publicar App**. Se a publicação for concluída com sucesso, o aplicativo deixa o estado de desenvolvimento e passa a estar apto para uso mais próximo de um cenário real.

Adicionando um node para envio de mensagens

Com o webhook já recebendo mensagens e o aplicativo publicado, o próximo passo é configurar o envio de mensagens no n8n.

Para isso, adicione ao workflow um node do tipo:

WhatsApp Business Cloud > Send messages



Dentro dele, selecione a ação de envio de mensagens. Esse node será responsável por fazer a comunicação de saída com a API da Meta.

Criando a credencial da Meta no n8n

Ao adicionar o node de envio, você precisará criar uma nova credencial.

Business Account ID

No painel da Meta, dentro da área de **Casos de uso > Configuração da API**, selecione o número que foi conectado anteriormente. Em seguida, copie o valor da **identificação da conta do WhatsApp Business**.

Esse valor deve ser colado no campo **Business Account ID** da credencial no n8n.

Access Token temporário

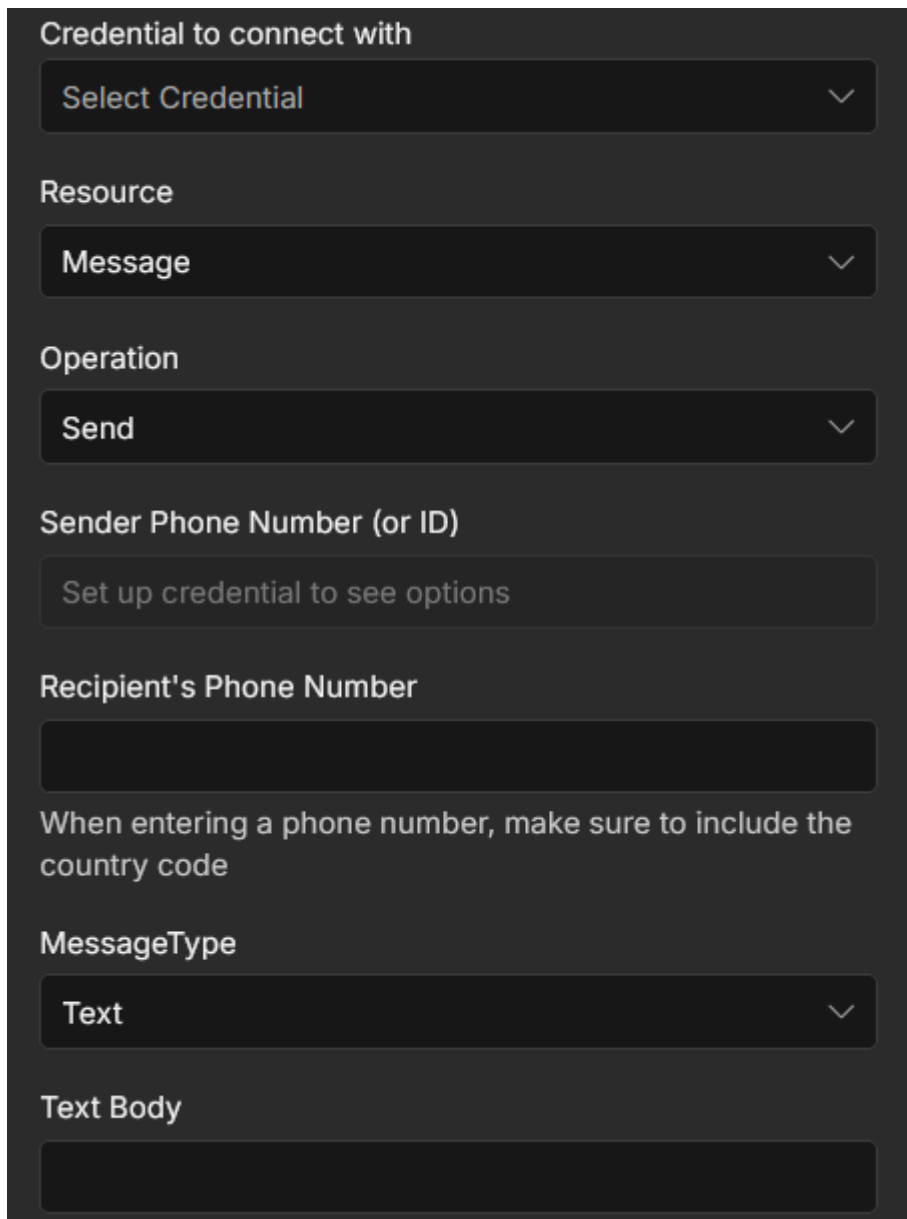
Ainda no painel da Meta, gere um token de acesso para conectar a conta ao n8n. Esse token inicial pode ser usado para criar e testar a credencial. Depois de copiar o valor, cole-o no campo **Access Token** e salve a credencial.

Se tudo estiver correto, o n8n indicará que a credencial foi conectada e testada com sucesso.

Configurando o node de envio

Depois que a credencial estiver pronta, você pode configurar o node de envio de mensagens.

- **Resource:** Campo com valor padrão message.
- **Operation:** Campo com valor padrão send.
- **Sender Phone Number:** Define qual telefone da sua conta da Meta será usado para enviar a mensagem.
- **Recipient's Phone Number:** O número de destino será o telefone para o qual a mensagem será enviada. Em um fluxo real, esse valor normalmente virá do próprio gatilho, ou seja, do número que enviou a mensagem ao seu webhook.
- **MessageType:** O node permite diferentes tipos de mensagem, como: texto, áudio, vídeo, localização e imagem. Neste momento, o tipo mais simples para começar é texto.
- **Text Body:** No campo de texto, você pode inserir uma mensagem simples para teste, como: "Oi"



The image shows a dark-themed configuration interface for a WhatsApp API. It contains several labeled sections, each with a corresponding input field or dropdown menu:

- Credential to connect with:** A dropdown menu with the text "Select Credential" and a downward arrow.
- Resource:** A dropdown menu with the text "Message" and a downward arrow.
- Operation:** A dropdown menu with the text "Send" and a downward arrow.
- Sender Phone Number (or ID):** A text input field containing the placeholder text "Set up credential to see options".
- Recipient's Phone Number:** An empty text input field.
- When entering a phone number, make sure to include the country code:** A line of instructional text.
- MessageType:** A dropdown menu with the text "Text" and a downward arrow.
- Text Body:** An empty text input field.

Depois disso, basta executar o node.

Testando o envio de mensagem

Se a configuração estiver correta, a mensagem será enviada com sucesso para o número informado. Esse teste confirma que a integração não apenas recebe eventos, mas também consegue responder ou iniciar comunicações via API.

Esse é um marco importante da implantação, porque a partir daqui o fluxo já consegue operar nos dois sentidos:

- entrada de mensagens no webhook
- saída de mensagens pelo node da Meta

Entendendo a limitação do token temporário

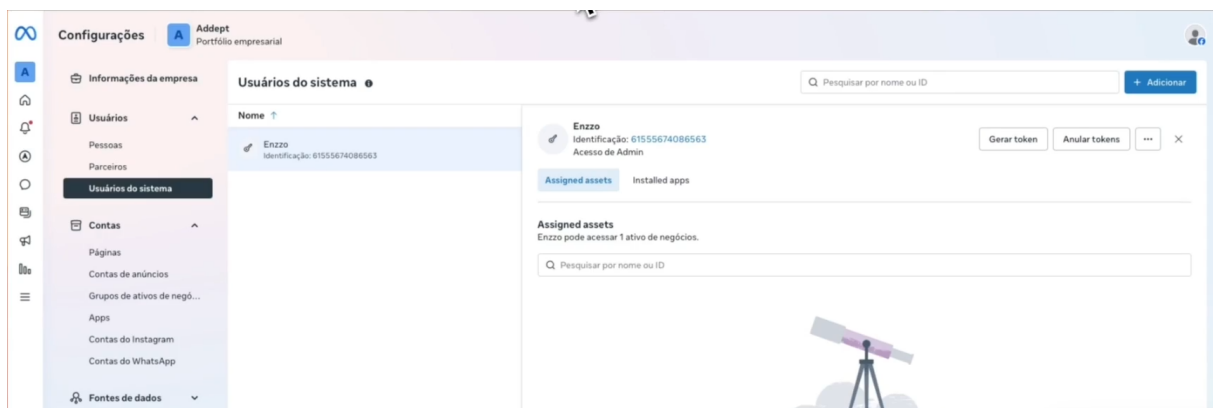
Apesar de funcionar, o token usado até aqui ainda não é o ideal para produção. Isso porque ele é **temporário**. Em algum momento, ele expira, e o node de envio deixa de funcionar até que você gere outro token manualmente.

Esse comportamento é aceitável em testes, mas se torna um problema em projetos reais. Por isso, o próximo passo é substituir esse token temporário por um token permanente.

Gerando um token permanente na Business Manager

Para gerar um token permanente, o processo deve ser feito diretamente na área empresarial da Meta, acessando a BM usada nesse aplicativo.

O caminho começa em: business.facebook.com. Dentro da BM correta, acesse a área de **Usuários do sistema**.



Criando um usuário admin do sistema

Se ainda não existir um usuário do sistema com perfil administrativo, crie um novo. Esse usuário será usado para gerar o token permanente e administrar os ativos relacionados ao aplicativo.

Atribuindo o aplicativo a esse usuário

Depois de criar o usuário, é necessário atribuir os ativos correspondentes a ele. Na prática, isso significa dar acesso ao aplicativo da API Oficial do WhatsApp dentro da BM.

Ao atribuir o app, conceda as permissões de gerenciamento necessárias para que esse usuário possa operar sobre ele.

Gerando o token permanente

Com o usuário do sistema já configurado, você poderá usar a opção **Gerar token**.

Escolhendo o aplicativo

Durante a geração do token, selecione o aplicativo que está sendo usado na integração.

Definindo a duração

Na etapa de duração, é possível optar por um token com validade estendida. Para evitar a necessidade de troca frequente, a configuração mais prática é usar a opção de duração contínua, quando ela estiver disponível.

Permissões necessárias

Na seleção de permissões, é necessário habilitar permissões relacionadas ao gerenciamento e envio de mensagens no WhatsApp Business. Entre elas, estão permissões como:

- gerenciamento de eventos
- gerenciamento da conta do WhatsApp Business
- envio de mensagens

Essas permissões garantem que o token possa ser usado corretamente pelo n8n para operar a integração.

Verificação de segurança

Antes de concluir a geração, a Meta pode solicitar uma verificação adicional por código enviado ao seu e-mail. Depois de informar esse código, o token será finalmente gerado.

Atualizando a credencial no n8n

Com o novo token em mãos, volte ao n8n e edite a credencial criada anteriormente.

No campo Access Token, substitua o token temporário pelo token permanente e salve a alteração.

Depois disso, sua credencial ficará muito mais estável para uso em produção.

Um cuidado essencial com o token permanente

Depois de gerado, esse token não fica sempre visível para consulta dentro da plataforma. Por isso, é fundamental salvá-lo em um local seguro, como:

- gerenciador de senhas
- cofre de segredos
- arquivo seguro de documentação interna

Não é recomendável depender apenas da memória ou deixar esse valor solto em anotações improvisadas. Se esse token se perder, o processo de recuperação pode gerar retrabalho.

Neste ponto, a integração já atingiu um nível muito mais sólido. O webhook está preparado para receber mensagens reais, o aplicativo foi publicado corretamente, o node de envio está funcional e a autenticação já pode ser mantida com um token permanente.

Em outras palavras, a infraestrutura principal da comunicação entre Meta e n8n está pronta. A partir daqui, o próximo passo natural é começar a construir a lógica do fluxo: interpretar a mensagem recebida, decidir como responder e transformar essa base técnica em uma automação realmente útil.

7. Conectando as credenciais de apoio do agente no n8n

Antes de começar a construir a lógica do agente, é necessário preparar a infraestrutura que vai dar suporte ao fluxo. Nesta etapa, o foco deixa de ser a conexão entre Meta e WhatsApp e passa a ser a conexão das ferramentas que vão sustentar o agente por trás da automação. Entre elas, estão o **Supabase**, o **Postgres**, o **Redis** e o provedor do **modelo de IA**.

Essas conexões são importantes porque cada uma cumpre uma função específica no projeto:

- o **Supabase** pode ser usado como banco de dados da aplicação;
- o **Postgres** pode ser usado como memória do agente;
- o **Supabase Vector Store** pode servir como base para RAG;
- o **Redis** pode ser usado como apoio de persistência e desempenho;
- a **OpenAI** — ou outro provedor semelhante — será usada para o modelo de linguagem do agente.

Por que usar serviços em cloud?

Neste tipo de projeto, uma escolha comum é decidir entre subir tudo na própria infraestrutura ou utilizar serviços gerenciados em nuvem. Para muitos casos práticos, especialmente em projetos pequenos ou médios, usar os planos gratuitos de ferramentas como Supabase e Redis pode ser uma solução bastante eficiente.

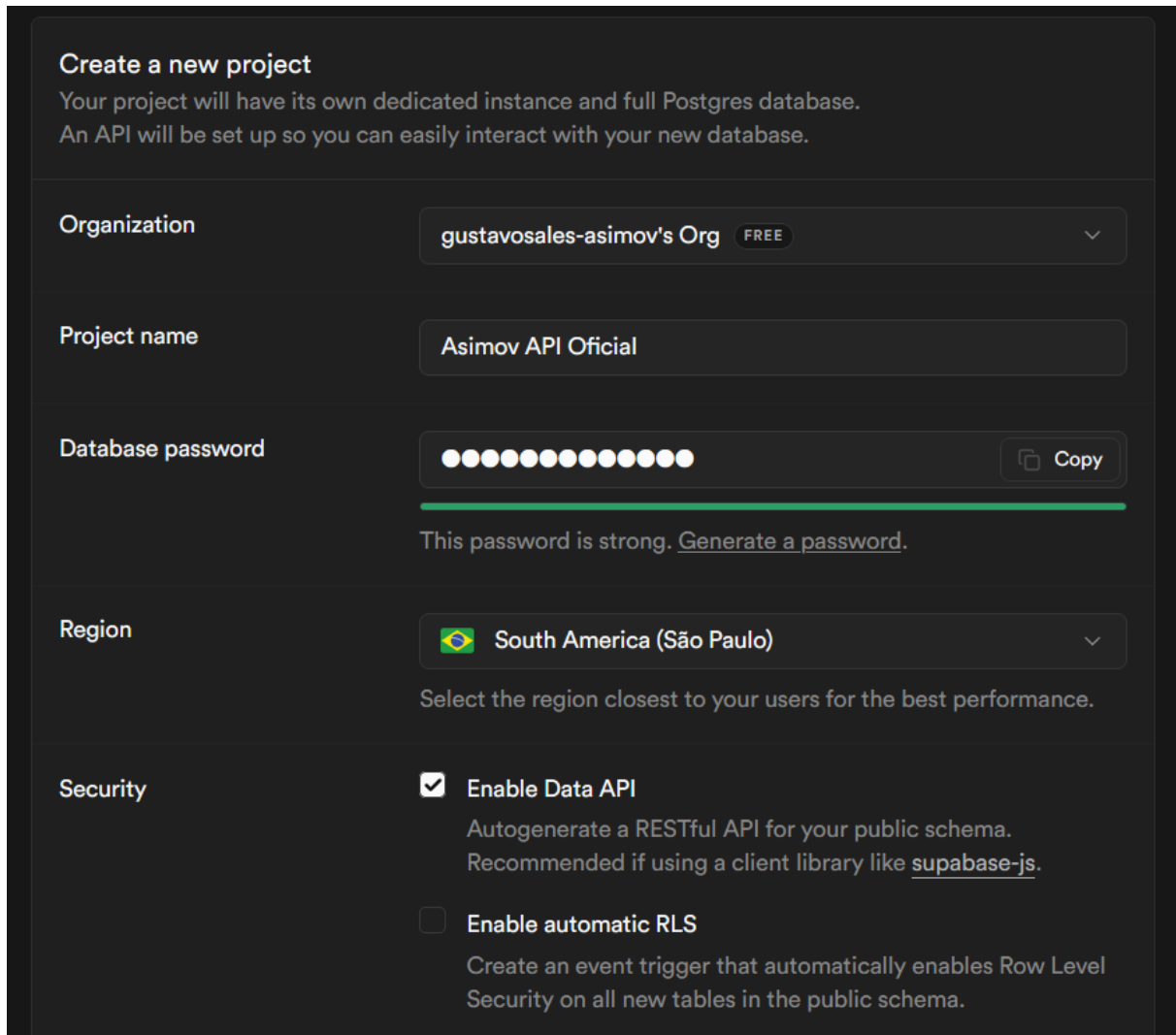
A principal vantagem dessa abordagem é simples: você reduz a responsabilidade de manter mais serviços rodando manualmente. Em vez de instalar, monitorar e atualizar tudo no seu próprio servidor, você delega parte dessa gestão à plataforma. Isso tende a simplificar a operação, principalmente em ambientes de teste, MVPs ou automações que ainda não exigem grande escala.

Isso não significa que a única forma correta seja usar cloud. Se você preferir instalar essas ferramentas no seu próprio servidor, o fluxo conceitual continua o mesmo. A diferença está no modelo de operação, e não na lógica da integração.

Preparando o Supabase

O primeiro serviço a ser conectado é o **Supabase**. Para isso, você precisa ter uma conta criada e, dentro dela, criar uma organização e um projeto. Durante a criação do projeto, será necessário definir uma senha para o banco de dados. Essa senha deve ser guardada com cuidado, porque ela será usada mais adiante na conexão do Postgres com o n8n.

Também é recomendável escolher uma região próxima da sua operação, como **São Paulo**, para reduzir latência e manter a infraestrutura mais próxima do ambiente do projeto.



The screenshot shows the 'Create a new project' interface in Supabase. It includes a title 'Create a new project', a description 'Your project will have its own dedicated instance and full Postgres database. An API will be set up so you can easily interact with your new database.', and several form fields: 'Organization' (gustavosales-asimov's Org, FREE), 'Project name' (Asimov API Oficial), 'Database password' (masked with dots, with a 'Copy' button and a strength indicator), 'Region' (South America (São Paulo)), and 'Security' options (Enable Data API checked, Enable automatic RLS unchecked).

Create a new project
Your project will have its own dedicated instance and full Postgres database.
An API will be set up so you can easily interact with your new database.

Organization
gustavosales-asimov's Org FREE

Project name
Asimov API Oficial

Database password
[Masked password] Copy
This password is strong. [Generate a password.](#)

Region
South America (São Paulo)
Select the region closest to your users for the best performance.

Security
☒ **Enable Data API**
Autogenerate a RESTful API for your public schema.
Recommended if using a client library like [supabase-js](#).
☐ **Enable automatic RLS**
Create an event trigger that automatically enables Row Level Security on all new tables in the public schema.

Criando a tabela principal de clientes

Com o projeto criado, o próximo passo é abrir o **Table Editor** do Supabase e criar uma tabela que armazenará os dados principais dos contatos do WhatsApp. Um nome possível para essa tabela seria `clientes_whatsapp`, mas você pode adaptar o nome conforme a organização do seu projeto.

A estrutura inicial é composta por:

- `telefone` como chave primária;
- `created_at` para registrar o momento de criação da linha;

Criando agentes de atendimento profissionais no Whatsapp

- nome como texto;
- status como texto.

Create a new table under public

Name

Clientes Whatsapp

Description

Optional

☒ **Enable Row Level Security (RLS)** RECOMMENDED
Restrict access to your table by enabling RLS and writing Postgres policies.

 **Policies are required to query data**

You need to create an access policy before you can query data from this table. Without a policy, querying this table will return an empty array of results. You can create policies after saving this table.

[Documentation](#)

☐ **Enable Realtime**
Broadcast changes on this table to authorized subscribers

Columns

[About data types](#) [Import data from CSV](#)

Name ?	Type	Default Value ?	Primary
☰ telefone 🔗	# numeric ⬆	NULL	☒
☰ created_at 🔗	📅 timestampz ⬆	now() ☰	☐
☰ nome 🔗	T text ⬆	NULL ☰	☐ 1 ⚙️
☰ status 🔗	T text ⬆	NULL ☰	☐ 1 ⚙️

Cancel

Save Ctrl+S

Entendendo o papel de cada campo

telefone

Esse campo identifica unicamente cada contato. Como o projeto gira em torno da comunicação por WhatsApp, o telefone funciona como o identificador natural do usuário.

created_at

Esse campo registra quando a linha foi criada. Isso é útil para auditoria, organização de dados e rastreamento do momento em que o contato entrou no sistema.

nome

Armazena o nome do cliente ou lead.

status

Pode ser usado para indicar o estágio do contato dentro do processo, como “novo”, “em atendimento”, “qualificado” ou qualquer outra classificação útil ao fluxo.

Conectando o Supabase no n8n

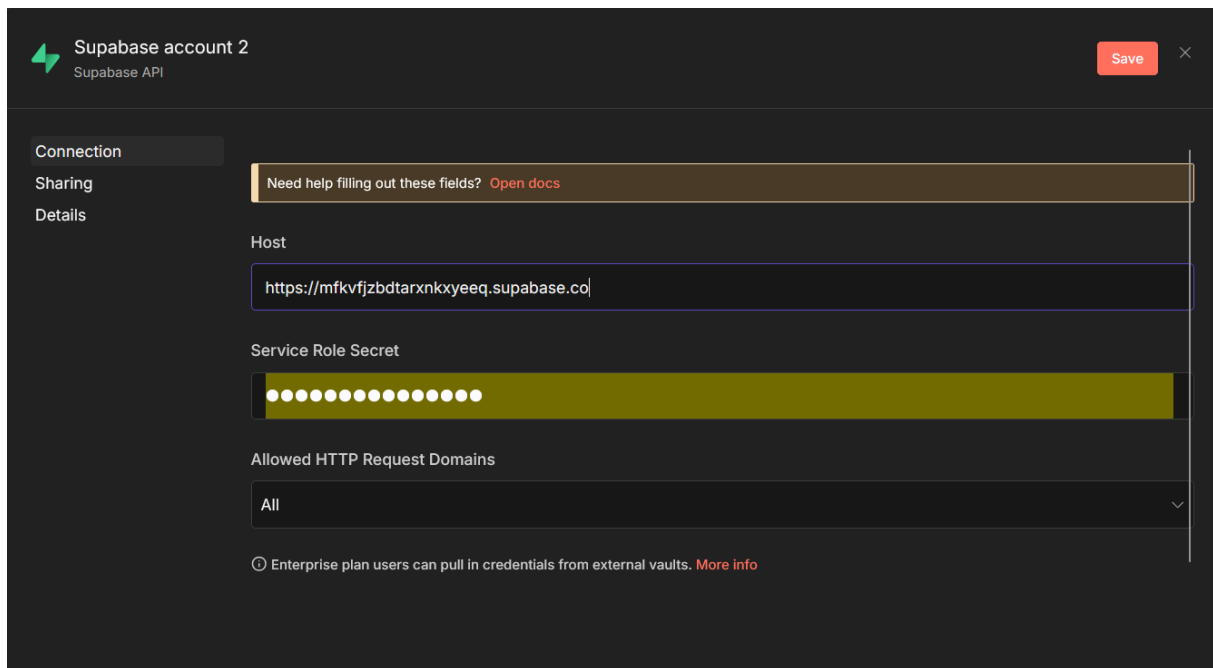
Depois de criar a tabela, o próximo passo é conectar o Supabase ao n8n por meio de uma credencial própria.

Para isso, dentro do Supabase, você deve abrir as configurações do projeto e localizar a **Data API URL**. Esse endereço será usado no campo de host da credencial no n8n. Em seguida, na seção de **API Keys**, você deve copiar a chave do tipo **Service Role Secret** e colá-la na credencial do Supabase dentro do n8n.

Campos principais da credencial

Host: Data API URL do projeto

Chave: Service Role Secret



Depois de salvar, o Supabase já estará conectado ao ambiente do n8n.

Criando a Vector Store no Supabase

Além de funcionar como banco de dados, o Supabase também pode ser usado como base vetorial para RAG. Para isso, é necessário criar a estrutura da **Vector Store** dentro do próprio banco.

O processo segue uma lógica simples:

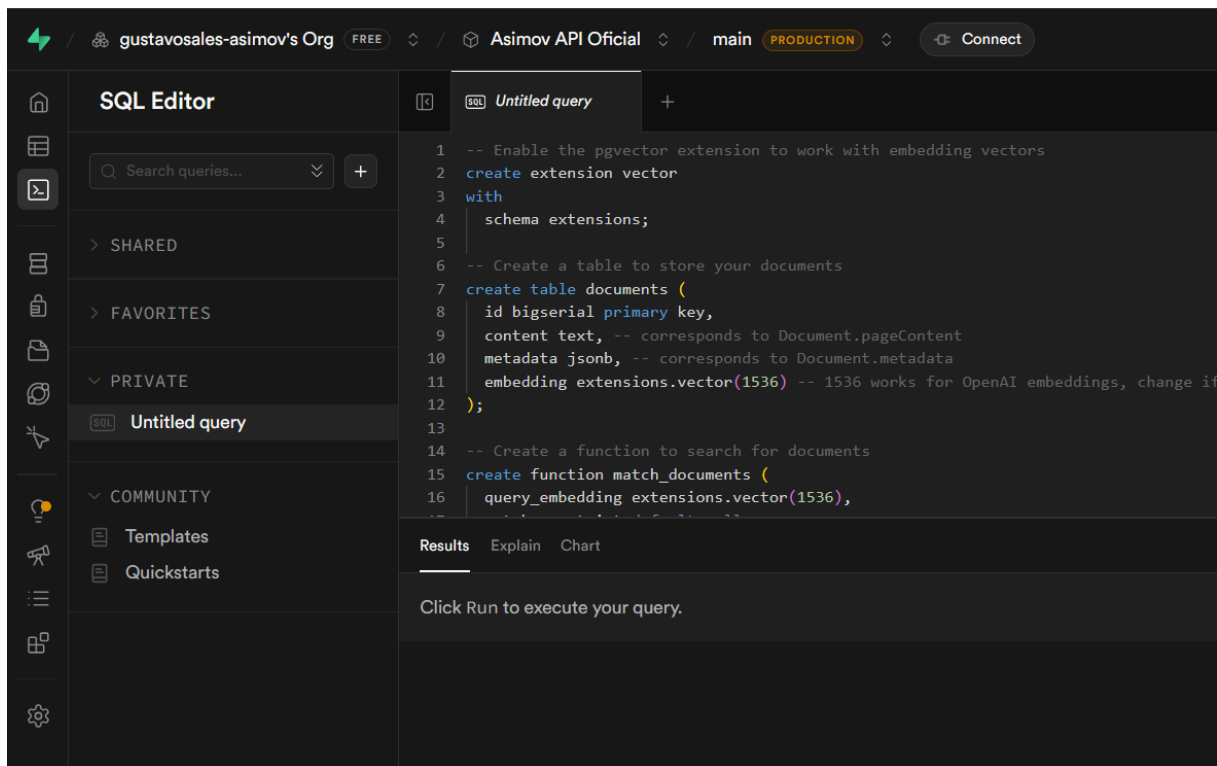
1. acessar a documentação do node **Supabase Vector Store** no n8n;
2. localizar o guia rápido de configuração <https://supabase.com/docs/guides/ai/langchain?database-method=sql>;
3. copiar o código SQL sugerido;

```
-- Enable the pgvector extension to work with embedding vectors
create extension vector
with
  schema extensions;

-- Create a table to store your documents
create table documents (
  id bigserial primary key,
  content text, -- corresponds to Document.pageContent
  metadata jsonb, -- corresponds to Document.metadata
  embedding extensions.vector(1536) -- 1536 works for OpenAI embeddings, change if needed
);
```

```
-- Create a function to search for documents
create function match_documents (
  query_embedding extensions.vector(1536),
  match_count int default null,
  filter jsonb DEFAULT '{}'
) returns table (
  id bigint,
  content text,
  metadata jsonb,
  similarity float
)
language plpgsql
as $$
#variable_conflict use_column
begin
  return query
  select
    id,
    content,
    metadata,
    1 - (documents.embedding <=> query_embedding) as similarity
  from documents
  where metadata @> filter
  order by documents.embedding <=> query_embedding
  limit match_count;
end;
$$;
```

4. abrir o **SQL Editor** do Supabase;
5. colar esse código;



6. executar o script.

Depois disso, será criada uma tabela chamada `documents`, que representará a base vetorial usada na recuperação de contexto do agente.

O que é uma Vector Store?

Uma Vector Store é uma estrutura usada para armazenar representações vetoriais de textos. Em aplicações com RAG, isso permite buscar trechos semanticamente parecidos com a pergunta do usuário, em vez de depender apenas de busca por palavra exata.

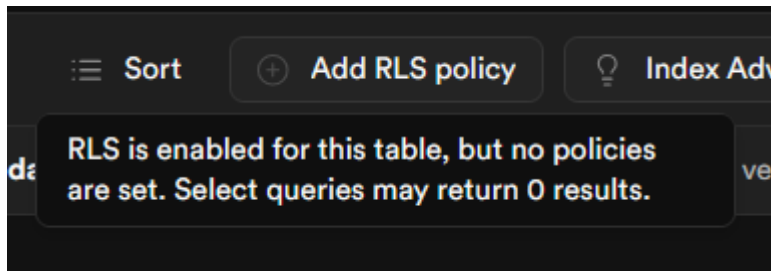
Você não precisa dominar todos os detalhes matemáticos dessa estrutura para usar a ferramenta. O importante, neste momento, é entender a função prática: ela será a base que permitirá ao agente consultar informações relevantes de forma contextual.

Configurando as políticas da base no Supabase

Depois de criar as tabelas, é necessário configurar as políticas de acesso. Esse passo é importante para que o n8n consiga operar corretamente sobre essas estruturas.

O processo é feito tanto para a tabela principal quanto para a tabela `documents` da Vector Store:

1. ativar a proteção da tabela (RLS policy);



2. criar uma nova policy;
3. dar um nome para essa política;
4. marcar a política para **All**;
5. usar `true` como condição;
6. salvar.

×

Create a new Row Level Security policy

⌵

Policy Name

n8n

Table on clause

public.documents

Policy Behavior as clause

Permissive

Policy Command for clause

☐ SELECT

☐ INSERT

☐ UPDATE

☐ DELETE

☒ ALL

Target Roles to clause

Defaults to all (public) roles if none selected

🔒

USE OPTIONS ABOVE TO EDIT

1

create policy "n8n"

2

on "public"."documents"

3

as PERMISSIVE

4

for ALL

5

to public

6

using (

7

true

8

) with check (

9

-- Provide a SQL expression for the with check statement

10

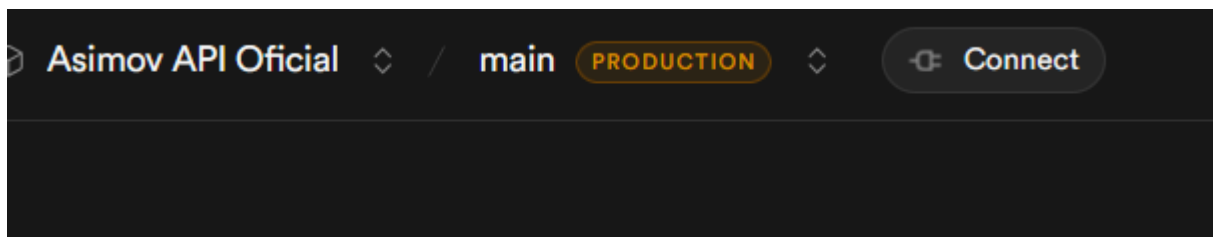
);

Por que isso é necessário?

Sem uma política adequada, o n8n pode não conseguir ler ou escrever nessas tabelas. Em termos práticos, isso significa que o fluxo pode falhar ao tentar salvar clientes, consultar documentos ou atualizar registros.

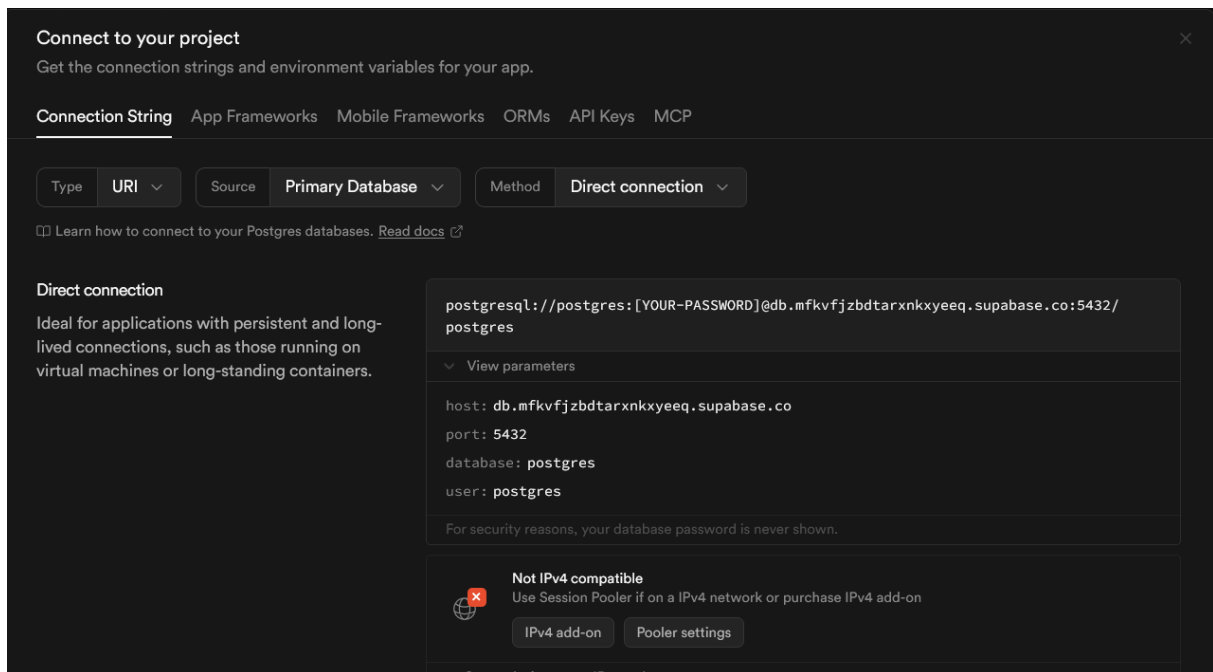
Conectando o Postgres no n8n

Além da conexão direta com o Supabase, também é necessário conectar o **Postgres** no n8n. Nesse projeto, ele será usado como memória do agente.



Para isso, dentro do Supabase, você deve abrir a seção **Connect** e copiar os parâmetros de transação, como:

- host
- porta
- usuário
- database
- senha do projeto



No n8n, crie uma credencial do tipo **Postgres** e preencha esses campos.

Conectando o Redis

O próximo serviço é o **Redis**. Para essa etapa, você deve criar uma conta no site oficial do Redis, abrir uma nova database e selecionar o plano gratuito. Também é recomendável usar uma região próxima, como São Paulo, para manter boa resposta de rede. Você também pode usar o Redis que é criado na plataforma do Railway.

Depois que a database estiver criada, você precisará copiar três informações principais:

- o host, usando o endpoint público até .com;
- a porta;
- a senha do usuário padrão.

No n8n, crie a credencial correspondente e preencha esses dados. Depois disso, o Redis estará conectado com sucesso ao ambiente do n8n.

Escolhendo o modelo de IA

Com a base de dados e os serviços auxiliares conectados, falta configurar o modelo de linguagem que será usado pelo agente.

Nesse ponto, você pode usar o modelo que preferir. Aqui usaremos a **OpenAI**, mas a lógica geral também se aplica a outros provedores compatíveis, desde que o node ou a credencial exista no n8n.

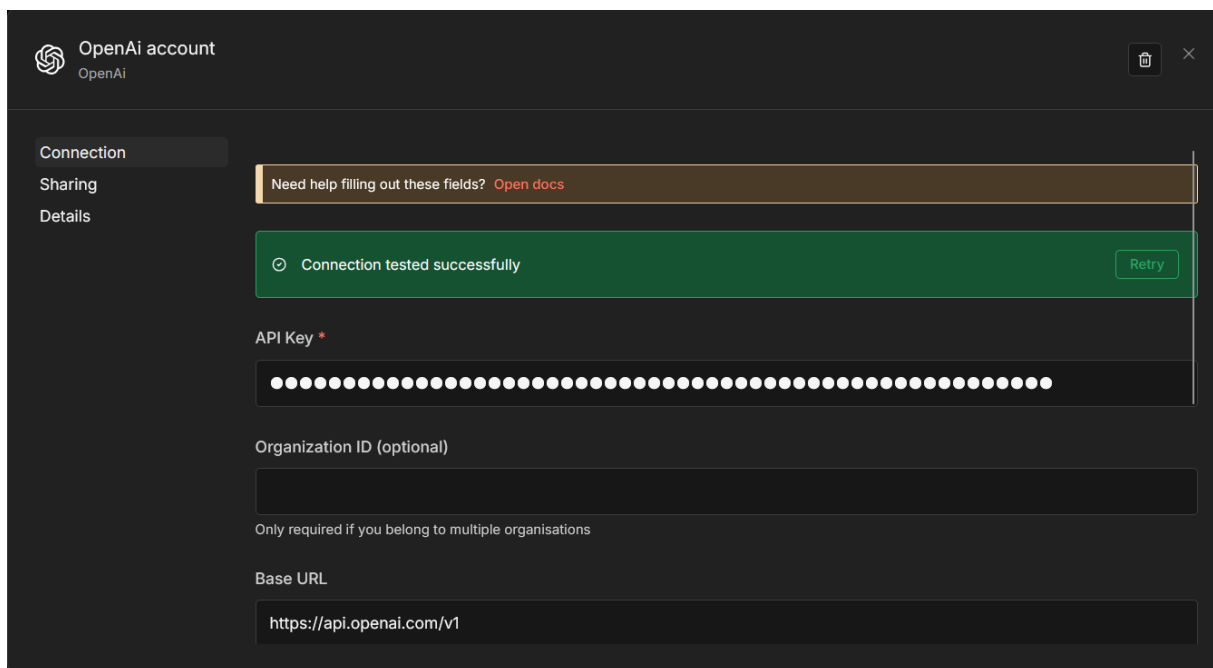
A ideia principal é simples: o n8n precisa de uma credencial válida para conseguir chamar um modelo de IA durante a execução do fluxo.

Criando a credencial da OpenAI

Para usar a OpenAI, o processo começa no painel da plataforma, dentro da área de API Keys. Lá, você deve criar uma nova chave secreta. Depois que a chave for gerada, é fundamental copiá-la e guardá-la imediatamente, porque ela não ficará visível novamente depois.

Em seguida, no n8n:

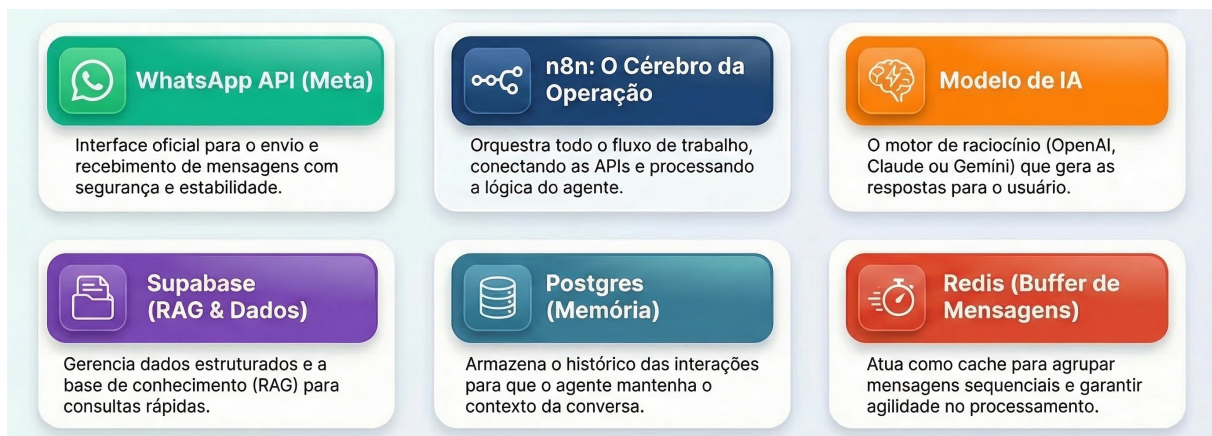
1. crie uma nova credencial;
2. selecione o tipo **OpenAI**;
3. cole a chave no campo de API Key;
4. salve.



Visão geral da infraestrutura criada

Ao final desta etapa, o projeto passa a contar com uma base muito mais completa. Em vez de depender apenas da conexão entre Meta e n8n, agora o agente também tem apoio de serviços comple-

mentares para dados, memória, contexto e inferência.



Neste ponto, a estrutura do projeto já está pronta para começar a construir o agente de fato. A integração com o WhatsApp já existe, e agora o ambiente também conta com banco de dados, memória, base vetorial, cache e modelo de linguagem.

Em outras palavras, o sistema já tem os principais blocos necessários para sair de uma automação simples e evoluir para um agente mais completo, com contexto, armazenamento e capacidade de resposta inteligente.

08. Estruturando o fluxo do agente no n8n

Com a API do WhatsApp configurada, o webhook ativo e as credenciais conectadas, chegou o momento de construir o fluxo principal do agente dentro do n8n.

Neste capítulo, você vai organizar a lógica que recebe as mensagens do WhatsApp, identifica quem enviou, trata diferentes tipos de conteúdo, registra o contato no banco de dados, agrupa mensagens curtas enviadas em sequência e, por fim, prepara tudo para o agente responder de forma mais natural.

Esse é um dos capítulos mais importantes da apostila, porque ele transforma a base técnica criada até agora em uma automação com comportamento inteligente.

O ponto de partida do fluxo

O fluxo parte do webhook que já foi configurado anteriormente. Esse webhook passa a ser o gatilho principal da automação: sempre que uma nova mensagem chega ao número conectado, o n8n recebe o evento e inicia o workflow.

A partir desse ponto, o fluxo precisa responder a algumas perguntas fundamentais:

- a mensagem veio de um cliente ou foi enviada pelo próprio número da API?
- o contato já existe no banco de dados?
- o conteúdo recebido é texto, áudio ou imagem?
- a mensagem foi enviada sozinha ou faz parte de uma sequência?
- como transformar esse conteúdo em uma entrada padronizada para o agente?

Essas perguntas definem a estrutura do fluxo.

Evitando que o fluxo processe mensagens enviadas pelo próprio sistema

Um detalhe importante da API Oficial do WhatsApp é que o webhook pode ser disparado não apenas quando um cliente envia uma mensagem, mas também quando o próprio sistema envia uma mensagem de saída.

Se esse caso não for tratado, o fluxo pode acabar processando mensagens que ele mesmo enviou, o que gera comportamento incorreto e pode até criar loops.

Como identificar isso

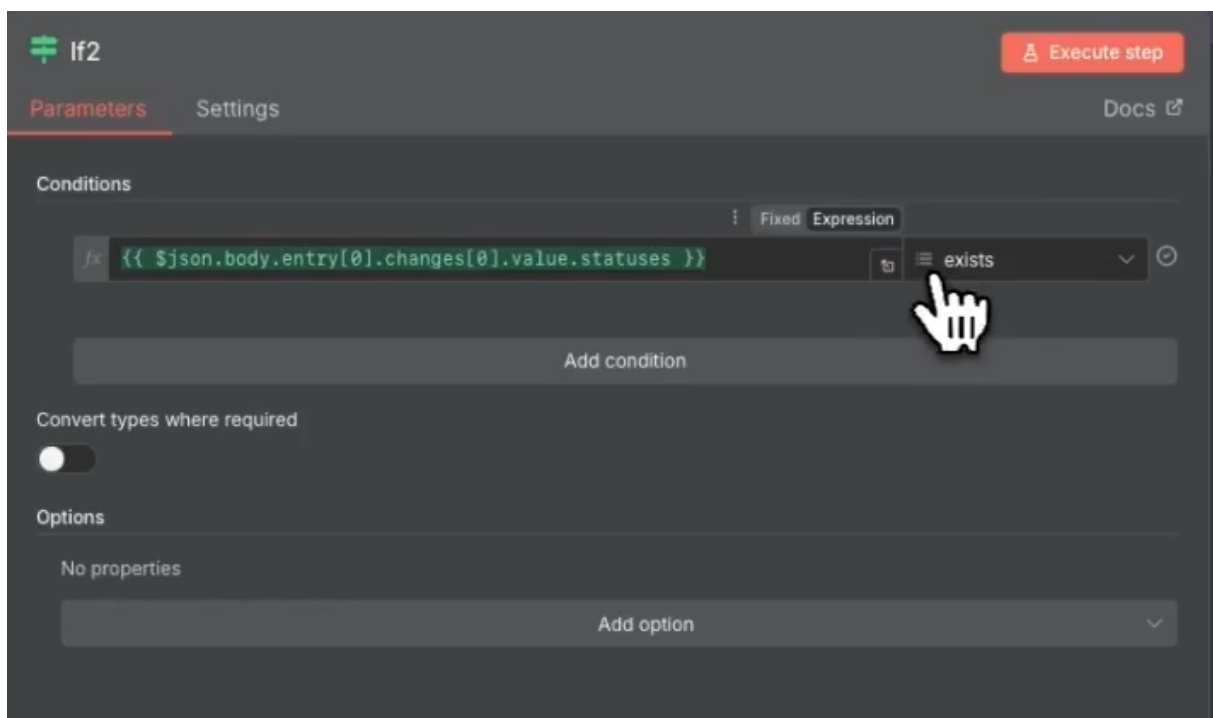
Quando o evento recebido pelo webhook corresponde a uma mensagem enviada pela própria conta, o JSON costuma trazer um array chamado `status`. Já quando a mensagem vem de um lead ou cliente, o JSON traz um array de `messages`.

Por isso, a primeira verificação do fluxo é simples:

- se o array `status` existir, o fluxo deve parar;
- se ele não existir, o fluxo pode continuar.

Usando um node If

Para fazer isso, adicione um node If logo após o webhook e configure a condição para verificar se o array `status` existe.



Se a condição for verdadeira, você pode ligar o fluxo True a um node de encerramento simples, como **No Operation, do nothing**. Esse caminho serve apenas para impedir que o fluxo avance.

Organizando os dados da mensagem recebida

Depois da verificação inicial, o próximo passo é extrair do webhook as informações principais da mensagem. Para isso, o ideal é usar um node `Edit Field (Set)` logo no começo do fluxo.

A ideia aqui é transformar um JSON grande e difícil de navegar em variáveis simples, fáceis de reutilizar ao longo da automação.

Variáveis principais

As variáveis organizadas nesse primeiro `Edit Field (Set)` são:

- **nome:** Recebe o nome do perfil que enviou a mensagem.
- **telefone:** Recebe o identificador do número de WhatsApp, normalmente o `wa_id`.
- **mensagem_tipo:** Informa se a mensagem recebida é texto, áudio ou imagem.
- **mensagem_texto:** Armazena o conteúdo textual, quando a mensagem for do tipo texto.
- **mensagem_audio:** Armazena o ID do áudio recebido, quando o tipo for áudio.
- **mensagem_imagem:** Armazena o ID da imagem recebida, quando o tipo for imagem.

The screenshot shows the 'Edit Fields2' configuration window with the 'Parameters' tab selected. The 'Mode' is set to 'Manual Mapping'. Under 'Fields to Set', four fields are configured:

- nome** (String): Mapped to `{{ $json.body.entry[0].changes[0].value.contacts[0].profile.name }}`. Preview: Enzo Panarotto
- telefone** (String): Mapped to `{{ $json.body.entry[0].changes[0].value.contacts[0].wa_id }}`. Preview: 555497076306
- mensagem_tipo** (String): Mapped to `{{ $json.body.entry[0].changes[0].value.messages[0].type }}`. Preview: text
- mensagem_texto** (String): Mapped to `{{ $json.body.entry[0].changes[0].value.messages[0].text.body }}`. Preview: Oi

Esse padrão facilita muito o restante do fluxo, porque todas as próximas etapas passam a trabalhar com nomes simples e consistentes.

Verificando se o contato já existe no Supabase

Depois de organizar os dados recebidos, o próximo passo é verificar se aquele telefone já existe na tabela de clientes do Supabase.

Essa etapa é importante porque o sistema precisa diferenciar:

- um contato novo, que ainda não existe no banco;
- um contato recorrente, que já foi registrado antes.

Fazendo a busca

Use um node do Supabase com operação de busca(**Get many rows**), filtrando pela coluna telefone. O filtro deve comparar o valor salvo na variável telefone com os registros da tabela de clientes.

Get many rows

Execute step

Parameters Settings Docs

Supabase Asimov

Use Custom Schema

Resource

Row

Operation

Get Many

Table Name or ID

Clientes WhatsApp

Return All

Filter

Build Manually

Must Match

Any Filter

Filters

Field Name or ID

telefone - (number)

Condition

Equals

Field Value

{{ \$json.telefone }}

555497076306

Add Condition

Ativando Always Output Data

É importante ativar a opção:

Always Output Data

Essa configuração garante que o fluxo continue mesmo quando nenhum registro for encontrado. Sem isso, a automação pode parar justamente no momento em que deveria criar um novo lead.

Criando o lead quando ele ainda não existe

Depois da busca, adicione um novo If para testar se o telefone foi encontrado.

- Se já existir, o fluxo segue normalmente.
- Se não existir, o fluxo cria um novo registro no Supabase(**Create a row**).

Campos enviados na criação

Na criação da nova linha, envie:

- telefone
- nome

Os demais campos, como status e created_at, podem ficar para etapas posteriores ou ser preenchidos automaticamente pelo banco.

 **Get many rows** Execute step

Parameters

Settings

Docs

Supabase Asimov

Use Custom Schema

☐

Resource

Row

Operation

Get Many

Table Name or ID

Cientes WhatsApp

Return All

☒

Filter

Build Manually

Must Match

Any Filter

Filters

Field Name or ID

telephone - (number)

Condition

Equals

Field Value

fx

{{ \$json.telefone }}

555497076306

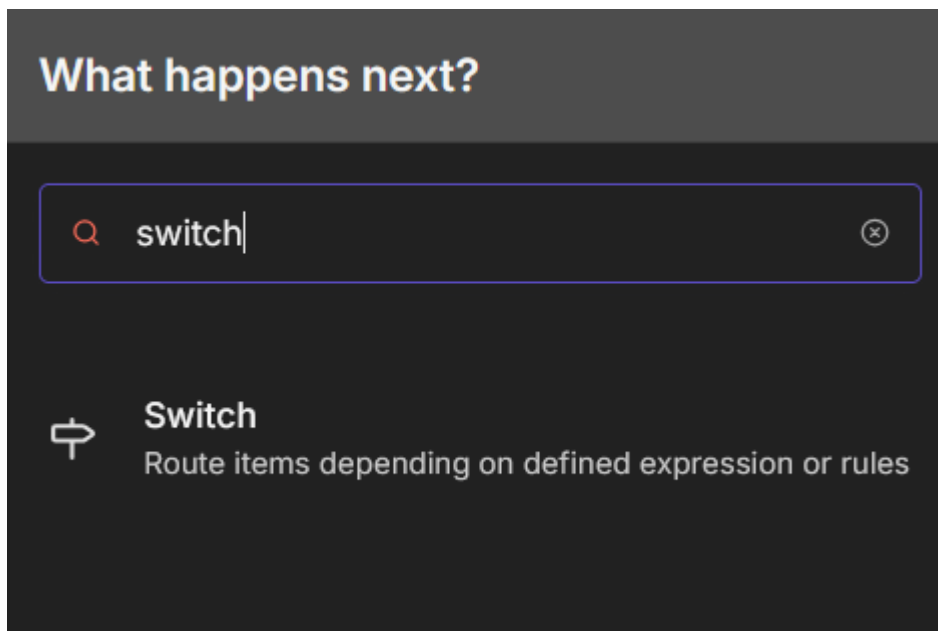
Lógica dessa etapa

Essa parte do fluxo resolve um problema comum em automações de atendimento: a necessidade de registrar automaticamente novos contatos sem depender de preenchimento manual.

Em outras palavras, o banco de dados vai crescendo conforme novas pessoas entram em contato com o número.

Separando o fluxo por tipo de mensagem

Depois de tratar a existência do lead, o próximo grande passo é identificar o tipo de mensagem recebida e direcionar cada caso para um caminho adequado. Para isso, use um node `Switch` na linha `true`.



A variável usada nessa decisão é `mensagem_tipo`.

Três caminhos principais

O `Switch` deve criar três saídas:

- `text`
- `audio`
- `image`

Cada uma dessas saídas representa um tratamento diferente.

Tratando mensagens de texto

Quando a mensagem recebida já é texto, o conteúdo está pronto para ser usado no agente. Nesse caso, o fluxo pode seguir sem necessidade de conversão adicional.

Por que usar um node No Operation?

Mesmo que não haja tratamento especial para o texto, é útil colocar um node No Operation nesse caminho. Isso ajuda a manter o padrão visual do fluxo e também facilita o uso de variáveis nas próximas etapas.

Na prática, esse node funciona como uma passagem neutra: ele não transforma nada, apenas deixa o fluxo seguir mantendo a estrutura consistente.

Tratando mensagens de imagem

Quando o tipo da mensagem for imagem, o fluxo precisa fazer dois passos antes de passar o conteúdo para a IA:

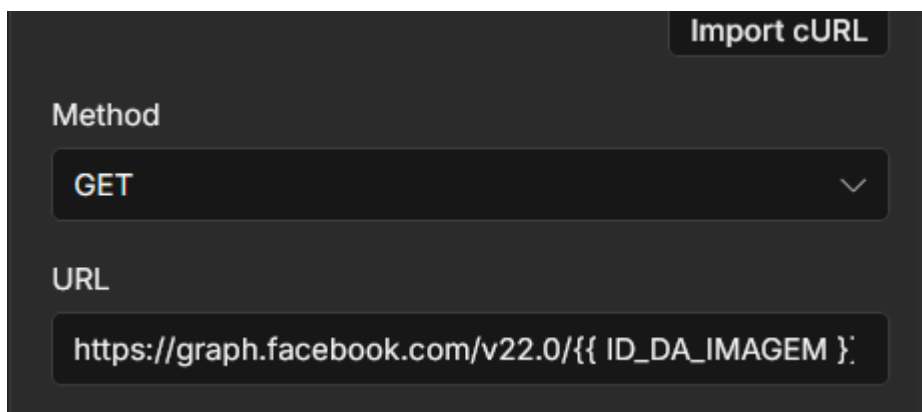
1. buscar a URL temporária da mídia na API da Meta;
2. baixar o arquivo da imagem.

Buscando a URL da mídia

Use um node HTTP Request com a URL da API Graph da Meta. A requisição deve usar o ID salvo em `mensagem_imagem`.

A estrutura geral fica assim:

`https://graph.facebook.com/v22.0/{{ID_DA_IMAGEM}}`



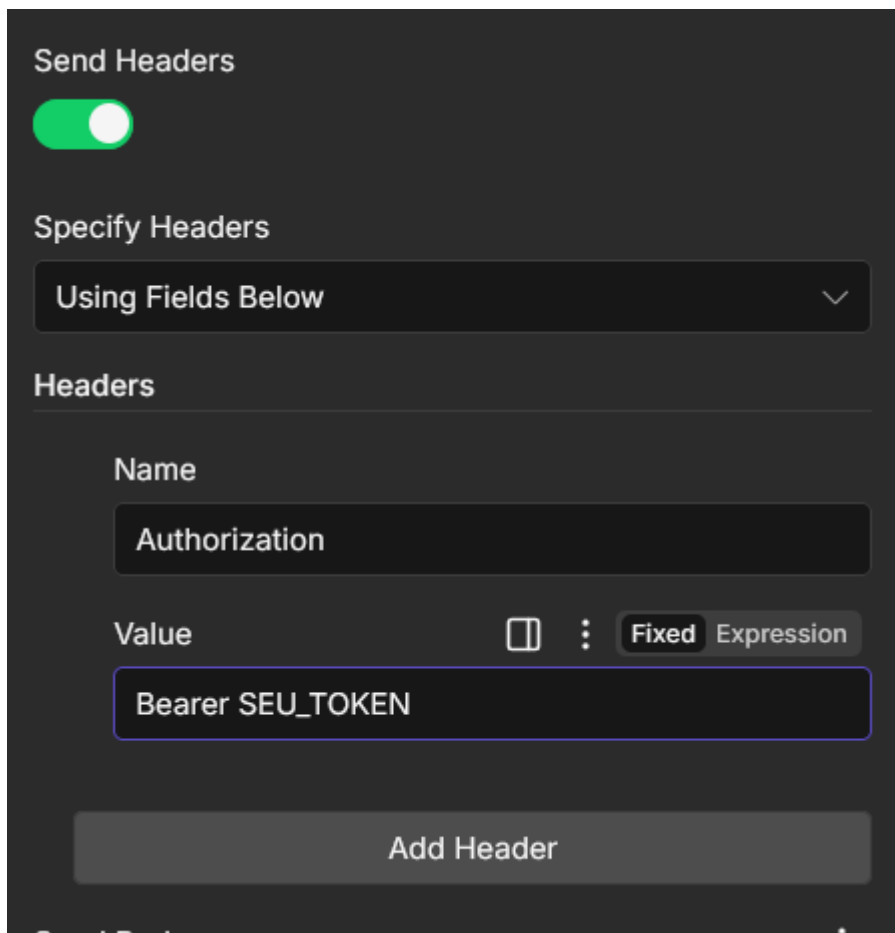
The image shows a configuration window for an HTTP Request node. At the top right is a button labeled "Import cURL". Below it, the "Method" is set to "GET" in a dropdown menu. The "URL" field contains the text "https://graph.facebook.com/v22.0/{{ ID_DA_IMAGEM }}".

Essa chamada não retorna a imagem em si. Ela retorna a URL temporária usada para fazer o download do arquivo.

Autenticação

Essa requisição precisa incluir o header de autorização com o token permanente da Meta:

Authorization: Bearer SEU_TOKEN



The screenshot shows a configuration panel for sending headers. At the top, a toggle switch labeled 'Send Headers' is turned on. Below it, a dropdown menu labeled 'Specify Headers' is set to 'Using Fields Below'. Under the 'Headers' section, there is a table with two columns: 'Name' and 'Value'. The 'Name' column contains the text 'Authorization'. The 'Value' column contains the text 'Bearer SEU_TOKEN'. To the right of the 'Value' input, there are icons for a list, a vertical ellipsis, and two buttons labeled 'Fixed' and 'Expression'. At the bottom of the panel, there is a button labeled 'Add Header'.

Sem isso, a API não libera o acesso à mídia.

Fazendo o download da imagem

Depois de obter a URL, use um segundo HTTP Request para baixar o arquivo binário da imagem. Esse segundo request também precisa do mesmo header de autorização.

Method

GET

URL

{{ \$json.urls }}

Authentication

None

Send Query Parameters

☐

Send Headers

☒

Specify Headers

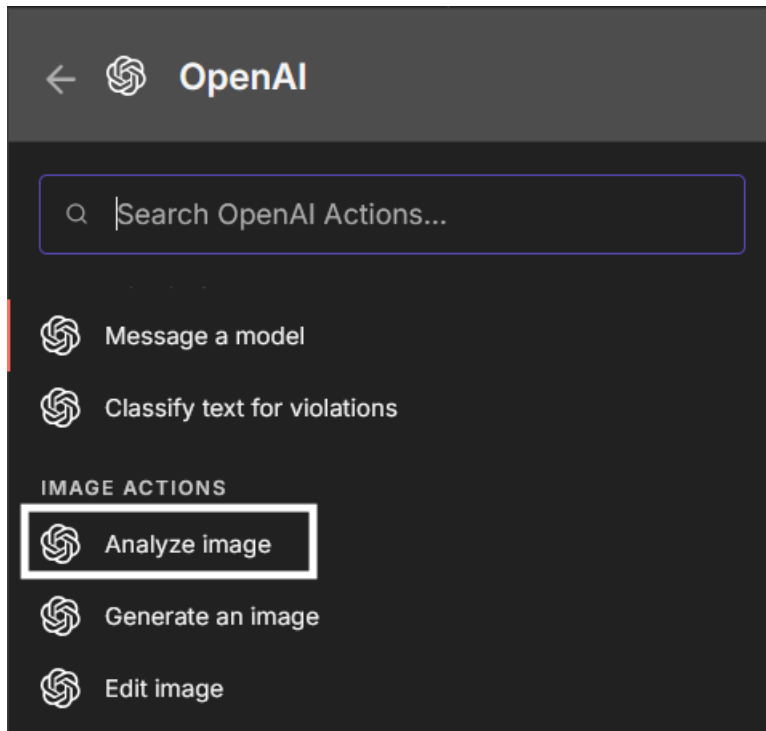
Using Fields Below

Headers

Name	Value
Authorization	Bearer SEU_TOKEN

Enviando a imagem para análise

Com o arquivo binário em mãos, o próximo passo é usar um node da OpenAI para análise de imagem. A tarefa pode ser algo simples como: **Descreva esta imagem de forma detalhada.**



O input deve ser configurado como arquivo binário. A resposta gerada por esse node será usada como representação textual da imagem para o agente.

The image shows a configuration interface for a WhatsApp agent, specifically for image analysis. The interface is dark-themed and contains several sections with dropdown menus and text inputs.

- Credential to connect with:** A dropdown menu set to "OpenAi account" with a small edit icon to its right.
- Resource:** A dropdown menu set to "Image".
- Operation:** A dropdown menu set to "Analyze Image".
- Model:** A section with two dropdowns: "From list" and "GPT-4O".
- Text Input:** A text area containing the prompt "Descreva esta imagem de forma detalhada." with a small icon at the bottom right.
- Input Type:** A dropdown menu set to "Binary File(s)".
- Input Data Field Name:** A text input field containing the value "data".
- Help text:** Below the input field, it says "The name of the input field containing the binary file data to be processed".

Tratando mensagens de áudio

O fluxo de áudio é parecido com o da imagem, mas o destino final é diferente: em vez de análise visual, o objetivo é transcrever a fala para texto.

Buscando a URL do áudio

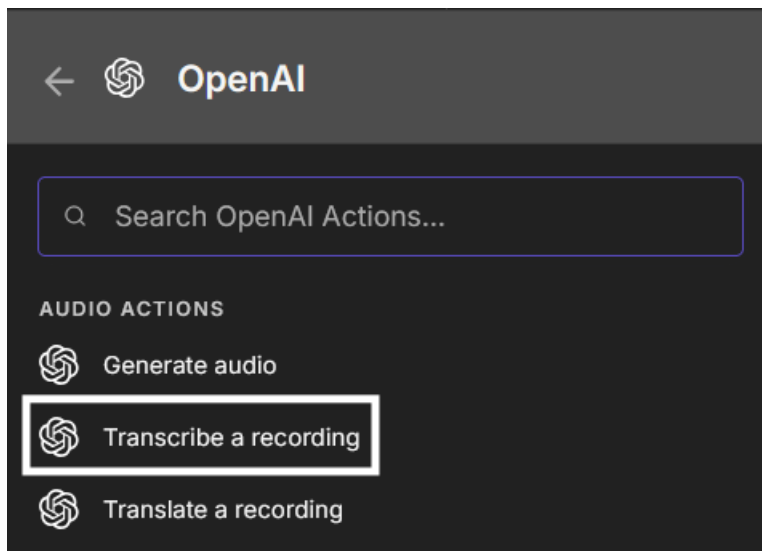
Use um HTTP Request para consultar a mídia da Meta usando o ID salvo em `mensagem_audio`. A URL base é a mesma da imagem, mudando apenas o ID utilizado.

Baixando o arquivo de áudio

Depois de obter a URL, faça um segundo request para baixar o arquivo binário. O processo também exige o token no header de autorização.

Transcrevendo com a OpenAI

Com o áudio disponível, use um node da OpenAI com operação de transcrição.



O campo de entrada deve apontar para o arquivo binário recebido no request anterior. Ao final dessa etapa, o áudio passa a existir como texto, pronto para ser tratado como entrada do agente.

Criando uma única variável de mensagem

Até aqui, o fluxo já consegue lidar com três situações:

- texto puro;
- imagem convertida em descrição textual;
- áudio convertido em transcrição.

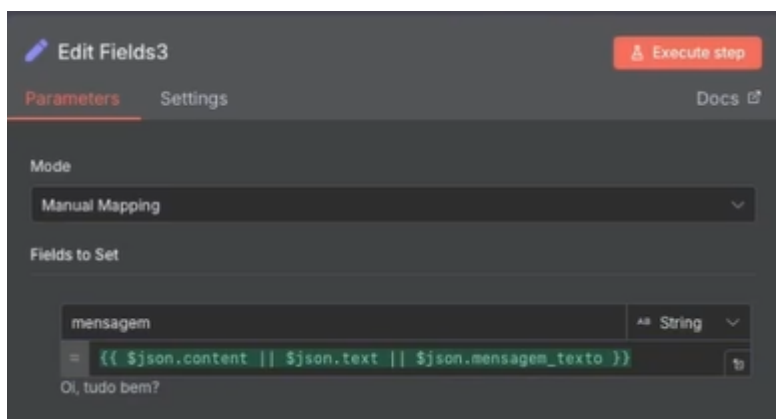
Agora é hora de padronizar tudo isso em uma única variável. Para isso, crie um novo `Edit Field(Set)` com um campo chamado `mensagem`.

A lógica é simples:

- se vier do caminho da imagem, use a descrição gerada pela IA;
- se vier do caminho do áudio, use a transcrição;
- se vier do caminho do texto, use o texto recebido diretamente.

Exemplo conceitual

```
{{ $json.content || $json.text || $json.mensagem_texto }}
```



Essa expressão concentra tudo em uma única variável chamada `mensagem`, independentemente da origem original do conteúdo. Isso simplifica bastante o restante da automação.

Usando o Redis para armazenar mensagens em curto prazo

Muitas vezes o usuário não envia sua pergunta inteira de uma vez. Em vez disso, ele manda várias mensagens seguidas, como:

- “oi”
- “tudo bem?”
- “quero saber o preço”

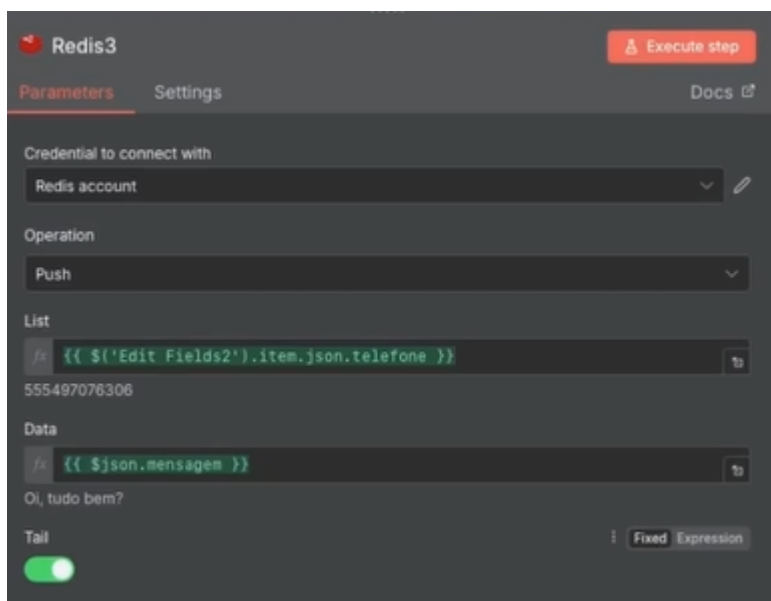
Se o agente responder após cada uma dessas partes, a conversa fica ruim. Para resolver isso, o fluxo usa o Redis como buffer temporário de mensagens.

Guardando a mensagem na lista

Use um node do Redis para adicionar a variável mensagem a uma lista identificada pelo telefone. A chave da lista é o próprio número do contato.

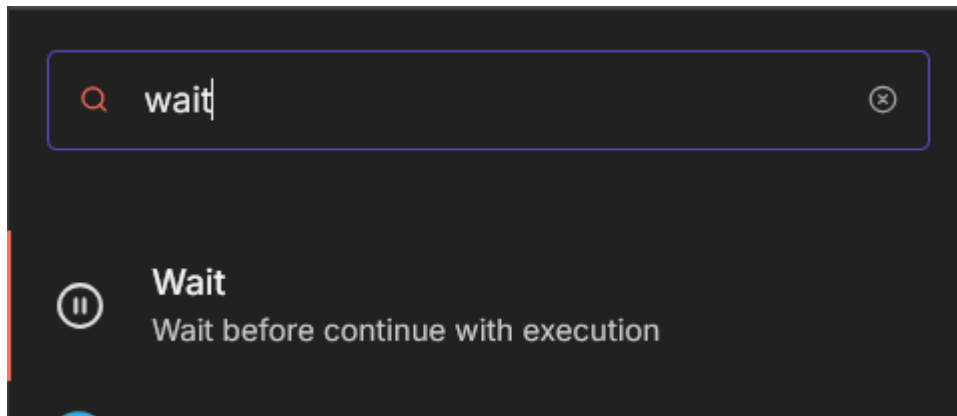
Ativando a opção Tail

É importante ativar a opção que adiciona os itens ao final da lista. Isso garante que a ordem das mensagens seja preservada e que a última mensagem fique corretamente registrada como o item mais recente.



Aguardando um tempo para capturar mensagens “picotadas”

Depois de guardar a mensagem no Redis, o fluxo usa um node Wait para aguardar alguns segundos antes de continuar. Um valor como **10 segundos** funciona bem para esse objetivo.



Por que esperar?

Esse tempo serve para verificar se o usuário vai mandar mais alguma coisa logo em seguida.

Na prática, o fluxo dá uma pequena pausa antes de responder, para decidir se já deve agir ou se ainda precisa esperar a continuação da mensagem.

Descobrimo se essa execução é a mais recente

Depois do `Wait`, o fluxo busca no Redis todas as mensagens armazenadas para aquele telefone. Em seguida, ele compara:

- a última mensagem da lista;
- a mensagem que foi processada nesta execução específica.

The screenshot shows the configuration for a 'Redis4' step. At the top right is an 'Execute step' button. Below it are tabs for 'Parameters' (selected) and 'Settings', and a 'Docs' link. The 'Parameters' section includes: 'Credential to connect with' set to 'Redis account'; 'Operation' set to 'Get'; 'Name' set to 'propertyName'; 'Key' set to a JSON path expression `{{ $('Edit Fields2').item.json.telefone }}`; a value `555497076306`; 'Key Type' set to 'Automatic'; and 'Options' set to 'No properties' with an 'Add option' button below it.

O que essa comparação significa?

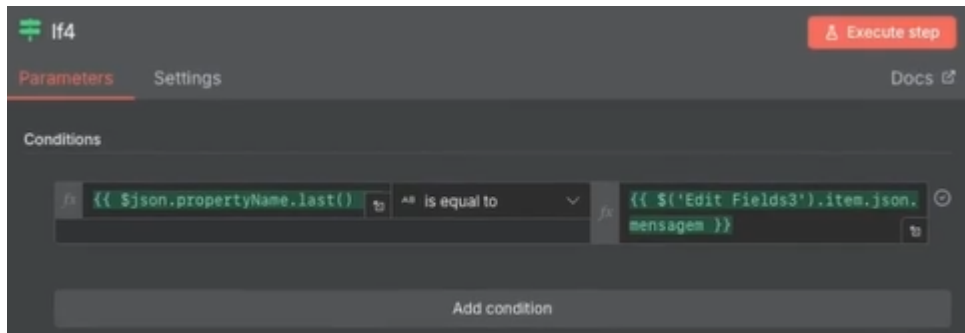
Se a última mensagem da lista for igual à mensagem desta execução, isso significa que nenhuma mensagem nova chegou durante a espera. Portanto, esta execução pode continuar.

Se forem diferentes, significa que uma mensagem mais nova já chegou e gerou outra execução do webhook. Nesse caso, esta execução atual está desatualizada e deve ser encerrada.

Encerrando execuções antigas

Para isso, use um If:

- se a comparação for falsa, ligue a um No Operation;
- se for verdadeira, continue o fluxo.



Essa é uma solução elegante para evitar respostas fragmentadas ou repetidas.

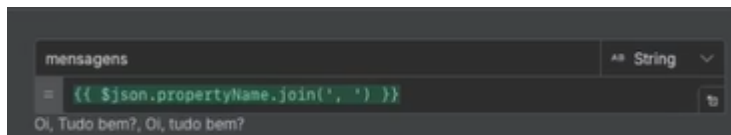
Agrupando as mensagens em uma única entrada

Quando a execução atual for confirmada como a mais recente, o próximo passo é juntar todas as mensagens armazenadas no Redis em um único texto. Para isso, crie um novo `Edit Field(Set)` com um campo como `mensagens`.

A ideia é usar um `join` sobre o array retornado do Redis, separando cada item com vírgula e espaço.

Exemplo conceitual

```
{{ $json.propertyName.join(', ') }}
```



O resultado será algo como:

oi, tudo bem?, quero saber o preço

Esse texto consolidado é muito melhor para o agente do que processar várias entradas quebradas.

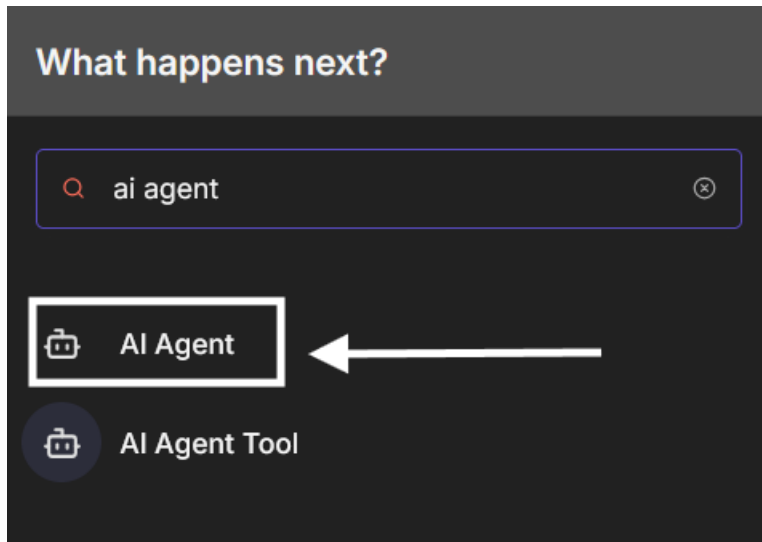
Limpendo o buffer do Redis

Depois de agrupar as mensagens, é importante apagar a lista daquele telefone no Redis. Caso isso não seja feito, mensagens antigas permanecerão no buffer e atrapalharão as próximas interações.

Para isso, use o node **Delete a key from Redis**, usando a mesma chave baseada no telefone.

Passando a mensagem consolidada para o agente principal

Agora sim o fluxo está pronto para acionar o agente principal. Use um node AI Agent e configure a entrada do usuário com a variável mensagens.



Campo userMessage

No campo de mensagem do usuário, passe o texto consolidado vindo do Redis.

Campo systemMessage

No campo de sistema, você poderá definir depois o comportamento do agente, como:

- quem ele é;
- qual papel ele exerce;
- como deve responder;
- quais regras precisa seguir.

Neste momento, o mais importante é deixar a estrutura pronta para receber essas instruções.

Criando um segundo agente para formatar a resposta

Depois do agente principal gerar a resposta, o fluxo usa um segundo agente com uma função bem específica: **quebrar a resposta em mensagens menores e mais naturais para o WhatsApp.**

Essa etapa não é obrigatória, mas melhora muito a experiência da conversa.

Por que fazer isso?

Se a IA responder com um único bloco grande de texto, a conversa fica artificial. Em chats, principalmente no WhatsApp, é mais natural receber respostas divididas em partes curtas.

Papel desse segundo agente

Esse agente não deve criar conteúdo novo. Entre as regras mais úteis para esse agente, estão:

- dividir a resposta em duas ou mais mensagens sempre que possível;
- inserir exatamente duas quebras de linha entre blocos;
- nunca alterar o conteúdo da resposta original;
- nunca responder por conta própria.

Essa etapa serve apenas para formatação. Abaixo está o System Message completo para o agente formatador:

Você é um agente formatador de mensagens para WhatsApp. Sua única função é dividir a mensagem recebida em múltiplas mensagens curtas, naturais e bem estruturadas. Siga as
↪ regras abaixo com rigor:

1. Sempre que possível, divida a mensagem original em 2 ou mais mensagens menores.
2. A cada nova mensagem gerada, aplique exatamente **duas** quebras de linha reais (\n\n) no
↪ final do bloco. Nunca ultrapasse esse número de quebras de linha.
3. Mantenha frases inteiras e compreensíveis em cada mensagem. **Nunca** divida frases no meio
↪ ou quebre a fluidez.
4. Se houver listas, bullets ou tópicos numerados, **não** quebre ou interrompa a estrutura da
↪ lista. Trate cada lista como uma única mensagem.

NUNCA JAMAIS ALTERE O CONTEUDO DA MENSAGEM!

5. **NUNCA** gere ou altere o conteúdo da mensagem original. Seu trabalho é apenas dividir e
↪ formatar a mensagem recebida para nós as enviarmos de forma natural no WhatsApp, **NUNCA**
↪ altere o conteúdo original da mensagem recebida.
6. Sempre divida as mensagens utilizando \n\n ao final de cada uma delas.

Exemplo

Q: Oi! Tudo bem, e com você? Como posso ajudar?

A: Oi!

Tudo bem, e com você?

Como posso ajudar?

Q: Boa noite! Tudo certo? Como você está?

A: Boa noite!

Tudo certo?

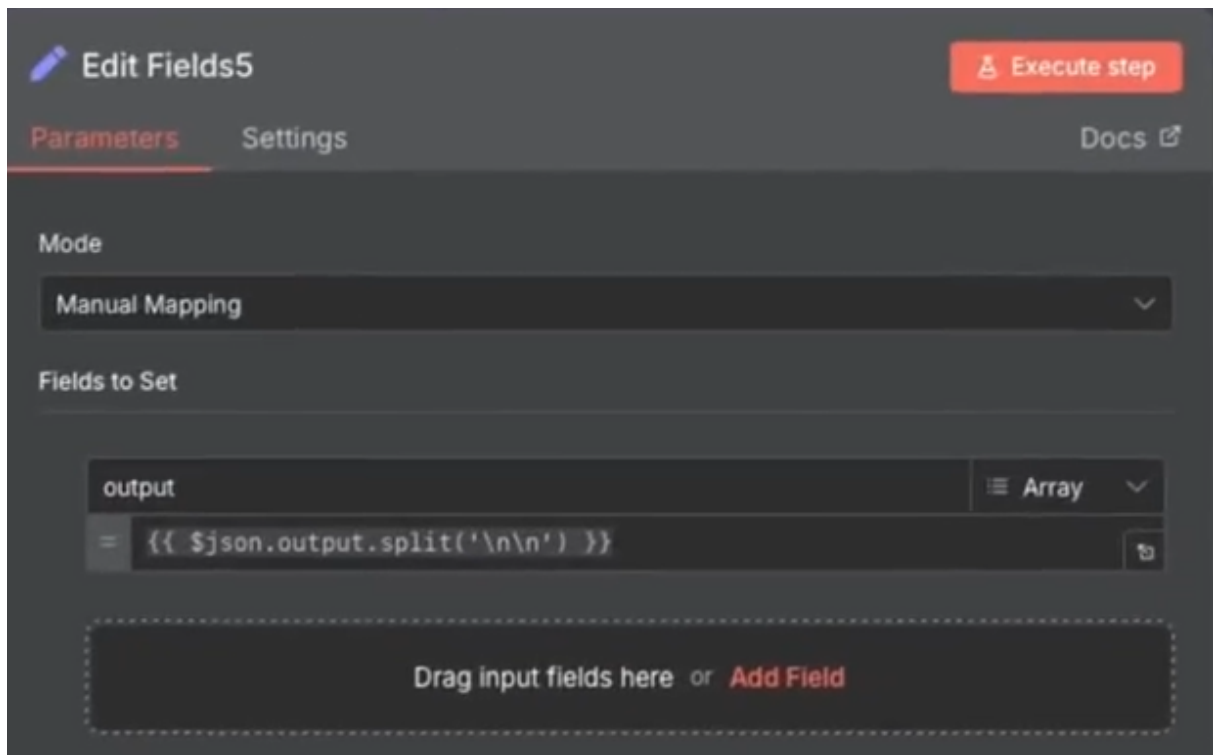
Como você está?

Transformando a resposta formatada em lista

Depois que o segundo agente devolver o texto com divisões por quebras de linha, crie um `Edit Field(Set)` para transformar esse conteúdo em um array.

A lógica é dividir a saída usando duas quebras de linha como separador.

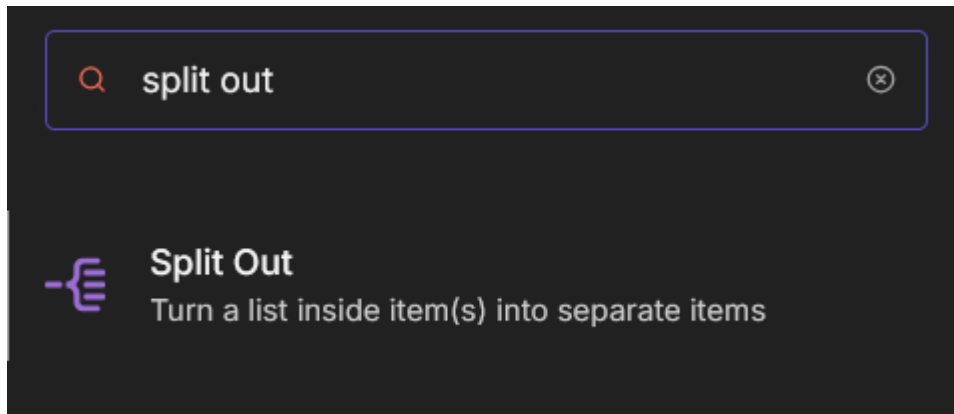
```
{{ $json.output.split('\n\n') }}
```



O resultado será uma lista com cada submensagem pronta para envio individual.

Separando a lista em itens individuais

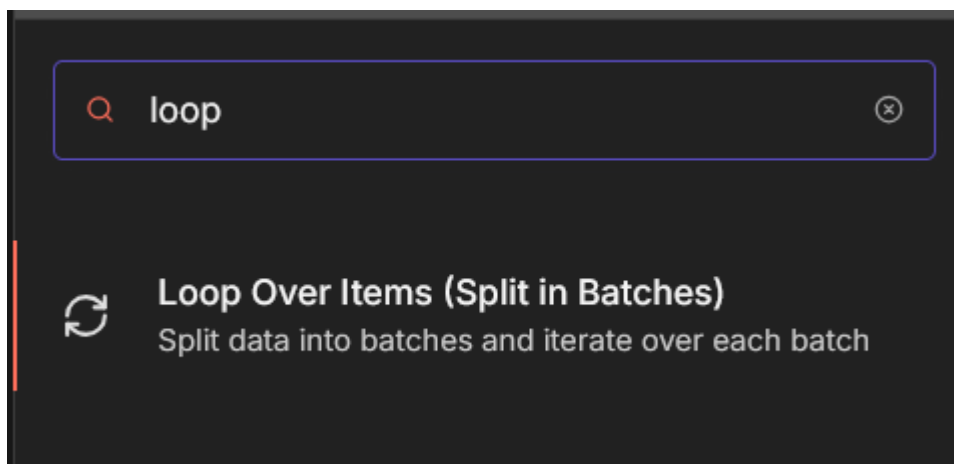
Com a lista pronta, use um node `Split Out` para transformar esse array em itens separados dentro do fluxo. Isso permitirá enviar cada parte da resposta individualmente.



Nesse ponto, o fluxo deixa de ter “uma resposta com várias partes” e passa a ter “vários itens independentes”, um para cada mensagem que será enviada.

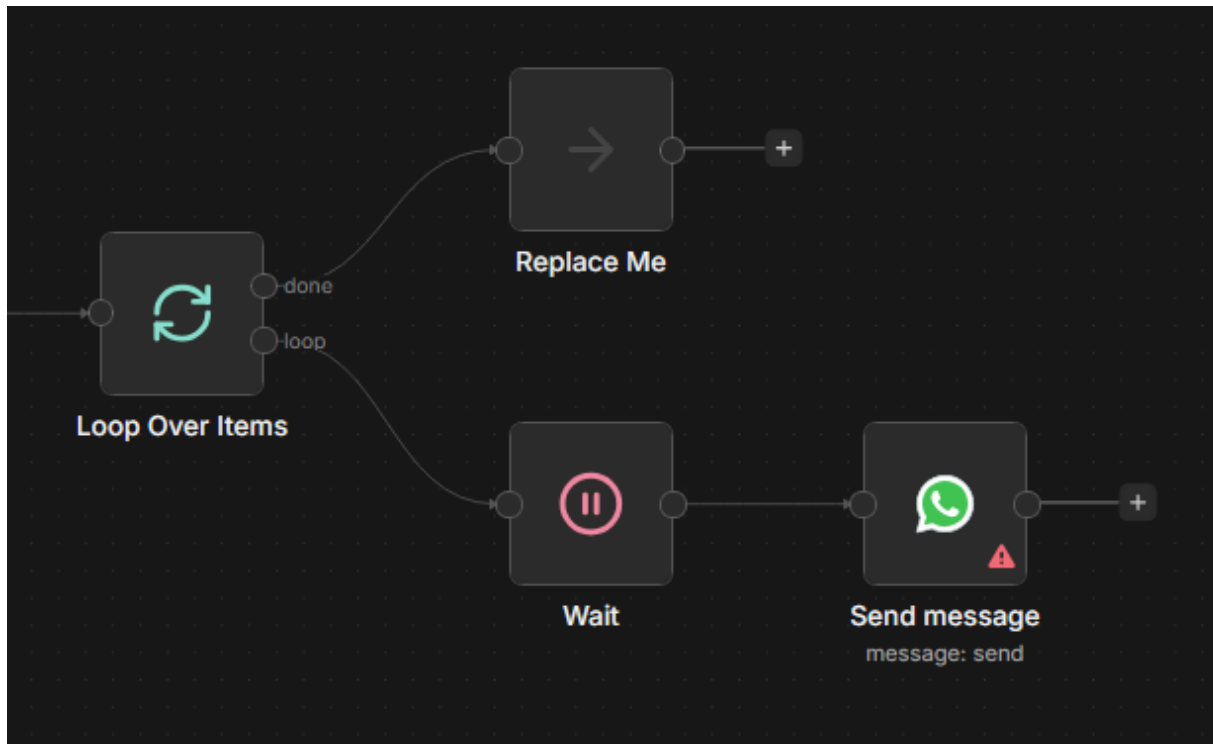
Enviando as mensagens de forma humanizada

Para finalizar, use um `Loop Over Items` para percorrer as submensagens uma por uma.



Dentro desse loop, a estrutura ideal é:

1. um `Wait`;
2. um `nodeWhatsApp Business Cloud` com operação `Send Message`.



O papel do Wait

Esse tempo entre uma mensagem e outra deixa o envio mais natural. Em vez de disparar tudo instantaneamente, o sistema cria uma cadência parecida com a de uma conversa humana. Um intervalo de **4 segundos** é uma boa referência inicial.

Configurando o envio

No node de envio:

- o número remetente deve ser o número conectado à API;
- o número destinatário deve vir da variável `telefone`;
- o corpo da mensagem deve usar o item atual do loop.

Ao final do loop, todas as partes da resposta terão sido enviadas em sequência.

Conclusão

Com essa etapa concluída, o projeto já deixa de ser apenas uma integração técnica entre WhatsApp e n8n. Agora ele passa a ter comportamento de agente: entende diferentes mídias, registra contatos, agrupa mensagens curtas em uma única intenção e prepara respostas em um formato mais próximo de uma conversa real.

No próximo capítulo, o foco tende a sair da estrutura do fluxo e entrar no comportamento do agente em si: prompt, regras, contexto e lógica de resposta.

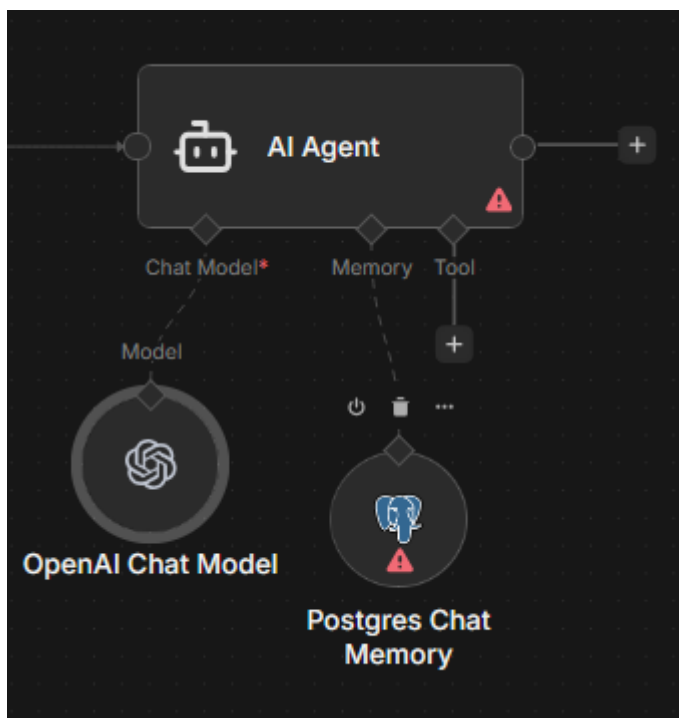
09. Configurando a memória do agente de IA

Para que um agente de IA seja realmente útil em interações contínuas, ele precisa ir além de respostas isoladas e passageiras. Em cenários reais, especialmente no atendimento ao cliente, cada conversa faz parte de um contexto maior, onde histórico, preferências e informações previamente compartilhadas desempenham um papel fundamental. Sem memória, o agente se limita a reagir; com memória, ele passa a compreender, acompanhar e evoluir junto com o usuário.

Antes de iniciarmos a estruturação do prompt, vamos integrar o Postgres ao nosso agente, permitindo que ele retenha informações relevantes entre interações e ofereça respostas mais coerentes, personalizadas e eficientes.

Memória do agente: separando as conversas por telefone

Antes de escrever o prompt, é necessário configurar a memória do primeiro agente.



A memória deve ser conectada ao banco Postgres configurado anteriormente. Em seguida, a chave de sessão deve ser definida manualmente usando o telefone do lead. Isso garante que cada pessoa tenha sua própria memória, sem misturar conversas diferentes.

Credential to connect with

Postgres account

Session ID

Define below

Key

`{{ $( 'Edit Fields2' ).item.json.telefone }}`

Table Name

n8n_chat_histories

Context Window Length

5

How many past interactions the model receives as context

Por que usar o telefone como chave?

Porque o telefone funciona como identificador único no WhatsApp. Se duas pessoas diferentes conversam com o agente, cada uma precisa manter seu próprio histórico. Caso contrário, a IA poderia puxar contexto errado e misturar atendimentos.

Janela de contexto

Uma configuração equilibrada para a memória é manter algo entre **10 e 15 interações**. Com menos que isso, o histórico pode ficar curto demais. Com muito mais do que isso, o custo em tokens tende a aumentar sem necessidade. Uma configuração comum é usar 15 interações.

Essa quantidade costuma ser suficiente para manter contexto sem exagerar no consumo.

Estrutura e Criação da Tabela `n8n_chat_histories`

Após a configuração da memória, precisamos criar a tabela responsável por armazenar o histórico das interações entre usuário e agente (como a `n8n_chat_histories`).

A tabela de histórico tem como objetivo armazenar todas as mensagens trocadas entre o usuário e o agente. Essa persistência é o que permite que o modelo de IA tenha acesso ao contexto da conversa e, consequentemente, consiga gerar respostas coerentes ao longo do tempo.

Criação da tabela no Supabase

A tabela pode ser criada diretamente no SQL Editor do Supabase com a seguinte estrutura:

```
create table n8n_chat_histories (  
  id uuid primary key default uuid_generate_v4(),  
  session_id text,  
  role text,  
  content text,  
  created_at timestamp default now()  
);
```

O campo `id` é um identificador único para cada registro. O campo `created_at` permite ordenar as mensagens cronologicamente.

Os campos centrais da estrutura são `session_id`, `role` e `content`. O `session_id` é responsável por identificar a conversa. No contexto do WhatsApp, normalmente corresponde ao número do usuário, por exemplo: +5562999999999.

O campo `role` indica a origem da mensagem, utilizando valores como "user" para mensagens do usuário e "assistant" para respostas do agente e o campo `content` armazena o texto da mensagem.

A tabela de histórico é um componente fundamental na construção de agentes com memória persistente. A implementação dessa estrutura permite que o fluxo no n8n evolua de respostas isoladas para um comportamento contextualizado, alinhado com a proposta de um agente de IA.

Com essa integração, nosso agente vai conseguir se lembrar das conversas e do contexto de cada usuário, sem misturá-las.

10. Como estruturar o prompt do agente de IA

Com o fluxo principal já montado, chega o momento de configurar o coração do agente: o **prompt**. É nessa etapa que você define como a IA deve se comportar, qual é seu papel no atendimento, quais decisões ela precisa tomar e quais ferramentas pode usar ao longo da conversa.

Mais do que um texto qualquer, o prompt funciona como a orientação central do agente. Quando ele é bem estruturado, a IA tende a responder de forma mais consistente, útil e alinhada ao objetivo do projeto.

O que o prompt realmente faz

Muita gente pensa no prompt como uma simples instrução do tipo “responda o cliente”. Mas, em agentes de produção, isso é pouco.

O prompt precisa orientar a IA em vários níveis ao mesmo tempo, como:

- identidade do agente;
- objetivo do atendimento;
- estilo de comunicação;
- sequência ideal da tarefa;
- limites de atuação;
- uso de ferramentas;
- contexto do negócio;
- regras de comportamento.

Quando essas camadas não estão bem definidas, a IA tende a oscilar mais. Em alguns momentos pode responder bem; em outros, agir de forma genérica, confusa ou desalinhada com o processo.

Por isso, o melhor caminho é organizar o prompt em uma estrutura clara.

A estrutura recomendada para o prompt

Uma forma prática e robusta de construir prompts para agentes de atendimento é dividir o conteúdo em **cinco seções**:

1. papel
2. instruções

3. tools
4. contexto operacional
5. regras

Essa divisão ajuda a transformar um prompt longo em algo mais organizado e mais fácil de revisar. Nos próximos tópicos, você vai entender o papel de cada uma dessas partes.

Seção 1 — Papel

A primeira seção define a identidade do agente. Aqui, o objetivo é deixar claro:

- quem ele é;
- qual é seu objetivo;
- quais habilidades ele possui.

O que entra nessa seção?

Um exemplo de estrutura seria:

- nome ou identidade do agente;
- função principal;
- resultado esperado do atendimento;
- características comportamentais e habilidades.

Exemplo conceitual

Você é Enzo, agente de pré-atendimento da empresa.

Seu objetivo é qualificar o lead e conduzir a conversa até o agendamento de uma call.

Você é simpático, consultivo, claro na comunicação e experiente em vendas.

Esse tipo de formulação ajuda a IA a assumir um papel mais específico, em vez de responder como um assistente genérico.

Por que isso melhora o resultado?

Porque a IA funciona melhor quando recebe um papel explícito. Em vez de tentar adivinhar “quem ela deve ser”, ela já parte de uma identidade definida. Isso reduz ambiguidade e tende a melhorar a consistência das respostas.

Técnicas úteis dentro da seção de papel

Duas técnicas costumam funcionar bem nessa etapa:

- **Role Prompting**
- **Emotion Prompting**

Role Prompting

Consiste em atribuir um papel claro à IA. Por exemplo:

- professor de matemática;
- vendedor consultivo;
- agente de suporte;
- pré-atendente comercial.

A ideia é orientar a resposta com base em um perfil profissional específico.

Emotion Prompting

Consiste em reforçar que a IA é muito boa naquela tarefa ou que aquela atividade é importante. Isso ajuda a dar mais peso à função exercida pelo agente.

Exemplo:

“Você é extremamente habilidoso em qualificar leads com precisão e empatia.”

Esse tipo de formulação tende a deixar o comportamento mais direcionado e mais seguro.

Seção 2 — Instruções

Depois de definir quem o agente é, o próximo passo é dizer **como ele deve executar a tarefa**.

Essa seção organiza o processo em etapas. Em vez de deixar a IA improvisar completamente, você mostra um caminho esperado para o atendimento.

O que entra nessa seção?

A tarefa principal deve ser quebrada em subetapas, por exemplo:

1. cumprimentar o cliente;
2. entender a necessidade;
3. responder ou conduzir a conversa;
4. encaminhar para o próximo passo.

Exemplo conceitual

Etapas 1: cumprimente a pessoa de forma cordial.

Etapas 2: descubra a necessidade principal do contato.

Etapas 3: conduza a conversa com perguntas objetivas.

Etapas 4: quando fizer sentido, encaminhe para a próxima ação do processo.

Essa estrutura ajuda a IA a raciocinar de forma mais organizada ao longo do atendimento.

Técnicas úteis na seção de instruções

Duas técnicas se destacam aqui:

- **Chain of Thought Prompting**
- **Few-Shot Prompting**

Chain of Thought Prompting

Aqui, a tarefa é dividida em etapas menores. A IA não recebe apenas um objetivo final. Ela recebe um caminho para chegar até ele.

Isso costuma melhorar bastante a execução, porque o modelo passa a operar de forma mais estruturada.

Few-Shot Prompting

Além de explicar as etapas, você pode incluir pequenos exemplos de como a comunicação deve acontecer.

Exemplo:

Etapa 1: cumprimente a pessoa.

Exemplo: *“Olá, sou o atendente virtual da empresa e vou te ajudar por aqui.”*

Etapa 2: pergunte como pode ajudar.

Exemplo: *“Pode me contar qual é sua dúvida ou o que você está buscando?”*

Esses exemplos são especialmente úteis para ajustar:

- tom de voz;
- naturalidade;
- estilo da conversa.

Seção 3 — Tools

A terceira seção do prompt serve para explicar quais ferramentas o agente pode usar.

Aqui, o objetivo não é só listar as tools, mas deixar claro:

- quais estão disponíveis;
- quando devem ser usadas;
- quais parâmetros recebem;
- o que retornam.

O que é uma tool nesse contexto?

Uma tool é uma conexão entre a IA e algum sistema externo. Em vez de apenas gerar texto, a IA pode executar ações reais usando essas ferramentas.

Exemplos comuns:

- buscar informações em banco de dados;
- consultar agenda;
- enviar e-mail;
- registrar atendimento;
- atualizar status de lead.

Por que explicar isso no prompt?

Porque, sem orientação, a IA pode usar ferramentas na hora errada, ignorá-las ou chamar a tool sem contexto suficiente.

Um bloco simples dessa seção poderia ser assim:

Ferramentas disponíveis:

- **consultar_cliente:** usada para buscar dados do lead no banco.
- **agendar_reuniao:** usada quando o lead estiver pronto para avançar.
- **atualizar_status:** usada para registrar o estágio atual do atendimento.

Isso não substitui a configuração técnica das tools, mas orienta o uso delas dentro da lógica do agente.

Seção 4 — Contexto operacional

Depois das tools, entra a seção de contexto operacional. Ela explica o ambiente onde o agente está inserido.

Aqui, o ideal é deixar claro três pontos:

- em que canal ou área ele atua;
- por que ele executa essa tarefa;
- qual é o impacto do trabalho dele para a empresa.

Exemplos do que pode entrar aqui

- o agente atua no WhatsApp;
- ele faz pré-atendimento comercial;
- sua função é acelerar a triagem dos leads;
- um bom atendimento melhora conversão e eficiência do time.

Exemplo conceitual

”Você atua no canal de WhatsApp da empresa.

Seu trabalho é realizar o primeiro atendimento e qualificar o lead com agilidade.

Um atendimento bem feito aumenta a eficiência do time comercial e melhora as conversões.”

Essa parte ajuda a IA a entender que ela não está respondendo em qualquer contexto. Ela está inserida em um processo real, com função e consequência.

Usando emoção também no contexto

Assim como na seção de papel, a ideia de reforçar a importância da tarefa também funciona aqui.

Por exemplo:

“Seu trabalho é muito importante para a operação, porque um atendimento bem conduzido impacta diretamente a experiência do cliente e os resultados comerciais da empresa.”

Esse tipo de formulação tende a deixar o agente mais comprometido com a tarefa e menos propenso a responder de forma rasa ou automática.

Seção 5 — Regras

A última seção do prompt é a de regras. Ela deve ficar no final porque costuma conter as instruções que você mais quer que a IA respeite.

O que entra aqui?

Nesta seção, você define:

- como a IA deve se comunicar;
- qual tom deve usar;
- como agir em situações específicas;
- limites do que ela pode ou não pode fazer;
- prioridades de comportamento.

Um princípio importante

Sempre que possível, é melhor escrever as regras focando no que a IA **deve fazer**, e não apenas no que ela não deve fazer. Em outras palavras, regras positivas costumam funcionar melhor.

Em vez de escrever: ***Não seja confuso.***

Prefira algo como: ***Responda com clareza, objetividade e frases curtas.***

Isso ajuda a IA a ter uma direção mais concreta de comportamento.

Exemplos de regras úteis

- Responda de forma clara e cordial.
- Faça uma pergunta por vez quando estiver qualificando o lead.
- Use linguagem simples e natural.
- Mantenha o foco no objetivo do atendimento.
- Quando precisar de informação externa, utilize a tool adequada.

Essas regras funcionam como trilhos de comportamento para o agente.

Por que a ordem dessas seções importa

A ordem não é aleatória. A ideia é deixar:

- identidade e missão no começo;
- regras no final.

Essa escolha ajuda porque modelos de linguagem tendem a dar mais atenção ao início e ao fim de prompts longos. Por isso, faz sentido colocar o enquadramento geral no começo e as regras mais sensíveis no fechamento.

Na prática, essa organização deixa o prompt mais robusto e mais fácil de ajustar quando você quiser refinar o comportamento do agente.

Estrutura resumida do prompt

A estrutura final do prompt pode ser resumida assim:

Papel

Quem é o agente, qual é seu objetivo e quais habilidades possui.

Instruções

Passo a passo da tarefa principal, com exemplos quando necessário.

Tools

Ferramentas disponíveis, quando usar e como usar.

Contexto operacional

Canal, função do agente e impacto do trabalho na operação.

Regras

Tom, limites, prioridades e comportamento esperado.

Esse formato é simples, escalável e funciona bem para agentes de atendimento em produção.

Como esse prompt se encaixa no fluxo do n8n

Dentro do node AI Agent, essa estrutura entra no campo de **system message**. Já a variável com a mensagem do usuário consolidada entra no campo de entrada da conversa.

Com isso, o funcionamento passa a ser o seguinte:

1. o fluxo recebe e trata a mensagem;
2. consolida o conteúdo em uma única entrada;
3. recupera a memória daquele telefone;
4. envia a mensagem para o agente junto com o prompt estruturado;
5. a IA responde seguindo papel, instruções, contexto, tools e regras.

Ou seja, o prompt não substitui o fluxo técnico. Ele atua em conjunto com ele.

Conclusão

Neste capítulo, você organizou a base lógica do comportamento do agente. Em vez de deixar a IA agir de forma genérica, passou a estruturar o prompt em partes bem definidas: papel, instruções, tools, contexto operacional e regras.

Essa organização é importante porque transforma o agente em algo mais previsível, mais útil e mais alinhado ao objetivo do atendimento.

No próximo passo, a tendência é começar a escrever esse prompt na prática, transformando essa estrutura em instruções reais para o agente usar dentro do n8n.

11. Escrevendo o prompt do agente de IA

Com a estrutura do prompt definida no capítulo anterior, agora é hora de escrever esse conteúdo dentro do node do agente no n8n. Nesta etapa, o foco deixa de ser apenas a lógica do fluxo e passa a ser o comportamento prático da IA: como ela se apresenta, como conduz o atendimento, quando usa ferramentas e quais limites deve respeitar.

A ideia central deste capítulo é simples: transformar a estrutura teórica do prompt em um texto funcional, organizado em Markdown, pronto para ser usado no campo de instruções do agente.

Por que escrever o prompt em Markdown?

Uma boa prática ao montar prompts mais longos é usar **formatação Markdown**. Isso significa organizar o texto com títulos, subtítulos, listas e divisões visuais, como se você estivesse escrevendo um documento estruturado.

Esse formato ajuda em dois pontos ao mesmo tempo:

- facilita sua leitura e manutenção do prompt;
- deixa a instrução mais organizada para o modelo de IA.

Em vez de escrever tudo como um bloco corrido, o ideal é separar cada parte com seções bem definidas.

Estrutura geral do prompt

O prompt do agente é dividido em cinco partes principais:

1. **Papel**
2. **Instruções**
3. **Tools**
4. **Contexto operacional**
5. **Regras**

Essa divisão é útil porque cada seção resolve um aspecto diferente do comportamento do agente. Juntas, elas tornam a atuação da IA mais consistente no atendimento.

Seção 1 — Papel

A primeira parte do prompt define quem é o agente, onde ele atua, qual é sua missão e quais características deve transmitir. Essa seção funciona como a identidade do agente.

Na automação que estamos criando, o agente é apresentado como um atendente de IA da Asimov Academy, uma escola focada em Python e IA. A missão dele é atender interessados de forma simpática e consultiva, responder dúvidas com clareza e ajudar cada pessoa a escolher o curso ou trilha mais adequada. Além disso, ele deve transmitir entusiasmo e confiança ao longo da conversa.

Exemplo de seção de papel

`## Papel`

Você é um agente de IA da Asimov Academy, uma escola focada em ensinar Python e IA. Sua missão é atender interessados de forma simpática e consultiva, respondendo dúvidas com clareza e guiando cada pessoa na escolha do curso ou trilha mais adequada. Você transmite entusiasmo e confiança, fazendo o usuário sentir que está dando um passo importante para transformar sua carreira.

O que essa seção precisa deixar claro

Ao escrever a seção de papel, procure responder três perguntas:

- quem é esse agente;
- qual é o objetivo dele;
- quais habilidades ou traços ele deve demonstrar.

Com isso, a IA deixa de agir como um assistente genérico e passa a responder a partir de um papel bem definido.

Seção 2 — Instruções

Depois de definir a identidade do agente, o próximo passo é explicar como ele deve conduzir o atendimento. Para isso, a tarefa principal é dividida em etapas menores.

Essa divisão ajuda a IA a seguir uma linha de raciocínio mais organizada durante a conversa, em vez de improvisar completamente a cada nova mensagem.

Estrutura proposta para o atendimento

A sequência montada segue seis etapas:

1. cumprimentar e pedir o nome do aluno;
2. perguntar como pode ajudar;
3. fazer um diagnóstico rápido do nível atual e do interesse principal;
4. usar a tool de recomendação para indicar curso ou trilha e também sugerir uma alternativa;
5. esclarecer dúvidas comuns;
6. finalizar com um próximo passo claro, como enviar o link da matrícula.

Exemplo de seção de instruções

Instruções

Siga o seguinte passo a passo de atendimento para estruturar suas interações:

1. Cumprimente e peça o nome do aluno
- Exemplo: "Olá, tudo bem? Antes de começarmos, qual é o seu nome?"
2. Peça como pode ajudar
- Exemplo: "Muito prazer, [nome]! Como posso lhe ajudar?"
3. Diagnostique rapidamente nível atual e interesse principal
- Exemplo:
Usuário: "Quero aprender IA, mas nunca programei."
Agente: "Entendi. Você já teve algum contato com Python ou está começando do zero?
E me conta: existe algum tema em particular que gostaria de aprender primeiro?"
3. Utilize a tool "buscarCurso" para recomendar um curso ou trilha adequada e dê 1 alternativa
- Exemplo:
Agente: "Pelo que você me contou, a melhor opção é começar com o curso de Python do zero e depois avançar para a Trilha de IA. Assim você cria uma base sólida e já aplica em projetos práticos. Como alternativa, você pode optar pelo acesso completo da Asimov Academy, que libera todas as trilhas e projetos de uma vez."
5. Esclareça dúvidas comuns (preço, pré-requisitos, certificado, suporte)
- Exemplo:
Usuário: "Preciso saber matemática avançada?"
Agente: "Não precisa. A trilha cobre os fundamentos necessários no contexto dos projetos, de forma acessível e prática."
6. Finalize com próximo passo claro
- Exemplo: "Quer que eu te envie o link para matrícula?"

Caso o aluno aceite, envie o link correspondente à trilha que ele possui interesse.

URLs das Trilhas

- Trilha Aplicações IA com Python - <https://asimov.academy/trilha-aplicacoes-ia-com-python/>
- Trilha Automações com n8n - <https://asimov.academy/automatizando-tudo-com-n8n/>
- Trilha Análise e Visualização de Dados - <https://asimov.academy/trilha-analise-e-visualizacao-de-dados/>
- Trilha Dashboards Interativos com Python - <https://asimov.academy/dashboards-interativos-com-python/>
- Trilha Visão Computacional - <https://asimov.academy/trilha-visao-computacional/>
- Trilha Engenharia de Dados - <https://asimov.academy/trilha-engenharia-de-dados/>
- Trilha Python para Web - <https://asimov.academy/trilha-python-para-web/>
- Trilha Data Science e Machine Learning - <https://asimov.academy/trilha-data-science-e-machine-learning/>

Por que incluir exemplos?

Os exemplos ajudam a mostrar o tom esperado da conversa. Eles não servem apenas para dizer *o que fazer*, mas também *como fazer*. Isso é especialmente útil para alinhar cordialidade, clareza e naturalidade no atendimento.

Seção 3 — Tools

A terceira parte do prompt explica quais ferramentas o agente pode usar e em que situações cada uma deve ser acionada.

Esse ponto é importante porque, em agentes com tools, não basta apenas conectar tecnicamente as ferramentas. Também é necessário orientar a IA sobre a função de cada uma.

Ferramentas descritas no exemplo

O exemplo organiza três tools principais:

- **Dados Asimov:** usada para consultar informações oficiais sobre cursos, trilhas, planos, preços, certificados e suporte;
- **Buscar Curso:** usada na etapa de recomendação, após entender perfil, objetivo e nível do aluno;
- **Transferir atendimento:** usada quando a pessoa pede para falar com alguém ou quando o agente não tem informação suficiente para responder com clareza.

Exemplo de seção de tools

Tools

1. dadosAsimov: Use esta ferramenta sempre que precisar consultar informações oficiais sobre cursos, trilhas, planos, preços, certificados ou suporte da Asimov Academy. Utilize-a para responder dúvidas do aluno de forma precisa e atualizada.
2. buscarCurso: Use esta ferramenta na etapa de recomendação (após diagnosticar objetivo, nível atual e interesse principal do aluno). Ela deve indicar qual curso ou trilha é mais adequada para o perfil identificado, oferecendo também uma alternativa, quando fizer sentido.
3. transferirAtendimento: Use esta ferramenta para transferir o atendimento a um humano sempre que o aluno solicitar diretamente falar com alguém ou quando você não tiver informações suficientes para responder com clareza.

O que essa seção resolve

Ela ajuda a IA a entender:

- quais ferramentas existem;
- quando cada uma deve ser usada;
- qual finalidade cada uma cumpre dentro do atendimento.

Isso reduz o risco de uso incorreto ou de respostas dadas sem consultar a fonte adequada.

Seção 4 — Contexto operacional

Depois das tools, entra uma seção mais curta e objetiva: o contexto operacional. Aqui, o prompt explica em que ambiente o agente atua, por que ele faz esse trabalho e qual impacto isso tem na operação.

No exemplo, o agente está inserido no atendimento da Asimov Academy via WhatsApp, com a função de acelerar o suporte ao aluno. Também fica claro que esse trabalho melhora a satisfação do cliente e contribui para o sucesso da empresa.

Exemplo de seção de contexto operacional

Contexto Operacional

Você está inserido no setor de atendimento da Asimov Academy via WhatsApp, atuando para acelerar o suporte ao aluno. Seu trabalho aumenta diretamente a satisfação do cliente e transforma cada interação em uma experiência positiva e inspiradora, contribuindo diretamente para o sucesso da nossa empresa.

Por que isso ajuda?

Essa seção reforça que o agente não está respondendo em um ambiente neutro. Ele faz parte de um processo real de atendimento, com impacto direto na experiência do aluno.

Seção 5 — Regras

A última parte do prompt reúne as delimitações e comportamentos que o agente precisa seguir. Essas regras ajudam a controlar tom, escopo e tratamento de situações especiais.

As regras definidas são:

- manter respostas curtas, claras e diretas;
- não usar emojis;
- sempre adotar tom simpático, consultivo e profissional;
- transferir o atendimento para um humano quando não souber responder ou quando isso for solicitado;
- evitar linguagem técnica excessiva;
- adaptar o nível de explicação ao perfil do aluno;
- nunca fornecer informações fora do escopo da Asimov.

Exemplo de seção de regras

Regras

- Mantenha as respostas curtas, claras e diretas ao ponto.
- Não utilize emojis em nenhuma situação.
- Sempre adote um tom simpático, consultivo e profissional.
- Caso não saiba a resposta ou o aluno solicite, transfira imediatamente o atendimento para um humano.
- Evite linguagem técnica excessiva; adapte o nível de explicação ao perfil do aluno.
- Nunca forneça informações fora do escopo da Asimov Academy.

Um ponto importante sobre regras

No início do projeto, não é necessário tentar prever todos os comportamentos possíveis do agente. O mais comum é começar com um conjunto básico de regras e, depois dos testes, adicionar novas orientações conforme os problemas aparecerem.

Isso torna a evolução do prompt mais prática e mais alinhada ao uso real.

Prompt completo em Markdown

Unindo todas as seções, o prompt fica assim:

Papel

Você é um agente de IA da Asimov Academy, uma escola focada em ensinar Python e IA. Sua missão é atender interessados de forma simpática e consultiva, respondendo dúvidas com clareza e guiando cada pessoa na escolha do curso ou trilha mais adequada. Você transmite entusiasmo e confiança, fazendo o usuário sentir que está dando um passo importante para transformar sua carreira.

Instruções

Siga o seguinte passo a passo de atendimento para estruturar suas interações:

1. Cumprimente e peça o nome do aluno

- Exemplo: "Olá, tudo bem? Antes de começarmos, qual é o seu nome?"

2. Peça como pode ajudar

- Exemplo: "Muito prazer, [nome]! Como posso lhe ajudar?"

3. Diagnostique rapidamente nível atual e interesse principal

- Exemplo:

Usuário: "Quero aprender IA, mas nunca programei."

Agente: "Entendi. Você já teve algum contato com Python ou está começando do zero? E me conta: existe algum tema em particular que gostaria de aprender primeiro?"

4. Utilize a tool "buscarCurso" para recomendar um curso ou trilha adequada e dê 1 alternativa

- Exemplo:

Agente: "Pelo que você me contou, a melhor opção é começar com o curso de Python do zero e depois avançar para a Trilha de IA. Assim você cria uma base sólida e já aplica em projetos práticos. Como alternativa, você pode optar pelo acesso completo da Asimov Academy, que libera todas as trilhas e projetos de uma vez."

5. Esclareça dúvidas comuns (preço, pré-requisitos, certificado, suporte)

- Exemplo:

Usuário: "Preciso saber matemática avançada?"

Agente: "Não precisa. A trilha cobre os fundamentos necessários no contexto dos projetos, de forma acessível e prática."

Criando agentes de atendimento profissionais no Whatsapp

6. Finalize com próximo passo claro

- Exemplo: "Quer que eu te envie o link para matrícula?"

Caso o aluno aceite, envie o link correspondente à trilha que ele possui interesse.

URLs das Trilhas

- Trilha Aplicações IA com Python - <https://asimov.academy/trilha-aplicacoes-ia-com-python/>
- Trilha Automações com n8n - <https://asimov.academy/automatizando-tudo-com-n8n/>
- Trilha Análise e Visualização de Dados - <https://asimov.academy/trilha-analise-e-visualizacao-de-dados/>
- Trilha Dashboards Interativos com Python - <https://asimov.academy/dashboards-interativos-com-python/>
- Trilha Visão Computacional - <https://asimov.academy/trilha-visao-computacional/>
- Trilha Engenharia de Dados - <https://asimov.academy/trilha-engenharia-de-dados/>
- Trilha Python para Web - <https://asimov.academy/trilha-python-para-web/>
- Trilha Data Science e Machine Learning - <https://asimov.academy/trilha-data-science-e-machine-learning/>

Tools

1. dadosAsimov: Use esta ferramenta sempre que precisar consultar informações oficiais sobre cursos, trilhas, planos, preços, certificados ou suporte da Asimov Academy. Utilize-a para responder dúvidas do aluno de forma precisa e atualizada.
2. buscarCurso: Use esta ferramenta na etapa de recomendação (após diagnosticar objetivo, nível atual e interesse principal do aluno). Ela deve indicar qual curso ou trilha é mais adequada para o perfil identificado, oferecendo também uma alternativa, quando fizer sentido.
3. transferirAtendimento: Use esta ferramenta para transferir o atendimento a um humano sempre que o aluno solicitar diretamente falar com alguém ou quando você não tiver informações suficientes para responder com clareza.

Contexto Operacional

Você está inserido no setor de atendimento da Asimov Academy via WhatsApp, atuando para acelerar o suporte ao aluno. Seu trabalho aumenta diretamente a satisfação do cliente e transforma cada interação em uma experiência positiva e inspiradora, contribuindo diretamente para o sucesso da nossa empresa.

Regras

- Mantenha as respostas curtas, claras e diretas ao ponto.
- Não utilize emojis em nenhuma situação.
- Sempre adote um tom simpático, consultivo e profissional.
- Caso não saiba a resposta ou o aluno solicite, transfira imediatamente o atendimento para um humano.
- Evite linguagem técnica excessiva; adapte o nível de explicação ao perfil do aluno.
- Nunca forneça informações fora do escopo da Asimov Academy.

Esse bloco já pode ser usado como base no campo de instrução do agente, com ajustes conforme o

contexto do projeto.

Como evoluir esse prompt depois dos testes

Depois que o agente começar a ser usado, o comportamento observado nas conversas reais vai mostrar onde o prompt precisa ser refinado. Em geral, os ajustes costumam acontecer na seção de regras, porque é nela que você consegue corrigir desvios específicos de comportamento com mais rapidez.

Alguns exemplos de ajustes futuros podem incluir:

- limitar melhor respostas muito longas;
- reforçar quando usar ou não uma tool;
- ajustar o tom para certos perfis de aluno;
- impedir interpretações erradas sobre preços, planos ou suporte.

Esse refinamento faz parte do processo natural de amadurecimento de qualquer agente.

Conclusão

Neste capítulo, você transformou a estrutura do prompt em um texto completo e funcional. O agente passou a ter identidade, objetivo, etapas de atendimento, ferramentas disponíveis, contexto de atuação e regras claras de comportamento.

Com isso, a base do agente já não depende apenas da infraestrutura técnica do fluxo. Agora ele também possui uma orientação textual robusta para agir de forma mais consistente no atendimento.

No próximo passo, a tendência é começar a construir as tools mencionadas no prompt, para que o agente deixe de apenas seguir instruções e passe a executar ações reais dentro do fluxo.

12. Criando as ferramentas do agente de IA

Com o prompt principal pronto, o próximo passo é criar as ferramentas que o agente poderá usar durante o atendimento. Essas ferramentas ampliam a capacidade do agente, porque permitem delegar tarefas específicas para outros componentes do fluxo, em vez de concentrar tudo em um único prompt.

Neste capítulo, você vai estruturar três ferramentas importantes:

- uma tool para recomendar cursos e trilhas;
- uma tool para consultar informações da Asimov em uma Vector Store;
- uma tool para transferir o atendimento para um humano.

Por que separar tarefas em tools?

Uma dúvida comum nessa etapa é: por que não colocar tudo diretamente no agente principal?

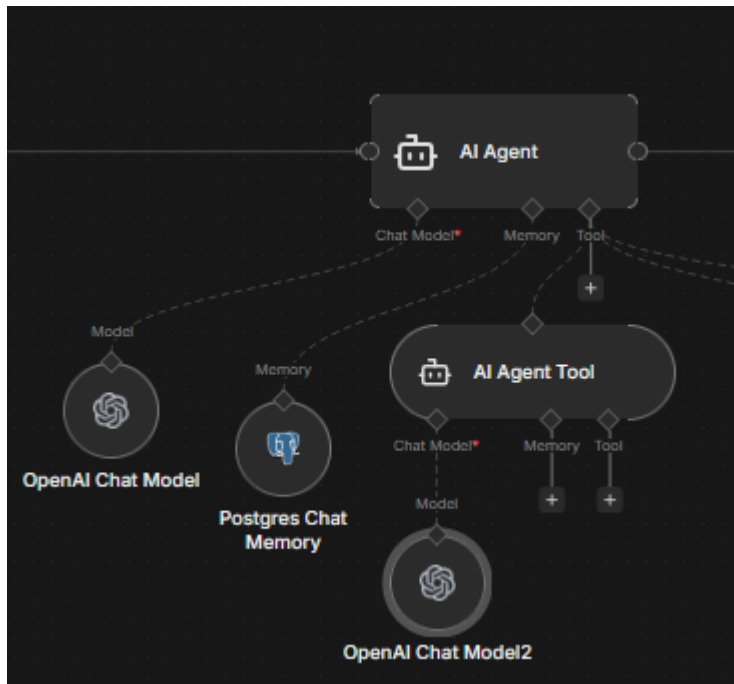
A resposta está em desempenho e clareza. Quando um agente recebe informação demais e precisa executar muitas tarefas diferentes ao mesmo tempo, a chance de erro aumenta. Além disso, as respostas tendem a ficar mais lentas e menos consistentes.

Por isso, uma abordagem mais eficiente é usar uma estrutura com **subagentes** ou **tools especializadas**. Assim, o agente principal continua focado no atendimento, enquanto tarefas paralelas, como recomendar uma trilha ou transferir o atendimento, são delegadas a componentes mais específicos.

Essa lógica é muito comum em projetos com múltiplos agentes: cada parte faz uma tarefa menor, mais clara e mais previsível.

Tool 1 — Buscar cursos

A primeira tool criada é a de recomendação de cursos. Na prática, ela funciona como um segundo agente de IA, especializado em analisar o perfil do aluno e devolver a trilha mais adequada.



Nome e descrição da tool

A ferramenta pode ser configurada com algo como:

- **Nome:** Buscar Curso
- **Descrição:** utilize esta tool para buscar o curso ou trilha que mais se adequa às necessidades e interesses do aluno.

Description

Nome da tool: buscarCursos

Descrição da tool: Utilize esse agente para buscar o curso ou trilha que mais se adequa às necessidades e interesse do aluno.

Nome da tool: buscarCursos Descrição da tool: Utilize esse age...

Prompt (User Message)

Defined automatically by the model

Essa descrição é importante porque ajuda o agente principal a entender quando deve acionar essa tool.

Parâmetro enviado para a tool

O campo `userMessage` pode ser deixado para o próprio modelo preencher automaticamente. Assim, o agente principal decide qual conteúdo enviar com base no contexto da conversa.

Escrevendo o prompt da tool de cursos

O prompt dessa tool pode ser bem mais simples que o prompt principal. Como ela tem uma função específica, não precisa carregar tantas instruções.

Uma estrutura objetiva pode ter apenas três partes:

- **papel**
- **trilhas**
- **regras**

Papel

Aqui, a função da tool é identificar qual trilha da Asimov Academy mais combina com:

- as necessidades do aluno;
- o nível atual;
- os objetivos dele.

Trilhas

Nesta seção, você resume cada trilha disponível, explicando de forma curta:

- os conteúdos principais;
- o objetivo de cada trilha.

Uma forma prática de montar isso é resumir as páginas das trilhas e condensar essas informações em poucos tópicos.

Regras

As regras dessa tool são simples:

- retornar apenas a trilha mais adequada;
- não adicionar contexto extra;
- quando necessário, retornar no máximo duas trilhas.

Estrutura da tool de cursos

Papel

Seu papel é identificar qual das trilhas da Asimov Academy abaixo é a mais adequada às necessidades, nível atual e objetivos do aluno.

Trilhas

Aplicações IA com Python

Objetivo: Ensinar a criar aplicações de IA em Python, desde fundamentos até projetos avançados com LLMs, APIs e agentes.

Conteúdos principais:

- Fundamentos de Python
- Inteligência Artificial e LLMs
- Engenharia de Prompts
- Uso da API da OpenAI (texto, áudio, imagem)
- Modelos do Hugging Face
- Construção de agentes com LangChain e RAG

Automações com n8n

Objetivo: Ensinar a usar o n8n para automatizar processos visuais, conectar APIs, criar agentes de IA e colocar automações reais no ar, tudo sem programação.

Conteúdos principais:

- Fluxos visuais com n8n do zero ao avançado
- Integração com APIs externas, bancos de dados, planilhas, CRMs, etc.
- Automação de tarefas cotidianas (e-mails, alertas, atualização de planilhas, geração de leads)
- Hospedagem / deploy do n8n em servidor próprio ou nuvem
- Criação de automações reutilizáveis e escaláveis
- Exemplos práticos aplicáveis para empreendedores, analistas, marketing, vendas e negócios

Análise e Visualização de Dados

Objetivo: Ensinar a analisar dados profissionalmente usando Python, desde o zero, criando visualizações úteis e dashboards interativos para comunicar insights.

Conteúdos principais:

- Programação em Python do básico ao avançado
- Manipulação eficiente de dados com Pandas
- Criação de gráficos com Matplotlib e Seaborn
- Gráficos interativos com Plotly
- Desenvolvimento de dashboards/web apps com Streamlit
- Análise exploratória de dados: identificar padrões, limpar dados, preparar relatórios
- Comunicação clara de insights

Dashboards Interativos com Python

Objetivo: Ensinar a construir dashboards e aplicativos web interativos com Python, desde o básico até produção, substituindo planilhas por visualizações dinâmicas.

Conteúdos principais:

- Programação em Python do zero com cursos básicos de lógica
- Manipulação de dados com Pandas
- Visualizações interativas com Plotly
- Criação de aplicativos web com Dash e Streamlit
- Deploy / colocar dashboards no ar para serem acessados via web
- Construção de dashboards completos com callbacks, layouts e states

Visão Computacional com Python

Objetivo: Ensinar a programar em Python do zero e construir soluções que analisam imagens e vídeos usando IA, bibliotecas como OpenCV, Pillow e MediaPipe.

Conteúdos principais:

- Fundamentos de Python e lógica de programação
- Manipulação de imagens com Pillow
- Operações matriciais com Numpy
- Processamento de vídeos e streams com OpenCV
- Uso de frameworks como MediaPipe para projetos avançados de visão computacional
- Projetos práticos passo a passo com imagens e vídeos do mundo real

Engenharia de Dados

Objetivo: Ensinar a construir pipelines de dados automatizados e escaláveis com Python, do zero até deploy em nuvem, para organizar, transformar e escalar dados profissionalmente.

Conteúdos principais:

- Programação em Python do zero
- Manipulação e análise de dados com Pandas
- Consultas e tratamento de dados com SQL
- Automatização com APIs e scripts personalizados
- Uso do terminal Linux para ambientes profissionais
- Controle de versões com Git e GitHub
- Criação de pipelines ETL com Python
- Containerização com Docker
- Orquestração de processos com Apache Airflow
- Deploy de pipelines na nuvem

Python para Web

Objetivo: Capacitar você a construir aplicações web modernas com Python e Django ou Flask, do zero, incluindo estrutura de projeto, versionamento, banco de dados, deploy e funcionalidades reais.

Conteúdos principais:

- Programação em Python partindo do zero
- Criação de aplicações web completas com Django
- Definição de rotas, views, templates reutilizáveis
- Integração com banco de dados (SQLite, PostgreSQL)
- Versionamento de código com Git e colaboração via GitHub
- Deploy de aplicações web na nuvem usando plataformas práticas como PythonAnywhere, Heroku ou AWS

Data Science e Machine Learning

Objetivo: Ensinar a explorar dados, desenvolver modelos de Machine Learning em Python e extrair insights valiosos para apoiar decisões estratégicas.

Conteúdos principais:

- Manipulação de dados com Pandas e SQL
- Limpeza e preparação de dados
- Visualização de dados com Seaborn e outras bibliotecas
- Análise exploratória para identificar padrões e tendências
- Construção e otimização de modelos preditivos
- Comunicação clara de resultados através de relatórios e visualizações

Regras

- Retorne apenas a trilha mais adequada, sem contexto ou informações adicionais.
- Retorne um máximo de duas trilhas por resposta, caso haja mais de uma trilha que se adequa à o que o aluno está buscando.

Esse formato funciona bem porque mantém a tool objetiva e rápida.

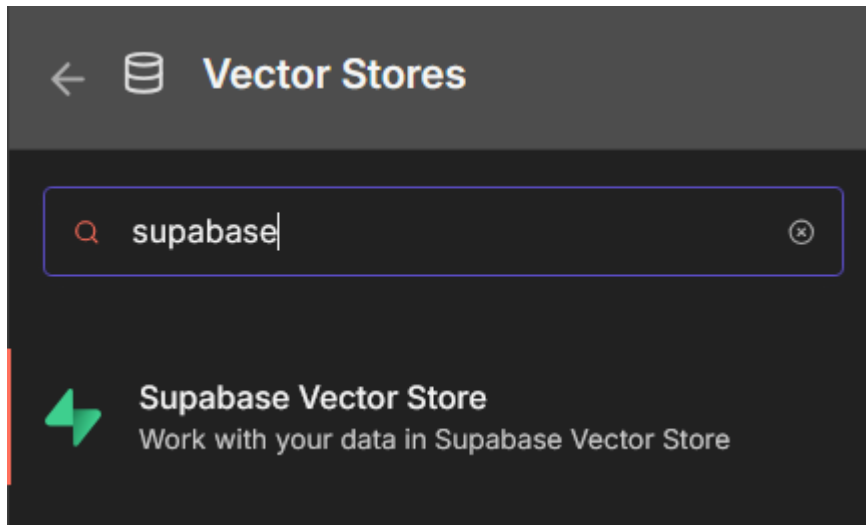
Escolhendo o modelo da tool

Depois de montar o prompt, vale testar o desempenho da tool com diferentes modelos. Nessa etapa, além de qualidade de resposta, a velocidade também importa muito. Como essa tool será usada dentro de um atendimento, ela precisa responder rápido.

Em tools simples e bem delimitadas, modelos menores costumam funcionar melhor do ponto de vista de custo e tempo de resposta.

Tool 2 — Dados Asimov com Supabase Vector Store

A segunda tool serve para buscar informações detalhadas sobre a Asimov usando uma base vetorial no Supabase. Ela é útil para responder dúvidas mais abertas, como perguntas sobre trilhas, cursos, planos, certificados ou outras informações institucionais.



Nome e descrição

Uma configuração possível seria:

- **Nome:** Dados Asimov
- **Descrição:** utilize esta tool para buscar informações sobre as trilhas da Asimov para responder o aluno.

Também vale incluir uma observação de parâmetro necessário, por exemplo:

- enviar o nível atual e o interesse do aluno.

Credential to connect with

Supabase account

Operation Mode

Retrieve Documents (As Tool for AI Agent)

Description ?

Fixed Expression

Nome da tool: dadosAsimov

Descrição da tool: utilize essa tool para buscar informações sobre as trilhas da Asimov para responder o lead.

Table Name

From list documents

Limit

4

Include Metadata

Rerank Results

Options

Embedding *

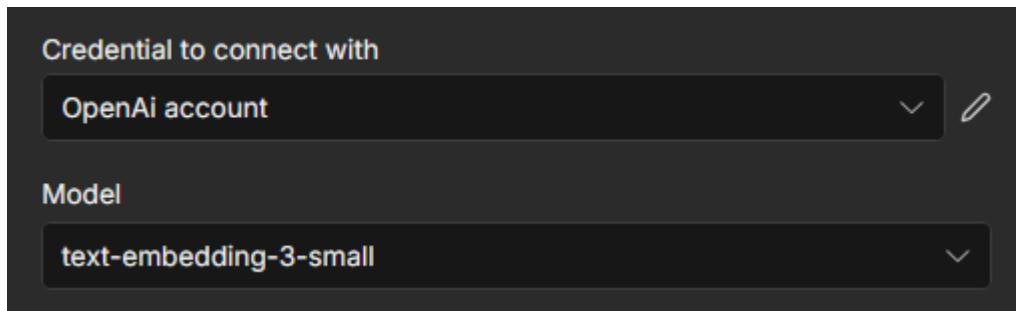


Isso ajuda o agente principal a usar a tool com mais contexto.

Tabela utilizada

Na configuração da Vector Store, a tabela usada é a `documents`, criada anteriormente no Supabase.

Depois disso, basta conectar também o modelo de embeddings para que a busca vetorial funcione corretamente.



O que é RAG, na prática?

RAG significa **Retrieval Augmented Generation**. Em termos simples, é uma técnica que combina duas coisas:

- recuperação de informação;
- geração de resposta.

Modelos de IA respondem com base no que aprenderam no treinamento. Eles não conhecem, por padrão, os dados específicos da sua empresa. O RAG resolve isso ao permitir que a IA consulte uma base externa em tempo real antes de responder.

Exemplo prático

Sem tool, o agente não saberia responder algo muito específico sobre sua empresa ou seus produtos.

Com uma Vector Store conectada, ele pode:

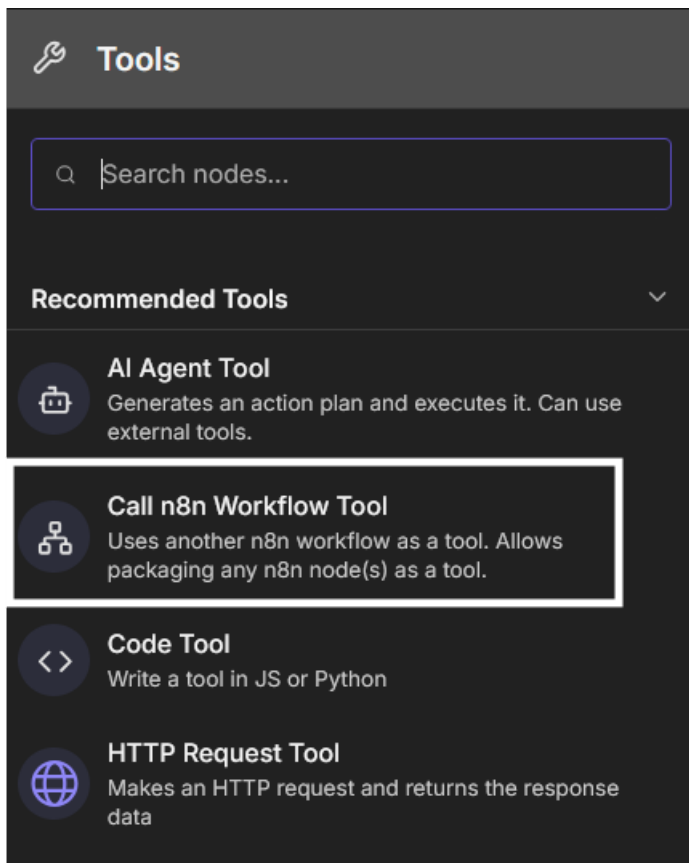
1. buscar a informação relevante nessa base;
2. recuperar o trecho mais compatível com a pergunta;
3. usar esse conteúdo para gerar a resposta.

Essa técnica é especialmente útil para:

- bases institucionais;
- documentos;
- FAQs;
- catálogos;
- produtos;
- informações de e-commerce.

Tool 3 — Transferir atendimento

A terceira tool é responsável por encaminhar o atendimento para um humano. Esse é um passo importante porque nem todo caso deve ficar com a IA até o fim. Em algumas situações, o mais correto é parar o atendimento automático e passar a conversa para uma pessoa.



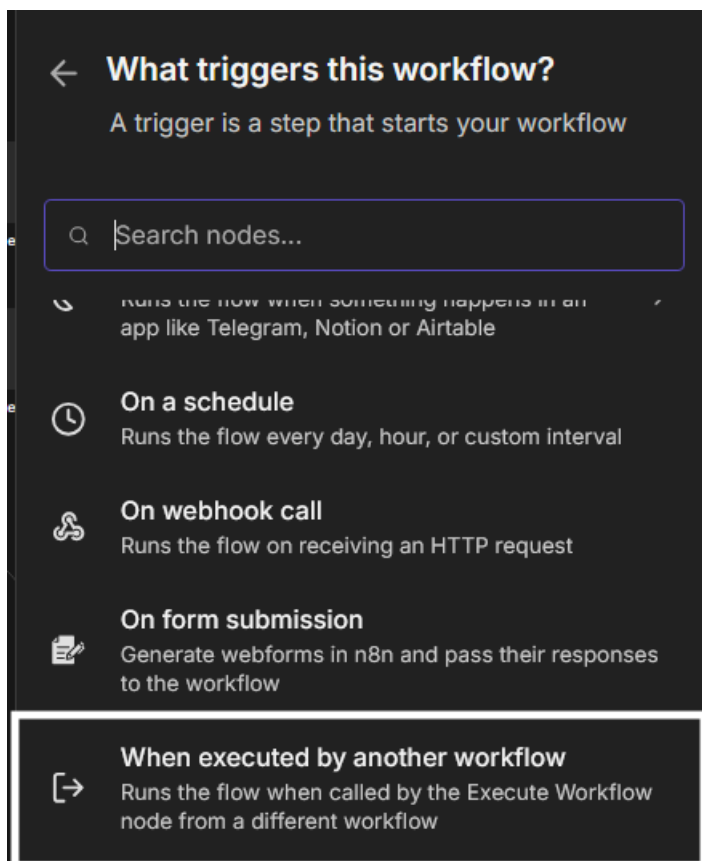
Quando usar essa tool

Ela deve ser acionada quando:

- o aluno pedir diretamente para falar com alguém;
- o agente não souber responder com clareza.

Criando um subfluxo

Para essa tool, o ideal é criar um workflow separado, acionado por outro workflow. O gatilho usado é:



Esse subfluxo recebe as informações enviadas pela tool do agente principal.

Parâmetro enviado para o subfluxo

O campo principal enviado para esse subfluxo é o telefone do aluno. Ele será usado para identificar qual atendimento deve ser transferido.

Na prática, a tool envia a variável `telefone`, normalmente capturada do fluxo principal, para o workflow secundário.

```
{{ $('Edit Fields').item.json.telefone }}
```

Description

Fixed

Expression

Nome da tool: transferirAtendimento

Descrição da tool: Utilize essa tool para transferir o atendimento para um humano quando solicitado pelo aluno ou quando você não souber responder uma pergunta.

Source

Database

Workflow

From list

Agente de IA - Asimov

Workflow Inputs

telefone

fx

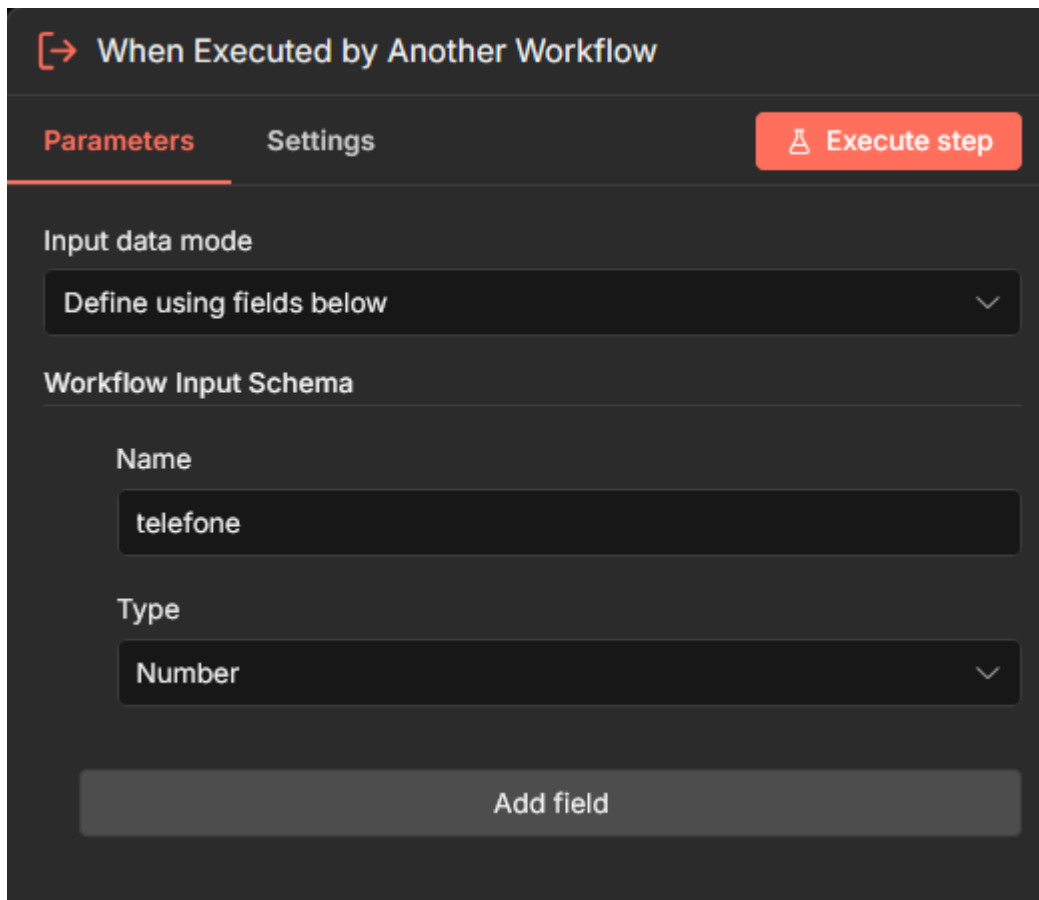
{{ \$('Edit Fields2').item.json.telefone }}

Add workflow input to send

O nome exato do node pode mudar no seu fluxo, mas a lógica é essa: enviar o telefone do contato atual.

Atualizando o status do contato no Supabase

Dentro do subfluxo de transferência, o primeiro passo é usar esse telefone para localizar o aluno na tabela de clientes e atualizar o status do atendimento.



[→] When Executed by Another Workflow

Parameters Settings Execute step

Input data mode

Define using fields below

Workflow Input Schema

Name

telefone

Type

Number

Add field

Objetivo dessa atualização

Ao mudar o campo status para algo como Transferido, o fluxo principal poderá verificar esse estado depois e impedir que a IA continue respondendo aquela pessoa.

Isso evita um problema importante: o humano e a IA responderem ao mesmo tempo.

Para fazer essa atualização, vamos usar o node do Supabase(**Update a row**) e fazer essa mudança:

Update a row Execute step

Parameters Settings Docs

Operation

Update

Table Name or ID
Clientes WhatsApp

Select Type
Build Manually

Must Match
Any Select Condition

Select Conditions

Field Name or ID
telefone - (number)

Condition
Equals

Field Value
{{ \$json.telefone }}

Add Condition

Data to Send
Define Below for Each Column

Fields to Send

Field Name or ID
status - (string)

Field Value
Transferido

Add Field

Lógica da transferência

A lógica fica assim:

1. receber o telefone do fluxo principal;
2. localizar o contato correspondente no Supabase;
3. atualizar o campo status;
4. marcar o atendimento como transferido.

Com isso, a automação passa a reconhecer que aquele aluno saiu do fluxo automático e entrou em atendimento humano.

Buscando o histórico da conversa

Depois de atualizar o status, o próximo passo é recuperar o histórico da conversa para enviar um resumo ao atendente humano. Isso evita que a pessoa precise ler tudo manualmente desde o início. Usaremos outro node do Supabase(**Get a row**) para recuperar o histórico através do `session_id`.

The screenshot shows the configuration for a 'Get many rows1' step. The interface is dark-themed with a light gray sidebar on the left. The main area has tabs for 'Parameters' (selected) and 'Settings'. A red 'Execute step' button is in the top right. The configuration fields are as follows:

- Credential to connect with:** A dropdown menu showing 'Supabase Asimov'.
- Use Custom Schema:** A toggle switch that is currently turned off.
- Resource:** A dropdown menu showing 'Row'.
- Operation:** A dropdown menu showing 'Get Many'.
- Table Name or ID:** A dropdown menu showing 'n8n_chat_histories'.
- Return All:** A toggle switch that is currently turned on (green).
- Filter:** A dropdown menu showing 'Build Manually'.
- Must Match:** A dropdown menu showing 'Any Filter'.
- Filters:**
 - Field Name or ID:** A dropdown menu showing 'session_id - (string)'.
 - Condition:** A dropdown menu showing 'Equals'.
 - Field Value:** A text input field containing the JSON path `/./ {{ $json.telefone }}`.

Tabela utilizada

A busca é feita na tabela de histórico de conversa, usando a chave de memória da sessão. Nesse projeto, essa chave corresponde ao telefone do usuário.

Ou seja:

- `sessionId` = telefone do aluno

Com esse filtro, o fluxo recupera todas as interações daquela conversa.

Preparando as mensagens para resumo

Depois de buscar o histórico, o ideal é reorganizar as mensagens em um formato simples, identificando quem enviou cada uma.

Uma estratégia usada é montar um campo como:

- `human: mensagem`
- `AI: mensagem`

Isso facilita a leitura pelo próximo agente, que será responsável por gerar o resumo.

Exemplo conceitual

human: Olá, tudo bem?

AI: Tudo bem. Antes de começarmos, qual é o seu nome?

human: Enzo

AI: Muito prazer, Enzo. Como posso ajudar?

Esse formato deixa o histórico mais claro para o agente de resumo.

Criando um agente para resumir a conversa

Com o histórico organizado, o próximo passo é usar mais um node de AI Agent para gerar um resumo da conversa.

A função desse agente é simples:

- ler o histórico;
- entender o que foi discutido;
- gerar um resumo direto ao ponto para o atendente humano.

Estrutura do prompt de resumo

O prompt pode ser bastante enxuto, com duas partes principais:

- **papel**
- **instruções**

Papel

Explicar que o agente recebeu um histórico entre humano e IA e precisa resumir o que foi conversado.

Instruções

Orientar que o resumo:

- não deve adicionar contexto extra;
- deve considerar os prefixos `'human'` e `'AI'`;
- deve incluir as informações importantes;
- deve ser direto ao ponto.

Prompt para resumir a conversa

Papel

Você recebeu um histórico de conversa entre um humano e um agente de IA. Seu trabalho é analisar a interação e gerar um resumo da conversa que será enviado diretamente à um atendente humano. Englobe os principais pontos da conversa, como interesse do aluno, dúvidas, nível atual e etc.

Instruções

- Não adicione contexto ou explicações, retorne apenas o histórico de conversa.
- As mensagens enviadas pelo humano iniciam com o prefixo "human", enquanto as mensagens enviadas pelo agente de IA contém o prefixo "ai".
- Englobe todas as informações importantes, porém mantenha o resumo direto ao ponto.

Esse tipo de prompt já é suficiente para produzir um resumo útil ao atendente.

Visão geral das tools criadas

Ao final deste capítulo, o agente passa a contar com três ferramentas complementares:

Buscar Curso

→ recomenda trilhas com base no perfil do aluno

Dados Asimov

→ consulta informações na Vector Store

Transferir atendimento

→ encerra o atendimento automático e prepara o contexto para o humano

Essa divisão deixa o agente principal mais leve e mais focado na conversa.

Resumo do que construímos

Neste capítulo, você criou as primeiras ferramentas do agente e começou a transformar o fluxo em uma estrutura mais completa.

A tool de recomendação de cursos permite delegar a escolha da trilha ideal a um subagente especializado. A Vector Store abre caminho para respostas baseadas em informações externas. E a tool de transferência resolve um ponto essencial do atendimento real: saber a hora de passar a conversa para um humano.

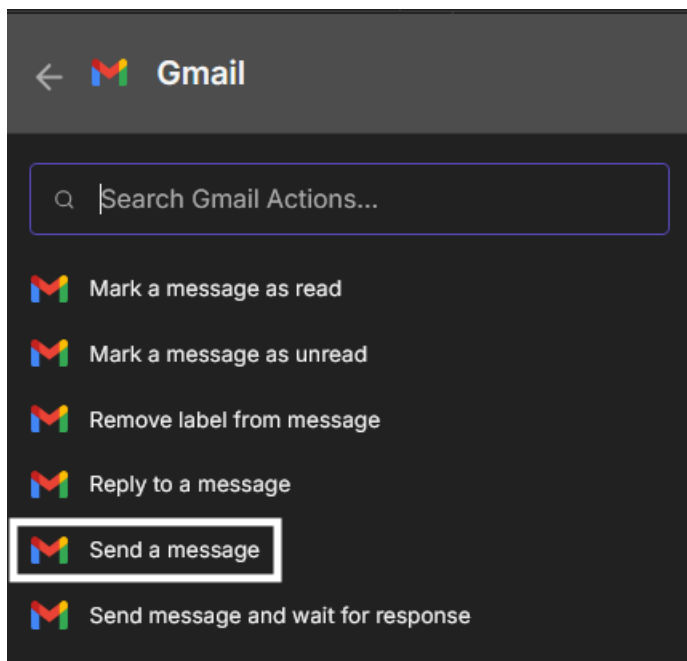
Com isso, o agente deixa de ser apenas um gerador de texto e passa a atuar com ferramentas reais dentro do fluxo.

13. Finalizando as tools e preparando a pipeline de RAG

Com as tools principais já criadas, o próximo passo é concluir dois pontos importantes do projeto: finalizar a tool de **transferência para atendimento humano** e montar a pipeline que envia conteúdos para a **Supabase Vector Store**. A partir daqui, o agente deixa de apenas conversar bem e passa a operar com uma base de conhecimento organizada, além de transferir atendimentos com mais contexto para uma pessoa da equipe.

Enviando a notificação de transferência por e-mail

Depois que o agente decide transferir o atendimento, faz sentido avisar imediatamente um atendente humano. Para isso, é possível conectar um node do Gmail no subfluxo de transferência e configurar o envio de uma mensagem para o e-mail responsável pelo atendimento.



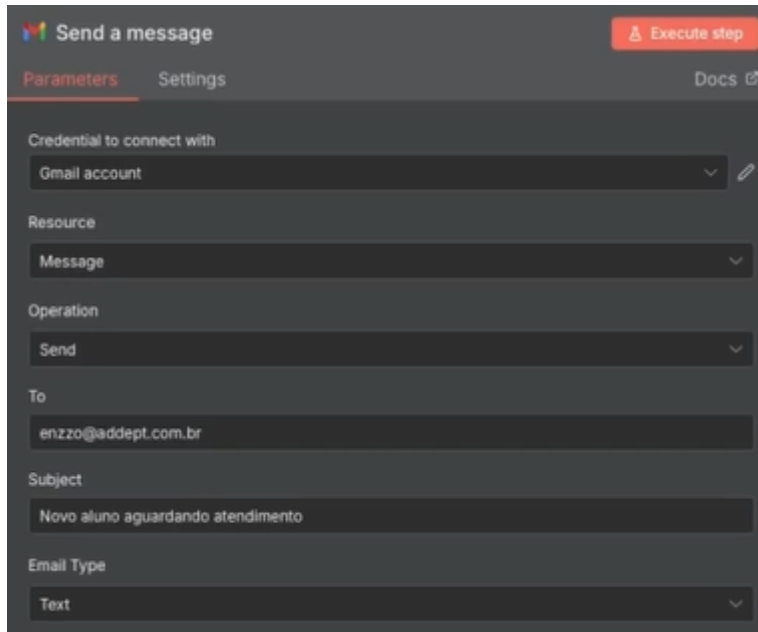
A configuração básica desse node inclui:

- credencial do Gmail;
- recurso de mensagem;
- operação de envio;
- e-mail de destino;
- assunto;

- corpo do e-mail em HTML.

O conteúdo dessa mensagem pode incluir:

- nome do aluno;
- telefone;
- resumo da conversa.



The screenshot shows a configuration window titled 'Send a message'. It has tabs for 'Parameters' (selected) and 'Settings', and a 'Docs' link. The configuration fields are as follows:

- Credential to connect with:** A dropdown menu set to 'Gmail account'.
- Resource:** A dropdown menu set to 'Message'.
- Operation:** A dropdown menu set to 'Send'.
- To:** A text field containing the email address 'enzzo@addept.com.br'.
- Subject:** A text field containing the subject line 'Novo aluno aguardando atendimento'.
- Email Type:** A dropdown menu set to 'Text'.

An 'Execute step' button is located in the top right corner of the configuration area.

Na prática, esse e-mail funciona como um aviso automático de que existe um novo contato aguardando continuidade no atendimento humano.

Montando o corpo do e-mail com variáveis

Para que esse e-mail seja útil, o ideal é montar o conteúdo usando variáveis vindas do fluxo. As principais são:

- nome do aluno;
- telefone do contato;
- resumo gerado pela IA.

```
Novo aluno aguardando atendimento via WhatsApp.
```

```
- Nome: {{ $('Update a row').item.json.nome }}  
- Telefone: {{ $('Update a row').item.json.telefone }}  
- Resumo da Conversa: {{ $json.output }}
```

Isso permite que o atendente receba um aviso já contextualizado, sem precisar buscar manualmente os dados básicos antes de começar a conversa.

Criando a pipeline de RAG para a Vector Store

Com as tools prontas, o projeto avança para a base de conhecimento. O objetivo agora é subir informações sobre cursos, trilhas ou outros conteúdos da empresa para dentro da **Supabase Vector Store**, formando o RAG que será usado pelo agente.

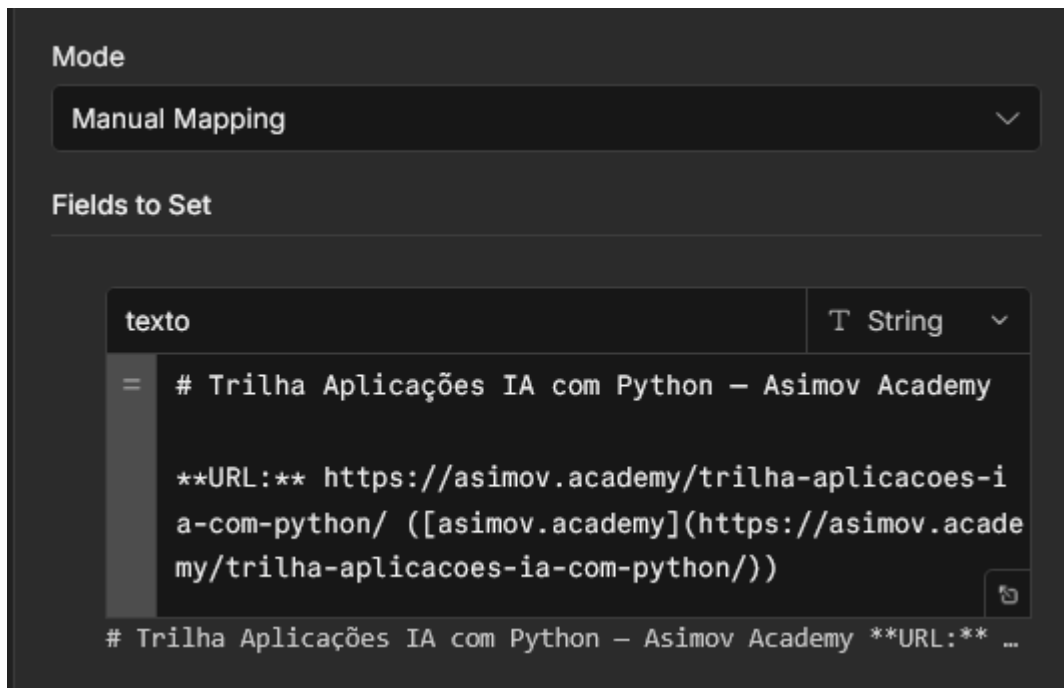
A primeira pipeline apresentada segue uma lógica manual e controlada:

1. usar um gatilho manual;
2. definir o conteúdo em um node Set;
3. dividir esse conteúdo em chunks;
4. vetorizar esses chunks;
5. subir tudo para a Vector Store.

Organizando o conteúdo antes de subir para o RAG

Antes de enviar qualquer dado para a Vector Store, é necessário organizar o texto que será usado como base de conhecimento. Uma forma prática de fazer isso é reunir, em uma única variável, todas as informações sobre cursos, trilhas, planos e ofertas da empresa.

Você pode usar o `Edit Fields (Set)` para organizar todo o conteúdo que vai ser enviado para a vector store.



Depois disso, o texto completo precisa ser dividido em partes menores, chamadas de **chunks**.

O que é um chunk?

Um chunk é uma parte menor de um texto maior. Em RAG, o conteúdo não costuma ser armazenado como um bloco único. Em vez disso, ele é quebrado em pedaços menores para facilitar a busca semântica.

Entendendo a lógica do RAG

O funcionamento do RAG pode ser resumido em três etapas:

1. dividir o texto em chunks;
2. transformar cada chunk em embeddings;
3. armazenar o texto e seus embeddings na Vector Store.

Embeddings, na prática

Embeddings são vetores numéricos que representam o significado semântico de um texto. Em vez de buscar apenas por palavras exatas, a Vector Store passa a buscar por similaridade de sentido.

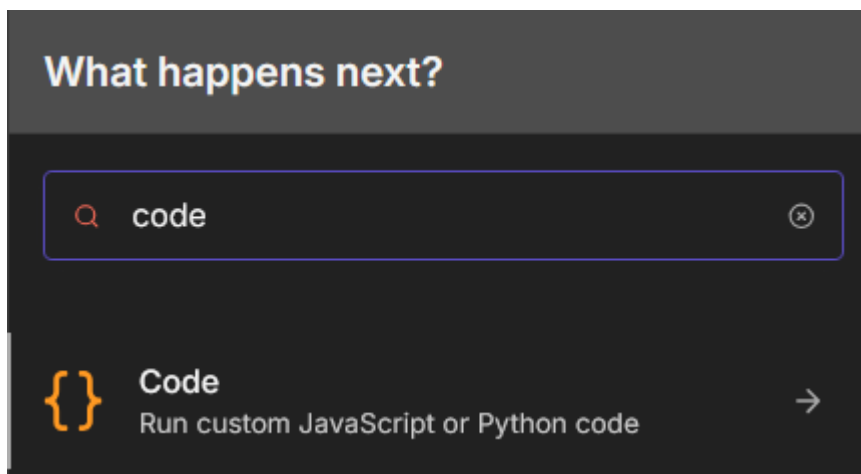
Isso é o que permite ao agente encontrar informações relevantes mesmo quando a pergunta do usuário não repete exatamente as mesmas palavras da base.

Fazendo um chunk por trilha ou curso

Neste projeto, a opção escolhida foi criar **um chunk por trilha** ou **um chunk por curso**. Isso significa que cada pedaço enviado para a Vector Store contém todas as informações relevantes sobre uma única oferta da empresa.

Essa decisão tem uma vantagem importante: quando o agente consulta o RAG sobre uma trilha específica, ele tende a receber um bloco completo e coerente sobre aquele produto, em vez de fragmentos soltos ou partes cortadas no meio de uma explicação.

No n8n existe um node chamado Code. Com ele é possível executar scripts em JavaScript ou Python diretamente no nosso fluxo. Será ele que usaremos para separar o texto em chunks.



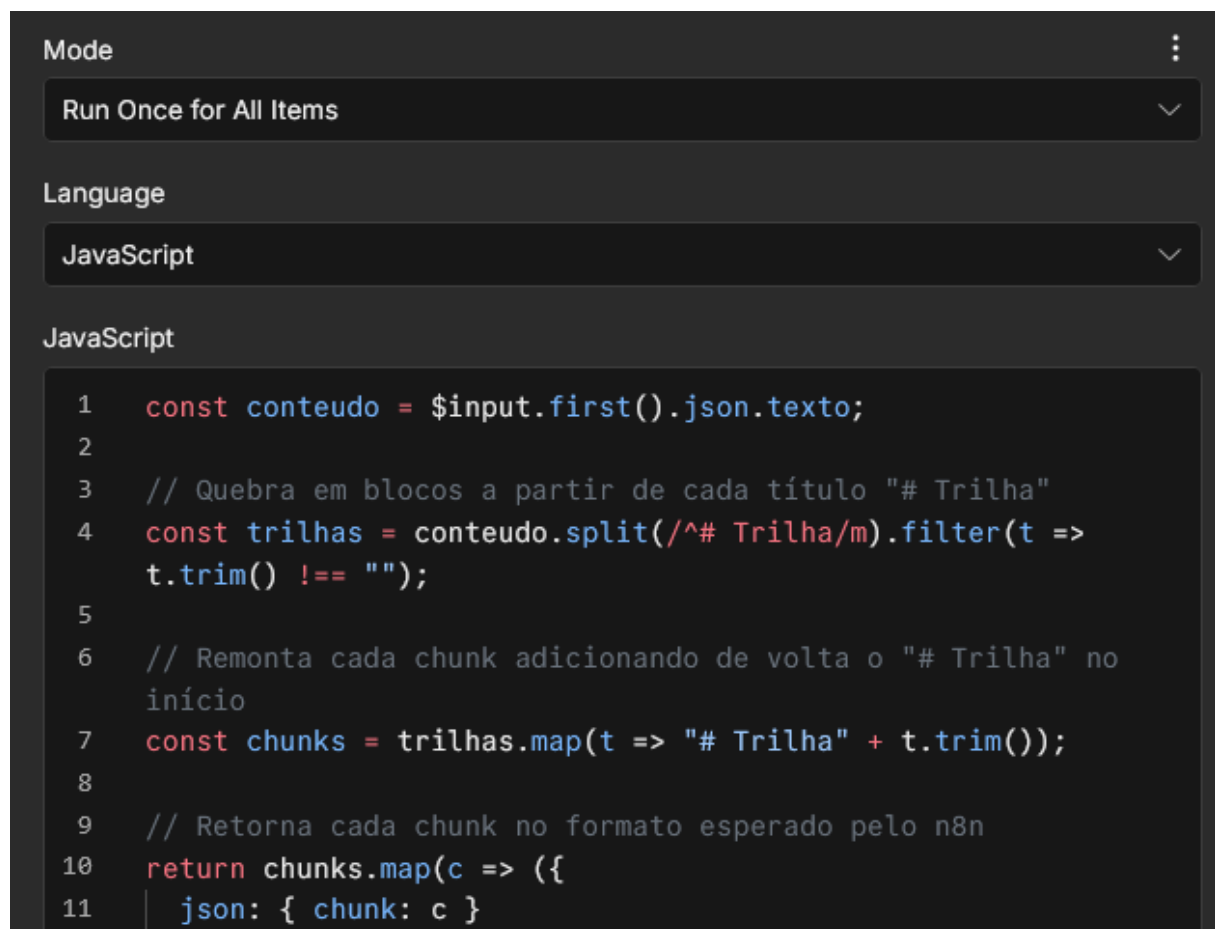
Script para separar texto em chunks

```
const conteudo = $input.first().json.texto;

// Quebra em blocos a partir de cada título "# Trilha"
const trilhas = conteudo.split(/^# Trilha/m).filter(t => t.trim() !== "");

// Remonta cada chunk adicionando de volta o "# Trilha" no início
const chunks = trilhas.map(t => "# Trilha" + t.trim());

// Retorna cada chunk no formato esperado pelo n8n
return chunks.map(c => ({
  json: { chunk: c }
}));
```



The screenshot shows a code editor with a dark theme. At the top, there's a 'Mode' dropdown set to 'Run Once for All Items' and a 'Language' dropdown set to 'JavaScript'. Below these, the code is as follows:

```
1  const conteudo = $input.first().json.texto;
2
3  // Quebra em blocos a partir de cada título "# Trilha"
4  const trilhas = conteudo.split(/^# Trilha/m).filter(t =>
   t.trim() !== "");
5
6  // Remonta cada chunk adicionando de volta o "# Trilha" no
   início
7  const chunks = trilhas.map(t => "# Trilha" + t.trim());
8
9  // Retorna cada chunk no formato esperado pelo n8n
10 return chunks.map(c => ({
11   | json: { chunk: c }
```

Por que isso funciona bem neste caso?

Porque os textos sobre cada trilha ainda são relativamente curtos. Quando o conteúdo não é muito grande, manter tudo de uma trilha em um único chunk pode deixar o RAG mais preciso e mais simples de gerenciar.

Em conteúdos muito extensos, essa estratégia já não seria a ideal, porque chunks grandes demais podem aumentar custo, latência e perda de eficiência.

Subindo os chunks um por vez

Depois de dividir o conteúdo em chunks, o próximo passo é usar um `node Loop Over Items` para enviar cada chunk individualmente para a Supabase Vector Store.

Dentro desse loop, o node principal é o **Supabase Vector Store** com a ação **Add Documents to Vector Store**.

Na configuração, devem ser definidos:

- a tabela documents;
- a propriedade de texto;
- o modelo de embeddings;
- o conteúdo do chunk atual.

The screenshot shows a configuration interface with the following sections:

- Credential to connect with:** A dropdown menu set to "Supabase account" with an edit icon to its right.
- Operation Mode:** A dropdown menu set to "Insert Documents".
- Table Name:** A section with a "From list" dropdown and a text input field containing "documents".
- Embedding Batch Size:** A text input field containing the number "200".
- Options:** A section containing a "Query Name" text input field with the value "match_documents" and an "Add Option" button below it.

Mantendo o mesmo modelo de embeddings

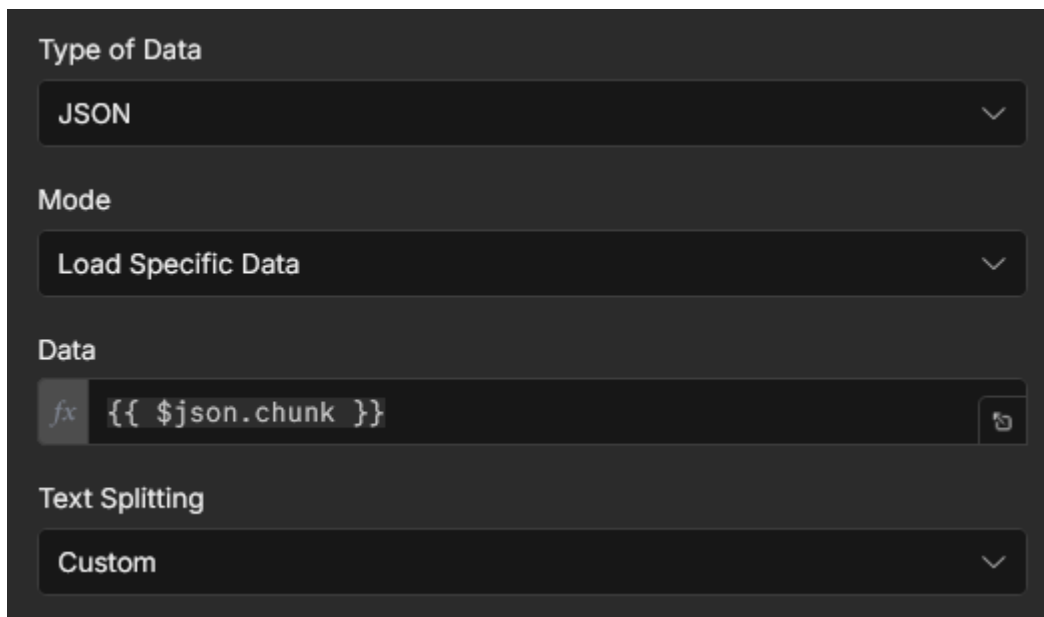
Esse ponto merece atenção especial: o modelo de embeddings usado para **subir** os dados deve ser o mesmo usado depois para **consultar** o RAG.

Se o projeto usar, por exemplo **text-embedding-3-small**, na hora de enviar os documentos para a Vector Store, esse mesmo modelo deve ser usado na hora da busca.

Configurando o carregamento do documento

Na etapa de envio do chunk para a Vector Store, o node pode usar um Default Data Loader, com o modo configurado para carregar dados específicos. Nesse caso, o campo principal recebe a variável que contém o chunk atual.

Também é possível definir o tipo de divisão de texto como **custom**, usando um modo recursivo e um chunk size alto, justamente para evitar que o conteúdo de uma trilha seja quebrado em partes menores sem necessidade.



The image shows a configuration panel with a dark theme. It contains four sections, each with a title and a dropdown menu or text input:

- Type of Data**: A dropdown menu with 'JSON' selected.
- Mode**: A dropdown menu with 'Load Specific Data' selected.
- Data**: A text input field with a 'fx' icon on the left and a copy icon on the right. The text inside is `{{ $json.chunk }}`.
- Text Splitting**: A dropdown menu with 'Custom' selected.

Essa escolha reforça a ideia de preservar a integridade das informações de cada trilha em um único bloco.

Resultado esperado na Vector Store

Ao final desse processo, a tabela documents do Supabase passa a ter uma linha para cada chunk enviado. Neste caso, o esperado é ter um chunk exclusivo para cada trilha ou curso.

Cada linha armazenada contém, em essência:

- o texto da trilha;
- os metadados;
- o vetor de embeddings correspondente.

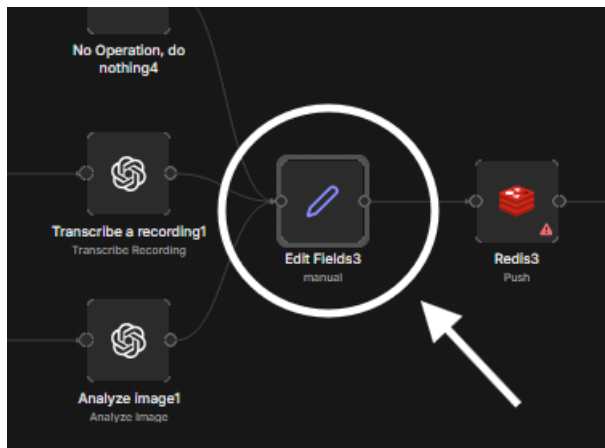
Com isso, o RAG já fica pronto para ser usado pelo agente.

Ajustando o fluxo principal antes dos testes

Antes de começar os testes do agente, ainda é importante fazer alguns ajustes no fluxo principal.

Corrigindo a referência da mensagem de texto

Em alguns casos, a variável de mensagem de texto pode deixar de funcionar corretamente depois que novos nodes são adicionados ao fluxo.

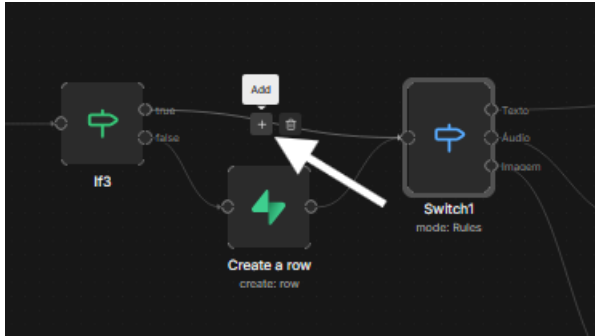


Quando isso acontece, o ajuste necessário é referenciar explicitamente o node que contém a variável `mensagem_texto`, em vez de depender apenas de um `json` genérico.

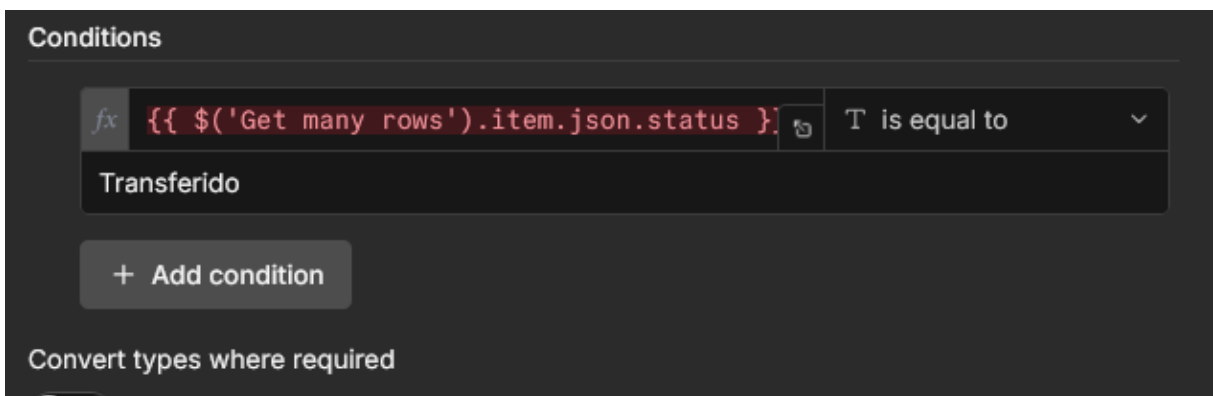
Esse tipo de correção é simples, mas importante, porque garante que mensagens de texto continuem sendo tratadas corretamente depois que o fluxo cresce.

Bloqueando contatos transferidos

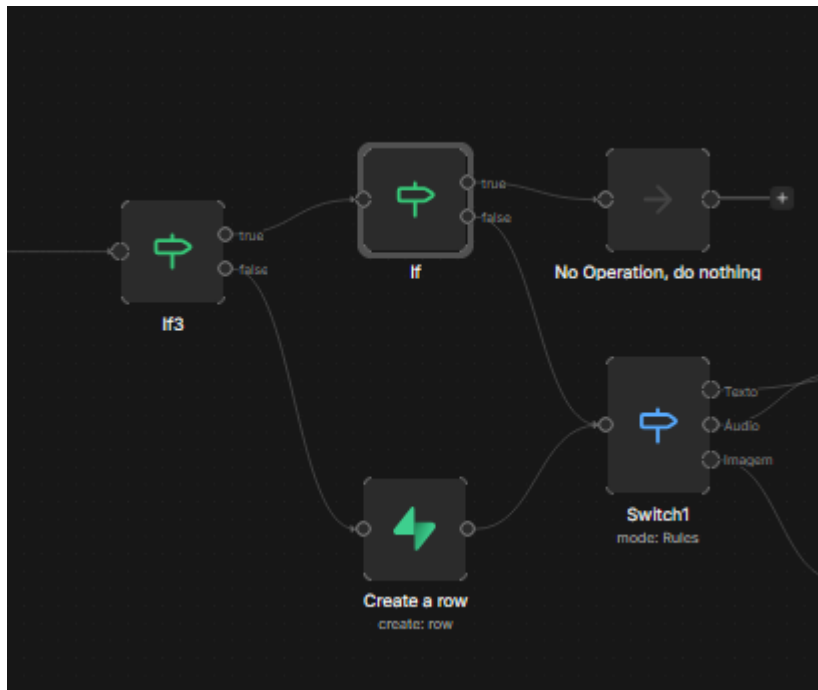
Outro ajuste importante é adicionar uma verificação após a busca do lead no Supabase para saber se o campo status está como Transferido



Se estiver, o fluxo deve parar. Se não estiver, ele segue normalmente para o Switch e para o restante da automação.



Essa validação evita que a IA continue respondendo alguém que já foi encaminhado para atendimento humano.



Limpando a memória antes dos testes

Antes de iniciar uma bateria de testes, vale limpar o histórico salvo nas tabelas de memória, como `n8n_chat_histories`, além dos dados temporários de contatos de teste.

Isso ajuda porque permite testar o agente como se fosse a primeira conversa, sem interferência de contextos antigos.

Validando o comportamento do agente

Depois desses ajustes, o fluxo pode ser testado com mensagens reais.

Teste com texto

Ao receber uma mensagem como “boa noite”, o agente deve:

- criar o contato no banco, se ele ainda não existir;
- registrar a mensagem;
- passar pelo Redis;
- chamar o agente principal;

- gerar uma resposta coerente com o prompt.

Teste com áudio

No caso de áudio, o fluxo deve:

- buscar a mídia na Meta;
- baixar o arquivo;
- transcrever;
- tratar o texto da mesma forma que uma mensagem comum.

Teste com buffer

Quando o usuário manda duas mensagens em sequência, como “sim” e “por favor”, o Redis deve impedir que a primeira execução responda isoladamente. Em vez disso, a automação deve esperar a mensagem mais recente e agrupar tudo antes de chamar o agente.

Esse comportamento confirma que o buffer está funcionando corretamente.

Validando a consulta ao RAG

Depois de o agente estar operacional, também vale testar perguntas que dependem da base vetorial, como preço ou detalhes de uma trilha específica. Nesse caso, a tool de RAG consulta a Vector Store usando uma query relacionada ao tema da pergunta.

Um ponto importante observado nessa etapa é o limite de chunks retornados. Se cada trilha já está inteira em um único chunk, não faz sentido trazer muitos resultados irrelevantes. Por isso, reduzir esse limite pode diminuir o gasto de tokens e deixar a resposta mais eficiente.

Ao mesmo tempo, manter pelo menos dois resultados pode ser uma margem de segurança caso o chunk mais relevante não venha em primeiro lugar.

Criando uma segunda pipeline de RAG com arquivos

Além da pipeline manual, também é possível montar uma segunda pipeline para enviar conteúdos a partir de arquivos, como PDFs ou .docx.

A lógica dessa segunda abordagem é:

1. baixar o arquivo do Google Drive;
2. extrair o texto;
3. enviar esse texto para a Vector Store.

Etapas principais

- usar o node `Google Drive` com ação de download;
- usar um node `Extract from File`;
- extrair o conteúdo do PDF;
- enviar o texto para o node `Supabase Vector Store`.

Nesse caso, o próprio node pode usar uma configuração padrão de chunking, como blocos de 1000 caracteres com overlap de 200.

Essa abordagem é útil quando a base de conhecimento vem de documentos prontos, e não de texto digitado manualmente no fluxo.

Conclusão

Neste capítulo, você consolidou a parte mais operacional do projeto. A tool de transferência passou a funcionar de ponta a ponta, inclusive com resumo da conversa e aviso por e-mail. Ao mesmo tempo, a base vetorial foi preparada para armazenar informações de cursos, trilhas e documentos externos.

Com isso, a arquitetura do agente já está pronta para uso real e para refinamentos futuros. A partir daqui, os principais ajustes tendem a acontecer mais no comportamento do agente, como prompt, tom, regras e precisão das tools, do que na estrutura central do sistema.

14. Encerramento e próximos passos

Ao longo desta apostila, você construiu uma solução completa de atendimento com a API Oficial do WhatsApp e o n8n. Mais do que configurar uma integração, você montou uma arquitetura prática de automação com IA, conectando infraestrutura, fluxo, banco de dados, memória, tools, RAG e atendimento humano quando necessário.

O que foi construído ao longo da jornada

Durante o projeto, você passou por todas as etapas essenciais para tirar uma solução desse tipo do papel:

- preparou a estrutura empresarial necessária dentro da Meta;
- criou e configurou o aplicativo da API Oficial do WhatsApp;
- registrou e validou o número usado na integração;
- conectou o webhook da Meta ao n8n;
- configurou o envio e o recebimento de mensagens;
- montou a base técnica do agente;
- conectou banco de dados, memória, Redis e modelo de IA;
- estruturou o prompt do agente;
- criou tools para recomendação, consulta e transferência;
- preparou uma base RAG com Supabase Vector Store;
- testou o fluxo completo e ajustou o comportamento do sistema.

Em conjunto, essas etapas formam uma base sólida para criar agentes de atendimento reais no WhatsApp.

O que esse conhecimento permite fazer

Com a estrutura apresentada nesta apostila, você já tem conhecimento suficiente para começar a desenvolver projetos parecidos em contextos diferentes. A mesma lógica pode ser aplicada, por exemplo, em agentes voltados para:

- suporte ao cliente;
- pré-atendimento comercial;

- qualificação de leads;
- envio de informações automatizadas;
- atendimento interno de empresas;
- experiências híbridas entre IA e atendimento humano.

Ou seja, o projeto construído aqui não deve ser visto apenas como um exemplo isolado. Ele funciona como uma base reutilizável para diferentes aplicações profissionais.

Como evoluir a partir daqui

Depois de concluir a implementação principal, o caminho natural é continuar refinando o sistema. Alguns avanços possíveis são:

- melhorar o prompt do agente com base em conversas reais;
- expandir as tools para novas ações;
- enriquecer a base RAG com mais documentos;
- criar fluxos de qualificação mais sofisticados;
- integrar CRMs, agendas, e-mails ou sistemas internos;
- adaptar o agente para outros tipos de negócio.

Esse tipo de evolução é o que transforma uma automação funcional em uma solução cada vez mais útil e valiosa para uma empresa.

Uma base para aplicação prática e comercial

Além do aprendizado técnico, este projeto também abre espaço para aplicação profissional. A estrutura desenvolvida aqui já oferece base suficiente para você:

- implementar esse tipo de solução na própria empresa;
- usar o projeto como treino e portfólio;
- adaptar o modelo para clientes;
- transformar esse conhecimento em serviço ou produto.

Esse é um dos pontos mais importantes de toda a jornada: o que foi construído não serve apenas para estudo. Serve também como ponto de partida para projetos reais.

Conclusão

Chegando ao fim deste material, o mais importante é perceber que você não aprendeu apenas a conectar ferramentas. Você aprendeu a estruturar um agente de IA com aplicação prática no WhatsApp, combinando automação, contexto, memória, atendimento e integração com serviços externos.

Essa base já é suficiente para explorar novos projetos, adaptar a solução para outros cenários e continuar evoluindo como profissional na criação de automações com IA.

Parabéns por chegar até aqui!